Ali Ahammad

Optimization of Data Transmission in IoT Applications using CoAP and MQTT protocols

UNIVERSIDADE DO ALGARVE

Faculdade de Ciências e Tecnologia

2025.

Ali Ahammad

Optimization of Data Transmission in IoT Applications using CoAP and MQTT protocols

Master in Informatics Engineering

Trabalho efetuado sob a orientação de:

Profª Doutora Noélia Correia

UNIVERSIDADE DO ALGARVE

Faculdade de Ciências e Tecnologia

2025

Optimization of Data Transmission in IoT Applications using CoAP and MQTT protocols

Declaração de autoria de trabalho

Declaro ser o(a) autor(a) deste trabalho, o qual é original e inédito. Os autores e trabalhos consultados estão devidamente citados no texto e constam na lista de referências incluída.

_____

Ali Ahammad

Copyright © 2025. Todos os direitos reservados em nome de Ali Ahammad. A Universidade do Algarve tem o direito, perpétuo e sem limites geográficos, de arquivar e publicitar este trabalho através de exemplares impressos reproduzidos em papel ou de forma digital, ou por qualquer outro meio conhecido ou que venha a ser inventado, de o divulgar através de repositórios científicos e de admitir a sua cópia e distribuição com objetivos educacionais ou de investigação, não comerciais, desde que seja dado crédito ao autor e editor.

Acknowledgments

I would like to express my deepest gratitude to my supervisor, whose expertise, understanding, and patience, added considerably to my graduate experience. The mentorship was paramount in providing a well-rounded experience consistent with my long-term career goals.

I would also like to thank the members of the interim presentation jury for their insightful comments and suggestions, which have greatly enriched this work.

To my family, thank you for always encouraging my pursuits and for providing unconditional love and support throughout my life. Your belief in me has always been a source of strength.

This accomplishment would not have been possible without you all. Thank you.

Abstract

In this work, the efficiency and reliability of resource-constrained *Internet of Things* (IoT) communication systems are explored. Such systems, integrating *Constrained Application Protocol* (CoAP) and *Message Queuing Telemetry Transport* (MQTT) protocols, are the consideration of this dissertation. The integration of both protocols is advantageous because of CoAP's lightweight, *User Datagram Protocol* (UDP)-based communication and MQTT's reliable, *Transmission Control Protocol* (TCP)-type communication system. Efficiency and reliability in such architectures is achieved through adjustment of network parameters such as frequency of packet transmission and MQTT QoS to actual sensor, network conditions and for optimized power management. A *Double Deep Q-Network* (DDQN) agent is developed and evaluated in both simulated environments and online interaction with the IoT pipeline. Its goal is to reduce energy consumption while maintaining data fidelity and increasing the network's lifespan. The reward function considers energy consumption, packet loss probability, end-to-end one-way packet delay, and sensor event criticality. The utility-based weights are adjusted for different operating scenarios. The system uses a realistic simulation environment and a set of Docker-based services. This implementation enables evaluation of the learned policies. Sensor telemetry and control actions are sent using MQTT and stored in Influx DB for subsequent visualization. Experimental results will be shown with episodic learning curves, accuracy, and picking up the right actions. Visualization will then highlight the agent's ability to control network management and optimize network longevity.

Keywords

Internet of Things (IoT), Constrained Application Protocol (CoAP), Message Queuing Telemetry Transport (MQTT), User Datagram Protocol (UDP), Quality of Service (QoS), Double Deep Q Network (DDQN), Reinforcement Learning (RL), Docker, Influx DB, Grafana.

Resumo

Nesta dissertação, exploram-se a eficiência e a fiabilidade de sistemas de comunicação *Internet das Coisas* (IoT) com recursos restritos. O estudo centra-se em sistemas que integram os protocolos *Constrained Application Protocol* (CoAP) e *Message Queuing Telemetry Transport* (MQTT). A integração de ambos os protocolos revela-se vantajosa devido à natureza leve da comunicação baseada em *User Datagram Protocol* (UDP) do CoAP, conjugada com o sistema de comunicação fiável do MQTT, baseado em *Transmission Control Protocol* (TCP). A eficiência e a fiabilidade nestas arquiteturas são alcançadas através do ajuste de parâmetros de rede — tais como a frequência de transmissão e a *Qualidade de Serviço* (QoS) do MQTT — às condições reais dos sensores e da rede, visando uma gestão de energia otimizada. Desenvolveu-se e avaliou-se um agente *Double Deep Q-Network* (DDQN), tanto em ambientes simulados como na interação em tempo real com a pipeline IoT. O objetivo consiste na redução do consumo energético, mantendo a fidelidade dos dados e aumentando a longevidade da rede. A função de recompensa considera o consumo de energia, a probabilidade de perda de pacotes, o atraso unidirecional ponta-a-ponta e a criticidade dos eventos dos sensores, sendo o ajuste dos pesos baseados na utilidade para diferentes cenários operacionais. O sistema utiliza um ambiente de simulação realista e um conjunto de serviços baseados em Docker, implementação esta que possibilita a avaliação das políticas aprendidas. A telemetria dos sensores e as ações de controlo são transmitidas via MQTT e armazenadas na Influx DB para visualização posterior. Apresentam-se resultados experimentais que incluem curvas de aprendizagem episódicas, precisão e a seleção adequada de ações. A visualização destaca a capacidade do agente no controlo da gestão de rede e na otimização da sua longevidade.

Palavras Chave

Internet das Coisas (IoT), Protocolo de Aplicação Restrita (CoAP), Transporte de Telemetria de Enfileiramento de Mensagens (MQTT), Protocolo de Datagrama de Utilizador (UDP), Qualidade de Serviço (QoS), Aprendizagem Profunda Q Duplo (DDQN), Aprendizagem por Reforço (RL), Docker, Influx DB, Grafana.

Contents

1. Introduction 1
1.1 Motivation 3
2. IoT Transfer Protocols 5
2.1 CoAP—MQTT Design Pattern 9
2.2 Related Work 11
2.2.1 Architectural Patterns for Protocol Interworking 12
2.2.2 Deep RL for Transfer-Level Control in Constrained Networks 12
2.2.3 Adaptive/Exception-Based Sampling 12
2.2.4 MQTT QoS Trade-Offs: Reliability, Latency, Energy 13
2.2.5 Criticality and PASA-Inspired Context Metrics 14
2.2.6 Reward Shaping and Algorithmic Enhancements 15
2.2.7 Action Masking and Safety Constraints 16
2.2.8 Simulating Realistic Conditions: Contextualizing the Problem 17
2.2.9 Algorithmic Enhancements for Stability and Efficiency 17
3. Problem Formulation (MDP) 20
3.1 Adaptive Control Problem 22
3.1.2 Critical Events and Class Imbalance 22
3.2 MDP Definition 24
3.2.1 State Space 25
3.2.2 Action Space 26
3.2.3 Reward Function 27
3.3 The PASA Principle and Design Rationale 34
3.4 Safety Constraints 36
3.5 Transition Assumptions and Simulator Abstractions 38
4. RL-based Framework 41
4.1 System Architecture Overview 42
4.2 State Encoding and Feature Pipeline 45
4.3 Action Space Realization 46
4.4 Practical Example: One Decision Step from Signal to Reward 48
4.4.1 Setup of the Example 48
4.4.2 Backlog Update (Step 1) 49

4.4.3 Energy per Step (Step 2) 51
4.4.4 Delay, Loss, and On-Time Delivery (Step 3) 54
4.5 Reward Computation in Using the Example 57
4.5.1 Utilities (What is Rewarded) 57
4.5.2 Costs and Shaping (What is Penalized) 59
4.6 Safety Enforcement in Runtime (using the Same State) 61
4.6.1 Hard Masks 61
4.6.2 Soft Gating with Criticality Probability 62
4.6.3 Non-Bypassable Penalties 64
4.7 Action Selection and ε-Greedy Choice 64
4.7.1 Q-values Before and After Gating 64
4.7.2 ε-greedy Exploration 65
4.8 Training Loop and Losses (Offline and Online) 66
4.8.1 Offline Training Loop 67
4.8.2 Losses on the Example Transition 68
4.8.3 Network Impairment Model 70
4.9 One-Glance Summary Table 70
4.10 Behavioural Summary 71
5. Evaluation and Results 73
5.1 Experimental Setup 74
5.2 Baselines (Static QoS/Interval, Heuristic) 75
5.3 Scenarios (Cases) Table 76
5.4 KPIs and Evaluation Method 78
5.5 KPIs for Offline Results (Sim) 79
5.6 KPIs for Online Results (Testbed) 87
5.7 Comparison Offline vs Online 90
5.7.1 Robustness Tests: Removing Safety Features and Variants 91
5.7.2 Sensitivity: Weights, Multipliers, and Thresholds 93
6. Conclusion 95
Appendix A — Algorithms and Pseudocode 103
Appendix B —Policy Probes 107

List of Figures

Figure 2. 1 CoAP—MQTT architecture……..10

Figure 2. 2 The agent-environment interaction in MDP……27

Figure 4. 1 Hybrid CoAP–MQTT system architecture with data plane, control plane, and RL (MDP) loop……...43

Figure 5. 1 Energy consumption per episode ……80

Figure 5. 2 Average delivery latency ……82

Figure 5. 3 Reward vs Energy scatter plot ……82

Figure 5. 4 Packet reliability metrics per episode ……83

Figure 5. 5 Critical classification accuracy (proxy) …….84

Figure 5. 6 Final action distribution (QoS x interval) ……..85

List of Tables

Table 2. 1 Comparison between different architectures including transfer protocols used in IoT …….8

Table 2. 2 Selected studies applying RL in IoT communication systems ...……19

Table 3. 1 Empirical prevalence: critical vs non-critical steps and count ….……...23

Table 4.5. 1 Overall reward for each action ...…….60

Table 4.9. 1 Entire pipeline parameters based on each example actions ...……71

Table 5. 1 Online vs Offline scenario comparison ...……...78

Table 5.5. 1 “probe-state” mini-table (for Appendix B cross-reference) ..……...86

Glossaries

AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
CoAP	Constrained Application Protocol
DDQN	Double Deep Q-Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DTLS	Datagram TLS
EMA	Exponential Moving Average
HTTP	Hypertext Transfer Protocol
IoT	Internet Of Things
IPv6	Internet Protocol version 6
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LoWPAN	Low-Power Wireless Personal Area Networks
MAC	Media Access Control
MDP	Markov Decision Process
MOMDP	Multi-Objective Markov Decision Process
MORL	Multi-Objective Reinforcement Learning
MQTT	Message Queuing Telemetry Transport
OSCORE	Object Security for Constrained RESTful Environment
PASA	Priority-based Adaptive Scheduling Algorithm
PER	Prioritized Experience Replay
QoS	Quality of Service

REST	Representational State Transfer
RL	Reinforcement Learning
RPL	Routing Protocol for Low power and Lossy Networks
RTT	Round Trip Time
SDN	Software Defined Networking
TCP	Transmission Control Protocol
TD	Temporal Difference
TLS	Transport Layer Security
UDP	User Datagram Protocol
WAN	Wide Area Network
XMPP	Extensible Messaging and Presence Protocol

List of Symbols

Problem Formulation (MDP)

$\hat{p}_{\text{crit}}$	Estimated proportion of steps that are critical.
N_{crit}	Number of time steps that are labelled as critical.
N	Total number of time steps observed.
$\mathcal{M}$	The full MDP tuple.
$\mathcal{S}$	Set of all possible states the system can be in.
$\mathcal{A}$	Set of all actions the agent is allowed to choose.
$\mathcal{P}$	Rule that gives the probability of the next state, given the current state and action.
$\mathcal{R}$	Rule that gives the reward for a transition (state, action, next state).
γ	The discount factor that controls how much future rewards matter.
s_t	State of the system at discrete time step t.
a_t	State of the system at decision step t.
s	A generic State.
a	A generic action.
s'	Next state reached after applying action a in state s.
P	A single transition probability value.
R_t	Scalar reward at step t.
G_t	Discounted return from step t, that is the sum of future rewards with discount γ.
k	Summation index used inside the definition of G_t.
t	Time index (step number in the episode).
ΔT	Rate of change bin in sensor data (0/1/2).
$anom$	Anomaly flag (0 or 1).
$bufT$	Backlog level bin (0/1/2).
L	Packet loss bin (0/1/2).
D	Delay bin (0/1/2).
B	Battery level bin (0/1/2).
I_{cur}	The current reporting-interval level used in the state (0/1/2 for $\tau = \{5, 30, 60\}\, s$).

Q_{cur}	The current MQTT QoS level bin (0/1/2).
F_{cur}	The current payload fidelity level bin (0/1/2).
q	MQTT QoS level chosen by the agent (0/1/2).
τ	Reporting interval between sensor transmissions (5, 30, 60 s).
f	The payload fidelity (0/1/2).
c	The critical flag (0 calm, 1 critical).
U_{info}	Information utility; measures how much useful information the chosen message delivers, given demand.
w_{I}	Weight that controls how important information utility is in the total reward.
U_{time}	Timeliness utility; measures how fresh the delivered data is (short intervals and low delay give higher value).
R_{idle}	Idle bonus; small positive reward given in calm states when the node uses the most economical settings.
w_{T}	Weight that controls how important timeliness is in the total reward.
U_{rel}	Reliability utility; measures how well the chosen QoS matches the current link quality (loss and delay).
κ	Size of the safety penalty used when the agent is clearly too "lazy" in an urgent situation.
$error\ (q, L, D)$	A Penalty term that grows when the chosen QoS q is too low for the observed loss L and delay D.
Δ_{QoS}	Normalized QoS mismatch between chosen QoS and the QoS level that the link actually requires.
$link_sev(s')$	Link severity score in [0,1], computed from the loss and delay bins in the next state s'.
q^*	QoS level that the model believes is required, based on current link severity.
w_{R}	Weight that controls how important reliability utility is in the total reward.
U_{crit}	Critical-handling utility; rewards correct use of the critical flag and penalizes missed or false critical.
σ	Severity score used inside U_{crit}; larger when change, backlog, or anomaly are more serious.

C_{E}	Energy cost; larger when the action consumes more battery in this step.
e_{ref}	Reference energy level used to normalize C_{E} (represents "high drain" per step).
μ_{E}	Context multiplier for energy cost; becomes larger when the battery B is low, so energy matters more.
C_{X}	Network strain cost; larger when large payloads are sent over a stressed link (high loss/delay).
μ_{X}	Context multiplier for network strain; increases the cost when the link is already in a bad state.
C_{backlog}	Backlog cost; penalty related to how full the queue is and how slowly it is being drained.
ρ	Parameter that sets the size of the idle bonus given in calm, frugal states.
$safety_pen$	Additional safety penalty applied when the situation is urgent, but the chosen action is clearly too slow or too low fidelity.
Ω	Generic "hard violation" penalty; takes the value 0, $\Omega_{\mathrm{overflow}}$ or $\Omega_{\mathrm{miss_crit}}$.
$\Omega_{\mathrm{overflow}}$	Large negative penalty applied when the send buffer overflow.
$\Omega_{\mathrm{miss_crit}}$	Large negative penalty applied when a truly critical event is not treated as critical.
r	Immediate (possibly scaled/clipped) reward, often used inside code; related to R_t.
$\mathcal{G}$	Gating operator that restricts the feasible action set based on $p_{\mathrm{crit}}(s)$ and thresholds.
$\mathcal{A}_{\mathrm{feas}}$	Set of feasible actions after applying hard masks (first safety layer).
θ_{lo}	Lower criticality threshold for gating; below this, the gate prefers non-critical actions ($c = 0$).
θ_{hi}	Upper criticality threshold for gating; above this, the gate prefers critical actions ($c = 1$).
$\mathcal{A}_{\mathrm{allow}}$	Final set of actions that are allowed after both masking and gating.
a_{safe}	Pre-defined safe fallback action used if $\mathcal{A}_{\mathrm{allow}}$ is empty.

RL-based Framework

C	Backlog capacity: maximum queue size of the bridge/simulator (maximum number of buffered items in the backlog before overflow).
B_{cap}	Battery capacity of the virtual device.
$burst_t$	Burst indicator at step t.
κ_{F}	Backlog-drain scaling factor that converts the chosen interval, fidelity, and burst/critical mode into an effective drain capacity in the backlog update.
d_{cap}	Effective backlog drain capacity in one decision interval: approximate number of backlog units that can be served (removed from the queue) at step t.
I_{tx}	Transmit current during radio transmission (mA), obtained from a base transmit current scaled by QoS overhead and duty factor.
I_{avg}	Average current (mA) over one decision interval, including base idle consumption, sensing, and transmit current.
$\chi(.)$	Indicator function: equals 1 when the condition inside the parentheses is true and 0 otherwise (used, for example, to switch burst behaviour on/off).
u_t	The duty factor at step t.
ϕ_f	Fidelity multiplier relating the discrete fidelity level f to relative payload size, backlog increase, and transmission time (larger for higher fidelity).
$t_{\text{tx}}^{\text{base}}$	Baseline transmission time per packet at reference fidelity and QoS (seconds per packet), used as the starting point for computing duty factor and energy.
$I_{\text{tx}}^{\text{base}}$	Baseline transmit current (mA) for a low-fidelity packet at reference QoS, before QoS overhead and duty-cycle scaling.
ω_q	QoS overhead factor (> 1 for higher QoS levels) that increases the effective transmit current or airtime to reflect extra acknowledgments and protocol overhead.
I_{base}	Base current (mA) drawn when the device is powered and idle, excluding sensing and radio transmission.

I_{sense}	Current used for sensing and CPU work per interval (mA), excluding the radio transmit phase.
$I_{\text{avg}}^{\text{k}}$	Nonlinear effective current used in the Peukert-style battery model: the average current raised to exponent $k > 1$ to model accelerated drain at higher loads.
ΔmAh_t	Battery charge consumed in decision step t, expressed in mAh (derived from $I_{\text{avg}}^{\text{k}}$ and the interval length).
β_t	Duty factor in step t: fraction of the decision interval during which the radio is actively transmitting.
δ_t	Additional delay component for step t, capturing the extra latency induced by the chosen interval and backlog (added on top of the baseline RTT).
μ	Baseline mean RTT / delay of the link at the given QoS level, before adding queueing and interval-related delay.
p_{loss}	Per-step packet loss probability from the impairment model, as a function of QoS, δ_t, and the baseline loss rate.
e_{ref}	Reference high-drain energy level (mAh per step) used to normalize the energy cost component to the range $[0,1]$.
p_{crit}	Predicted probability that the current state is critical, produced by the auxiliary criticality head and used by the gating layer.
θ_{lo}	Lower criticality threshold: if $p_{\text{crit}} \leq \theta_{\text{lo}}$, the state is treated as clearly non-critical and only non-critical actions (with $c = 0$) are preferred.
θ_{hi}	Upper criticality threshold: if $p_{\text{crit}} \geq \theta_{\text{hi}}$, the state is treated as clearly critical and only critical actions (with $c = 1$) are preferred.
Q_{gated}	Q-value of an action after applying soft gating based on p_{crit}; penalizes actions whose critical flag disagrees with the estimated state criticality.
ε	Exploration rate in the ε-greedy policy, applied over the gated safe action set (probability of sampling a random safe action instead of the argmax).
$\tilde{Q}()$	Masked/gated Q-value function: Q-values after applying hard masks and soft gating, used to select the argmax only over allowed actions.
y_{crit}	True critical label (0 or 1) assigned to the transition by the rule-based severity rubric; used as the target for the criticality head.

γ	Discount factor in the DDQN update, weighting future rewards when constructing the TD target y.
Q_{target}	Q-values produced by the target network and used in the DDQN target computation.
y	Temporal-difference target for a single transition or its n-step variant).
e	Temporal-difference error for one sample.
$L_{\text{Q}}^{\text{raw}}$	Unweighted value-loss term for the Q-head (Huber loss of the TD error) before applying importance-sampling weights from prioritized replay.
L_{Q}	Weighted Q-head loss after applying the importance-sampling weight (from PER) to $L_{\text{Q}}^{\text{raw}}$.
α	Loss-balancing coefficient used when combining value and criticality losses, in the total loss.
L_{total}	Total loss per transition used to update the shared network parameters (sum of value loss and scaled criticality loss).
L_{crit}	Classification loss of the criticality head (focal loss comparing p_{crit} to y_{crit}).

Evaluation and Results

σ	Standard deviation of the stochastic component in the RTT / delay model, controlling how much individual delays vary around the mean.
$p(loss)$	Baseline loss-rate parameter in the network impairment model (pre-QoS probability that a packet is lost, before the adjustments that depend on QoS and delay).
F	Lifetime factor: multiplicative factor by which device lifetime changes relative to the baseline lifetime (e.g., $F > 1$ means the policy extends lifetime).
s	Relative energy-saving fraction between two policies.
$\overline{E}_{[1:100]}$	Average energy per episode over the first 100 episodes of offline training (baseline energy window).
$\overline{E}_{[601:700]}$	Average energy per episode over episodes 601–700 (converged policy window) in offline training.

T_0	Baseline device lifetime (e.g., in hours) for a given battery and reference discharge current, before applying any energy-saving policy.
$Capacity$	Battery capacity of the device, measured in milliamp-hours (mAh). It quantifies the total stored charge available for discharge before the battery is depleted.
I_0	Reference discharge current (mA) used together with $Capacity$ to define the baseline lifetime T_0.
$\overline{\mathrm{OnTime}}_{[1:100]}$	Average on-time delivery rate (fraction of packets arriving within the on-time deadline) over the first 100 episodes.
$\overline{\mathrm{OnTime}}_{[601:700]}$	Average on-time delivery rate over episodes 601–700.
Δ_{pp}	Change in on-time delivery expressed in percentage points (pp), typically the difference between two averages such as $\overline{\mathrm{OnTime}}_{[601:700]}$ and $\overline{\mathrm{OnTime}}_{[1:100]}$.

1.

Introduction

Modern industry, urban infrastructure, and many other sectors of daily life have an impact from the progress of Internet of Things (IoT). Now, thousands of heterogeneous devices can exchange data and act autonomously due to such progression. Such innovations are affecting a wide variety of collections of physical objects, devices, and systems, each with different characteristics such as hardware, operating system, communication protocols, power levels, and functionalities. While deployments are scaling up, communication architectures need to ensure efficient, reliable, and adaptive data transmission within restricted energy limits, constrained bandwidth situations, and diverse wireless conditions[1][2]. Protocol-wise, Constrained Application Protocol (CoAP) and Message Queuing Telemetry Transport (MQTT) at the application layer stand as transfer protocols for resource-constrained settings: CoAP uses User Datagram Protocol (UDP) for minimum overhead, whereas MQTT prefers Transmission Control Protocol (TCP) for stronger delivery assurance at the cost of higher resource utilization[3][4][5].

Reliability in wireless sensor networks often relies on devices that have smaller batteries and low processing power. Earlier work usually ensures reliability by using fixed rules such as retransmission and redundancy [6]. Retransmission means sending a packet again if it is lost, and redundancy means sending additional copies of the same data to prevent errors. Repeated transmissions recover lost information, but it also causes extra traffic, increase energy consumption in sensor batteries, and make congestion worse. The survey in Mahmood et al. (2015) argues for a focus on event reliability rather than strict packet reliability. This approach is better for saving energy: event reliability requires that the monitoring system receive enough information to detect a relevant event, whereas packet reliability demands successful delivery of every individual packet regardless of importance [6].

Deep Reinforcement Learning (DRL) offers a more adaptive alternative to such fixed strategies. Deep Q-Networks (DQN) and Double Deep Q-Networks (DDQN) have already been applied to network control and parameter tuning, and both methods show strong performance in complex decision task [7][8]. Most current networking studies that use these methods mainly focus on theoretical models or computer simulations. They often depend on static protocol choices or hand-crafted heuristics for protocol selection, message scheduling, and Quality of Service (QoS) adaptation. This project puts together CoAP and MQTT into a single hybrid pipeline. A learning agent changes the QoS, reporting interval, and the details of

the data being sent while it's running. It makes these adjustments based on the battery level, how quickly messages arrive, and the number of lost messages.

A critical state in this context corresponds to a combination of fast environmental change, growing buffer backlog, rising loss or delay, or explicit anomaly flags, which calls for temporary escalation of QoS, reporting frequency, or payload fidelity. Since sustained aggressiveness drains batteries and shortens network lifetime, any escalation is kept brief by design. So far, some of the studied aspects include critical-state prediction, online parameter adjustment, and end-to-end validation from simulation to real deployments.

1.1 Motivation

This thesis uses a hybrid CoAP–MQTT architecture that combines CoAP's low-overhead UDP semantics for local sensing with MQTT's brokered, publish–subscribe model and QoS-based delivery guarantees. In fact, there is a key issue in relying on static, pre-configured settings for important factors like how often messages are sent and the level of QoS. These pre-set settings don't work well in variable resource-constrained conditions, where changes of data importance and finite energy budget happen all the time. This lack of flexibility can cause problems, like using too much energy when it's not needed during bad network conditions or emergencies (under-provisioning) or missing important data when using low-energy settings during calm periods (over-provisioning) [10].

The main challenge is finding a balance between reliable data delivery, timely responses, and using energy efficiently [11]. A recent study about Media Access Control (MAC) protocols for energy-efficient wireless sensor networks found that it is hard to balance these in event-driven applications such as environmental monitoring, where traffic patterns are bursty rather than steady [50]. In these situations, some nodes may remain idle for long periods and then experience sudden, high-traffic events that activate many nodes at once. Fixed communication rules and typical asynchronous MAC protocols behave efficiently during low-load periods but tend to incur higher packet loss and delay during these high-contention bursts.

This thesis treats this mismatch as a gap between static communication rules and the changing conditions of the network and sensors. In other words, the system lacks situation awareness: it does not recognise when traffic, link quality, or battery state have shifted, so the communication settings remain inappropriate. The system needs to figure out its current state

by looking at factors like battery level, network delays, and sensor activity. Then, it should change how it communicates in real time. When an important event happens, the system should temporarily adopt more aggressive settings, such as higher reporting frequency or QoS levels, and then return to energy-saving behaviour once the urgent situation has passed, thereby extending network lifetime. This thesis argues that not having a smart control system is a big problem in making these hybrid setups reach their full potential. This brings up the main question:

> Can an agent (DDQN) that adapts transmission parameters in a hybrid CoAP–MQTT system reduce energy consumption and extend network lifetime while maintaining or improving reliability and timeliness compared with fixed or heuristic baselines under non-stationary conditions?

To answer this, the thesis will create, test, and evaluate a system where a DDQN agent watches the network and learns the best ways to change settings like QoS levels or how often messages are sent, helping build a more effective and stronger IoT network.

2.

IoT Transfer Protocols

Picking up the right communication protocol is an important choice to design any IoT system. This choice has a direct impact on how well edge devices can grow, how reliable they are, and how efficiently they operate. The unique constraints of such IoT environments, like devices that use low power with unreliable networks in large-scale deployments, make things complicated. For example, standard web protocols like Hyper Text Transfer Protocol / Representational State Transfer (HTTP/REST) are widely used, but they have a verbose text-based format, and the high overhead of TCP makes them not ideal for battery-constrained devices. In the same way, enterprise-grade messaging protocols like Advanced Message Queuing Protocol (AMQP) offer robust features, but they are too complicated and use too many resources for resource constrained nodes.

These specific constraints of IoT have led to the creation of specialized, lightweight protocols, with CoAP and MQTT being the most leading ones. It's important to understand that these protocols are not only rivals; they actually work well together and help with solving different parts of the end-to-end communication problem. CoAP is great for quick and low-overhead communication right at the edge of the network. On the other hand, MQTT uses a flexible system where a central broker helps send messages. This makes it great for sharing data reliably with backend systems.

This complementary idea shows that a hybrid architecture, which uses the best parts of both types, offers an optimal choice for numerous end-to-end data pipelines. This thesis adopts and develops a hybrid design pattern that combines CoAP and MQTT. This chapter explains why this building design was chosen by looking at important aspects of various IoT application-layer protocols. The analysis starts with a simple comparison of popular options: HTTP/REST, MQTT, CoAP, AMQP, and XMPP. Table 2.1 highlights the key differences in their communication models, the extra data they use in headers, and how well they work for devices with limited resources compared to backend distribution. This comparison will show how the CoAP–MQTT design works well for handling data on devices and can grow easily on the server side. Such hybrid architecture has clear benefits over using just one protocol because it ensures reliable data delivery and keeps different tasks organized:

Feature	CoAP-only [1]	MQTT-only [1]	HTTP (REST) [15]	AMQP / XMPP [13] [15]	Hybrid CoAP–MQTT [14]

Network connection type	Uses UDP (lightweight) with optional confirmation; optional security (DTLS)	Uses persistent TCP connections with keep-alive mechanisms; optional security (TLS)	Using HTTP/2 allows multiple data streams over a single connection	Uses persistent TCP connections with rich feature sets	Combines CoAP (UDP) for edge devices with MQTT (TCP) for core network env.
Interaction and Application Programming Interface (API)	RESTful (GET, PUT, POST, DELETE) with built-in observation capabilities	Topic-based publish/ subscribe messaging	RESTful (GET, PUT, POST, DELETE)	AMQP: Queue-based messaging; XMPP: XML-based	RESTful at edge devices; pub/sub messaging in the core network
Discovery mechanism	Built-in discovery through standard endpoint (/. well-known/core); [1]CoRE Link Format [49]	Requires pre-configured broker address; no built-in discovery mechanism	Relies on external methods like DNS Service Discovery	Requires external service registries	Uses CoAP discovery at edge devices; Broker centric for core communication
Resource usage	Very low memory and processing needs; compact message format; no connection maintenance overhead	Moderate resource requirements: need memory for session management and keep-alive timers	High resource demands: verbose headers and connection management require more memory and processing power	Higher resource requirements: complex protocol features need significant processing capabilities	Low resource requirements at edge devices using CoAP; processing complexity offloaded to more powerful gateway devices
Message reliability and storage	Caching for temporary storage; no built-in message	Strong reliability: Broker-based persistence; multiple QoS	No built-in message storage: each request/response is independent;	Strong message queuing and routing capabilities;	Combines CoAP's simplicity at edge with MQTT

[1] Constrained RESTful Environments

	persistence for disconnected devices	levels; can retain latest messages on topics	relies on application logic for reliability	built-in persistence mechanisms for enterprise reliability	broker's message persistence and QoS guarantees
Large network performance	Moderate scalability; can use proxies to extend reach; struggles with very large numbers of subscribers to a single source	Excellent scalability: broker efficiently handles fan-out to thousands of subscribers; proven in large deployments	High scalability using existing web infrastructure like CDNs and load balancers	High scalability in enterprise environment; designed for large-scale messaging systems	Excellent scalability through MQTT broker fan-out while maintaining efficiency at the constrained edge devices
Best suited application	Resource-constrained sensors and actuators in local networks; direct device-to-device communication; battery-powered edge devices	Large-scale telemetry collection; situations with many subscribers to the same data streams; cloud integration	Web and mobile applications that need to integrate with IoT devices; scenarios where web developer familiarity is important	Enterprise messaging systems; business applications requiring transaction guarantees; human communication applications (XMPP)	Systems requiring both local efficiency and cloud scalability; industrial IoT with clear separation between edge devices and central processing

Table 2. 1 Comparison between different architectures including transfer protocols used in IoT

The comparison in the Table 2.1 explains why the hybrid CoAP–MQTT architecture has been chosen for this thesis. Such hybrid design is specially adopted to tackle the core problem in IoT, which is finding a good balance between how well devices work on their own and how trustworthy the main system is. This architecture has a reduced resource cost, which makes it great for constrained battery powered devices. At the same time, it inherits on the strong features of separating data and storing it, and excellent ability to scale across Wide Area Networks (WAN) that an MQTT-only system has. This is critical for handling large amounts of data by making sure that it reliably reaches backend applications. In simple terms, this approach avoids the high resource use of HTTP/AMQP while fixing the moderate scalability

and limited persistence issues of using only CoAP. This work uses a combination of CoAP and MQTT as the best way to create an adaptive and end-to-end data pipeline.

2.1 CoAP—MQTT Design Pattern

Based on the above discussion, the hybrid CoAP–MQTT architecture has been adopted because of the added value of combining both protocols. This creates a design pattern that is easy to expand and manage. In such hybrid design, each protocol is used in an environment where it offers the most substantial advantage.

- **CoAP at the Network Edge**: This is where CoAP helps the sensor devices talk to each other. CoAP is easy to use and doesn't take up much power or uses low bandwidth and header overhead, which makes it optimal for sensors that run on batteries.
- **MQTT at the Core Network**: The MQTT protocol is used for communication with the backend. It allows services to easily share information with each other. This is helpful even if some network connections are intermittent or unreliable.

The key part of this architecture is a bridge component that connects CoAP and MQTT. It translates between the two protocols and allows for control at the same time. In this study, a local MQTT broker (running inside the testbed) is implemented to manage the environment so that experiments can be repeated easily. By keeping the system localized, it also ensures that any changes observed in performance can be observed clearly due to what the RL agent does [14].

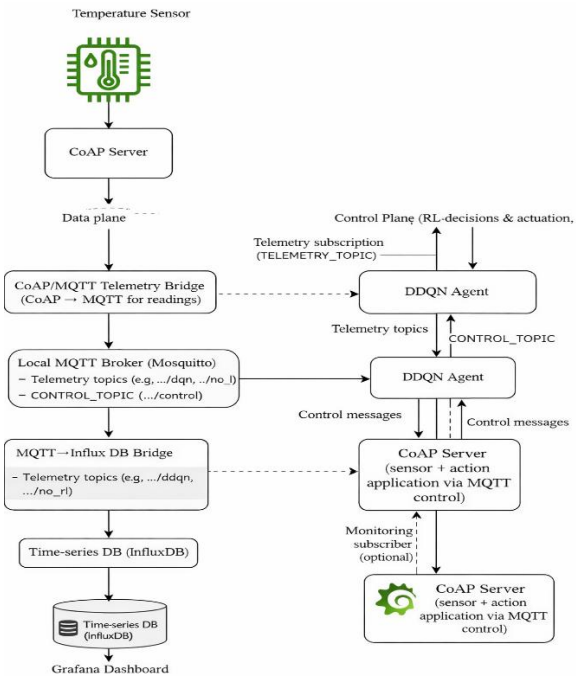

Figure 2. 1 CoAP—MQTT architecture

As shown in Figure 2.1, this design constitutes two separate communication pathways [16]:

- **Data Plane**: This plane outlines how sensor data moves from devices to backend services. The path can be presented like this: CoAP Sensor → CoAP/MQTT Telemetry Bridge → Local MQTT Broker → MQTT-Influx DB Bridge.
- **Control Plane**: This plane shows how commands go back from the RL agent to the sensors. The path can be presented like this: RL Agent → Local MQTT Broker → CoAP/MQTT Control Bridge → CoAP Sensor.

The RL agent chooses the adaptive actions, and these actions are carried out at specific times in this framework:

- **MQTT QoS**: After receiving the action from the RL agent via the control topic, the CoAP/MQTT bridge sets the QoS field of each MQTT PUBLISH message before sending it to the Local MQTT broker [17].
- **Reporting Interval**: The RL agent sends a control command through the local MQTT broker to the CoAP server, which updates its local timer so that sensor readings are generated and sent at the new interval [17].

- **Payload Fidelity**: The CoAP/MQTT control bridge applies the selected fidelity level when building the MQTT payload: it may forward only a subset of fields or an aggregated/rounded version of the sensor data so that the message size stays within the desired limit [17].
- **Critical Flag**: The RL agent sets the critical flag in its action. The bridge interprets this flag to temporarily apply more aggressive settings (for example, higher QoS or shorter interval) for high priority events and may tag these messages on a dedicated "critical" topic [19].

This architecture incorporates practical features designed to ensure robust system performance. Although the communication bridge is capable of supporting secure encryption standards—specifically DTLS for CoAP and TLS for MQTT— these security features are currently turned off. This design choice makes the experiment easier, because the project focuses on the analysis of energy consumption, transmission delay, and reliability trade-offs within a controlled environmental setting. Enabling security protocols would introduce additional processing tasks and data traffic, which would require a separate, dedicated evaluation to distinguish from the core network performance.

To check the system, the test setup utilizes a lightweight monitoring tool that subscribes to specific MQTT data streams. This monitor records pipeline performance based on three critical indicators: battery duration, message delivery speed (latency), and the success rate of message arrival. These metrics were selected to reflect the fundamental challenge of reducing energy usage without compromising the timeliness or dependability of the data. As standard measures for IoT systems, these indicators provide the necessary baseline to compare different architectural decisions and configurations in subsequent chapters [18].

2.2 Related Work

This section reviews other research or studies by examining topics such as how CoAP and MQTT work together, using DRL for network control, and adaptive sensing. Such review points out a gap that this thesis aims to fill: the necessity for a dynamic, context-aware control policy suited for application-layer transfer semantics within a hybrid CoAP–MQTT architecture.

2.2.1 Architectural Patterns for Protocol Interworking

An interesting area of research includes regarding how different protocols can work together efficiently. For example, Derhamy et al. (2016) suggested a service-oriented translator between CoAP and HTTP, allowing constrained devices to appear as RESTful web resources without a broker [13], But this approach lacks reliable message distribution, or QoS at topic level like MQTT provides with broke based system. Khaled et al. (2017) introduced Atlas, a marker-oriented framework that helps on saving energy while connecting MQTT/CoAP/HTTP [28]. Lee et al. (2016) demonstrates a multi-protocol proxy (CoAP/MQTT/XMPP) and use Software Defined Networking (SDN) to intercept traffic and flow across protocol boundaries, allowing for two-way flows without changing at the endpoints [29].

These above works address mainly how CoAP and MQTT work together. Once the bridge is set up, an important question still needs to be answered: What messaging settings should be used for the application (like MQTT QoS, rate, and quality) based on how the system is working at the moment. There hasn't been much research on this issue, and it needs to take into account the state of the network and the sensors.

2.2.2 Deep RL for Transfer-Level Control in Constrained Networks

Application-layer transfer information (QoS, rate, fidelity decision made above UDP/TCP) in constrained stacks have been the subject of deep RL applications. Donta et al. (2020/2023, iCoCoA) showed promising results in improving throughput, delivery ratio, and energy efficiency when using learned CoAP congestion, timeout, and backoff policies in comparison to the default CoAP settings [30]. Such advancements aim to fine-tune protocol parameters on refining communication protocols to reduce issues like data loss and network congestion. They have investigated how to optimize timeout and retransmission parameters for the CoAP protocol. What they found was that adjusting these factors can improve things like data delivery rates, overall throughput, and even energy efficiency when compared to standard UDP settings.

2.2.3 Adaptive/Exception-Based Sampling

In terms of adaptive sampling, often called as report-by-exception, several researchers have indicated its effectiveness in cutting down on unnecessary data transmissions. Sun et al.

(2025) applied a DQN to determine what data to sample and when to do so. This decision was based on a reward system that considered how useful the information was, how much energy it used, and how much similar data already existed. This method did better than fixed-rate plans [31]. Previous works (such as Gaber et al.) has also highlighted that this event-driven sampling approach keeps data valuable while still minimizing the costs [32].

While earlier studies have primarily focused on optimizing how data is generated at the sampling level, this thesis looks at a different issue: figuring out the best ways to move data that is already being streamed via MQTT. This particular area of literature hasn't been deeply explored and making it crucial to learn how to adjust the QoS settings in response to varying data conditions.

2.2.4 MQTT QoS Trade-Offs: Reliability, Latency, Energy

The QoS system within the MQTT protocol operates on three tiers. The simplest, QoS 0, delivers messages on a fire-and-forget basis, meaning that messages are sent once with no acknowledgment. This approach is quick and saves energy but doesn't guarantee that the message was received. The next tier, QoS 1, ensures that messages are received at least once through a handshake process, which increases reliability but can introduce some delays and possible duplications. At the top tier, QoS 2, messages are guaranteed to be delivered exactly once using a more complex four-way handshake. While this is the most reliable option, it comes with the biggest costs in terms of time, bandwidth, and energy use. Generally, experts recommend using QoS0 for less critical, high-frequency data, and opting for QoS1 or QoS2 for important notifications or commands. Studies like González (2023) and Thangavel (2014) suggest that using lesser demanding options can lead to energy savings, and there's potential for improved reliability in message handling through effective broker management [33][34].

When it comes to MQTT, it is QoS ladder, which illustrates the trade-off between reliability and overhead. There are three levels: QoS0 (best effort, minimal overhead), QoS1 (at least once, which can lead to duplicates), and QoS2 (exactly once, but it requires the most resources for handshakes). Studies and recommendations typically suggest using QoS0 for high-speed, low-priority data and QoS1 or QoS2 for alarms and commands. Research has shown that using lower-overhead modes can save battery life and that enhancing broker buffering and redelivery can improve reliability in specific situations [35].

Most recommendations are rather broad and do not readily adapt to changing circumstances like data loss or battery levels. Finding ways to dynamically adjust these quality settings could be an important advancement in this area.

2.2.5 Criticality and PASA-Inspired Context Metrics

Finding out what information is "urgent", or "critical", is a big challenge for any adaptive communication system. An intelligent system needs to understand when to save energy and when to use it to make sure an important message gets sent. The Priority-based Adaptive Scheduling Algorithm (PASA) by Suseendran in 2019 depicts how urgent something is by looking at different factors. These factors include how important the data is (like if it has a deadline), how much battery the device has left, and how much space is left in its temporary data storage. The PASA paper shows that a situation becomes "critical" and needs quick action if there is an emergency, the battery is running low, or the data buffer is almost full. The main idea is that PASA focuses on getting data through quickly instead of saving energy when there's a risk of losing data. For example, a device with a low battery is told to send its information so that it can share everything before it runs out of power. The main idea is that when something is urgent, like a low battery or a full memory, it becomes more important than saving energy. The "other data" (less urgent) is saved in the buffer and transmitted later using low-power, energy-saving settings [35].

This thesis considers a similar problem with its hybrid CoAP–MQTT setup. The MQTT protocol has a QoS ladder that helps balance how reliable something is with how much it costs. The lowest level (QoS 0) is quick and uses less energy, but it makes no guarantees, so messages might get lost. The highest quality levels (QoS 1 and 2) are very dependable, but they take more time, data, and energy. This thesis addresses the issue that many systems use fixed quality settings, which are not flexible and can't adjust to changing situations. This work takes the adaptable logic from PASA and uses it in this hybrid architecture. The agent created in this thesis will check how the system is doing, like the battery level, how full the buffer is, and the network delay. It will use this information to change the MQTT QoS or how often it reports. This helps the system save energy most of the time, but it will use more energy only when there's a real emergency.

2.2.6 Reward Shaping and Algorithmic Enhancements

In networking, RL may combine multiple objectives. The reward may use a single scalar with strong penalties for hard violations (buffer overflow, missed deadlines) and softer costs for latency/energy. Algorithmically, this thesis adopts Double-DQN with target networks, prioritized replay, and n-step returns following Rainbow's evidence for stability and data efficiency. This approach heavily penalizes serious issues like dropped packets or missed deadlines, while imposing lighter costs for things like latency and energy use [36]. For adaptive sampling, Huang (2025) offers a straightforward model, using the following scalar reward formula, R_t, at time stamp, t [37]:

$$R_t = w_{\mathrm{I}}\, U_{\mathrm{info}} - w_{\mathrm{E}} C_{\mathrm{E}} - w_{\mathrm{R}} C_{\mathrm{red}} \qquad (1)$$

According to Huang, in equation (1), the ideal weights are roughly 0.4 for w_{I} with information gain U_{info}, 0.4 for w_{E} with energy cost C_{E}, and 0.2 for w_{R} with redundancy cost C_{red}, striking a good balance between value and cost. Shresthamali (2021) warns that this might skew policies [35]. They recommend treating the underlying objectives explicitly—e.g. in a multi-objective RL (MORL) formulation or via a careful breakdown of the reward—so that important signals, such as energy neutrality in energy-harvesting sensors, are not drowned out by more frequent terms.

The key lesson is that collapsing many goals into a single scalar can hide (mask) individual objectives if the weighting is not chosen carefully. In this thesis, the different goals (information, timeliness, reliability, energy, backlog/safety) are kept as separate components in the reward calculation, but they are ultimately combined into one scalar reward with hand-chosen weights and state-dependent multipliers. This is a scalarised multi-objective controller with constraints, not a full vector-valued or MORL method with separate value functions per objective. Severe penalties are reserved for safety-critical events (missed critical alerts, buffer overflows), while latency and energy are penalised more gently. This design follows safe RL practice in which hard violations dominate the reward, effectively constraining the learned policy to remain within acceptable operating bounds.

On the learning side, techniques inspired by the Rainbow method—such as Double DQN, target networks, prioritized replay, and n-step returns—are known to enhance stability and make more efficient use of samples (Hessel et al., 2017), which is crucial when episodes

are costly [39]. While these concepts are fairly connected, there's limited documentation on how to apply them specifically to transfer policies in CoAP to MQTT pipelines, especially when considering context from PASA and safety measures like action masking against undesirable QoS or rate choices. In this sense, the present work should be viewed as a pragmatic, scalarised multi-objective heuristic with safety guardrails. A more principled multi-objective or constrained-MDP formulation, where objectives are represented by a vector reward and Pareto trade-offs are analysed directly, is left as future work due to time and resource limitations.

2.2.7 Action Masking and Safety Constraints

Another way to incorporate domain knowledge into a RL agent is to mask out actions that are considered infeasible or unsafe in a given state. In simple terms, action masking means that, at each decision step, the agent is only allowed to choose from a subset of actions that are considered safe or feasible in the current state. Prior research shows that such "invalid action masking" can speed up learning in large discrete action spaces because the agent does not waste time on actions that are impossible by design. For instance, Huang et al. (2020) show that, for policy-gradient methods, correctly applied invalid action masking still yields a valid policy gradient and scales well as the number of invalid actions grows. They also show that a naive variant—sampling from the masked policy but updating the gradient using the unmasked probabilities—leads to unstable training with very large policy updates, so the masking must be integrated carefully [43]. This idea connects directly to the IoT transfer control problem discussed in this thesis. Action masking can act like strict safety rules. For example, if the network is very busy, the system might temporarily "mask" or stop the agent from choosing an unreliable, "fire-and-forget" action. In another situation, when a device's battery is very low, the system might mask actions that need a lot of power, like sending frequent data or detailed information. This overall conclusion is backed Wang et al. (2024) who found that focusing the agent on only feasible choices accelerates training. Masks thus complement reward penalties by preventing unsafe choices upfront rather than only punishing them afterward [42].

In theory, this principle is realised mainly in the offline DDQN trainer through rule-based hard masks and gating that restrict the candidate actions in each state, while the Q-learning update remains unchanged. The online agent then uses softer gating in the same spirit, nudging decisions toward safe, feasible actions without fully removing every option.

2.2.8 Simulating Realistic Conditions: Contextualizing the Problem

A core goal of this thesis is to create a flexible controller that works well in real-life IoT settings. To properly check how well the agent is doing, a simulation is developed that mimics two important real-world limitations: limited battery life and unstable networks. To model battery life, this study uses Peukert's Law instead of a simple linear discharge model. A basic model assumes a battery drains up power at a steady speed, but that's not correct. Peukert's Law is a well-known model that shows a non-linear pattern: the faster a battery is drained, the less total energy it can provide [44]. This concept is directly relevant to this thesis because high-reliability actions, such as frequent reporting or using a high MQTT QoS, are "fast drains" that consume a disproportionate amount of energy. Using this model ensures the simulation accurately reflects the true energy cost of the agent's decisions. To model network unreliability, this work introduces two factors: random packet loss and network jitter. Jitter, in this specific context, refers to the variability in network delay (latency)—the fact that packets do not all take the same amount of time to arrive. This is a common real-world condition that can significantly impact queueing and delay performance, as noted in IoT networking studies [45].

By adding random noise to packet delays and sensor reading intervals, the simulation environment ensures the learned policy becomes robust and does not fail when faced with real-world timing fluctuations. Overall, incorporating Peukert's battery model and network jitter grounds the agent's training in conditions that are much closer to reality, which is critical for improving the policy's chances of success in an actual deployment.

2.2.9 Algorithmic Enhancements for Stability and Efficiency

A standard DQN usually has a hard time dealing with the complexity and unpredictability of a real-world control problem like this. To ensure the agent learns in a strong and reliable way to make decisions, this work uses several algorithmic enhancements, most of which are part of the advanced "Rainbow DQN" framework [39]. These enhancements are chosen to help with real problems.

The most critical challenge is learning stability. A basic agent's value estimates can sometimes spiral out of control. To prevent this, two techniques are used. First, a Double DQN architecture is adopted, which helps prevent the agent from becoming overly optimistic about

the value of its actions. Second, soft updates (Polyak averaging) are used. This means that instead of abruptly changing its "target", the agent updates its goals gradually, which smooths out the learning process and prevents the oscillations that can stop an agent from learning [46].

The second challenge is learning efficiency. To help the agent learn faster, a Dueling Network architecture is used. This allows the agent to separately learn the value of a given state (e.g., "a low battery is bad") and the advantage of a specific action (e.g., "changing QoS is good in this state"). This work also integrates Prioritized Experience Replay, which allows the agent to learn more from rare but important events—such as a critical data loss—rather than treating all memories as equally important. Finally, to ensure smart exploration, this work uses Noisy Nets, which allows the agent to learn how to explore its environment in an intelligent way, rather than just picking random actions [39]. By leveraging this set of proven techniques, the agent can more reliably converge on the subtle, context-aware policy required for this hybrid system.

Source (Year)	**Problem Domain**	**Study Focus and Protocols**	**RL technique**	**Key findings and performance impact**
Shresthamali et al. (2021)[38]	Managing power for long-term operation in energy-harvesting (EH) nodes.	Balancing energy use between sensing and communication tasks (Custom protocol).	Multi-Objective RL (MOMDP)	The policy successfully balanced tasks to achieve "energy-neutral" operation (using only harvested energy), outperforming single-goal controllers.
Donta et al. – iCoCoA (2023)[30]	Poor CoAP performance (data loss, low throughput) in congested or unreliable networks.	Learning optimal congestion control parameters (like timeouts) for CoAP over UDP.	DQN	The agent learned better timeout values than the CoAP default, resulting in higher throughput and reduced energy consumption.
Sun et al. (2025)[31]	Wasting energy by sampling and sending redundant or non-critical sensor data.	Deciding when to sample sensor data (generic HTTP/CoAP) to maximize information value vs. energy cost.	DQN	The policy learned to sample less often, reducing redundant packets while still maintaining high data quality.
Jin et al. – XGate (2025)[40]	Overload and anomaly handling at IoT API gateways	Routing API traffic (from HTTP/MQTT gateways) using an "explainable"	DQN with Explainability (XRL)	The agent reduced latency and increased throughput. The explainability

	(where data enters the cloud).	agent that can justify its decisions.		feature also improved operator trust versus "black box" models.
(T. C. Jermin Jeaunita, 2021)[41]	High latency and packet loss in IoT mesh networks (RPL protocol) under heavy traffic.	Optimizing the routing path (selecting the best "next hop") within a 6LoWPAN/RPL mesh network.	Q-Network (multi-agent)	The agents learned more efficient routes, leading to less network delay (faster convergence) and reduced control overhead.

Table 2. 2 Selected studies applying RL in IoT communication systems

Based on the studies in Table 2.2 show a clear pattern. Each work focuses on RL to solve one specific problem in communication systems. This includes choosing CoAP congestion-control parameters, deciding when to take sample, routing in specific routing protocol for low power and unreliable network, or managing traffic at an API gateway. The learning agent usually adjusts one protocol or one network hop at a time while leaving everything else to be the same. None of these studies targets the application-layer settings that control a complete CoAP–MQTT pipeline, such as MQTT QoS levels, how often data is reported, or payload fidelity, nor do they use battery, backlog, and delay together as explicit inputs to guide those choices.

This thesis builds on those ideas and changes the focus to an end-to-end, application-layer controller for a hybrid CoAP–MQTT architecture. A DRL agent uses PASA-style priority cues and safety mechanisms like action masking to adjust QoS, interval, and fidelity in real time based on the current state of the device and network. In this way, the work aims to address an issue: creating a smart system that uses context-aware control policy that jointly manages energy use, reliability, and timeliness across the full IoT data path, rather than only tuning one protocol parameter in isolation.

3.

Problem Formulation (MDP)

This chapter explains the adaptive communication problem as a Markov Decision Process (MDP) for a hybrid CoAP–MQTT IoT system. An MDP is a standard way to describe situations where an agent repeatedly observes the system, takes an action, and then receives feedback about how good that action was. In this work, the agent needs to figure out the best way to send sensor data so that energy is saved whenever possible, but important information is still delivered on time and reliably. At each decision step, the agent observes a compact description of the current situation. This description includes how quickly the sensor value is changing, how much data is waiting in the buffer, how much battery is left, how many packets have been lost recently, how large the delays are, and whether a simple anomaly detector has fired. Based on this state, the agent chooses an action. The action specifies the MQTT QoS level, how often the sensor reports (the reporting interval), how detailed the data should be (payload fidelity), and whether the current step should be treated as "critical" for a short time. After the action is applied, the environment responds with a new state and a single numerical reward that captures how good that decision was.

The reward is designed to encourage desirable behaviour and discourage undesirable one. Positive contributions reflect useful outcomes like delivering important information, finishing tasks on time, and maintaining reliable communication. Negative contributions represent costs, such as energy consumption and extra airtime or latency. Additional shaping terms are used to enforce safe and efficient behaviour. The main goal of the agent is to learn a policy that picks actions which maximize the long-term sum of these rewards.

Safety is a primary concern in this design. The agent is not allowed to always choose arbitrary actions. Instead, masks and gating logic stop obviously dangerous actions in risky situations. For example, actions that would overload the network or quickly drain the battery can be turned off before the agent makes a decision. Also, the learning process focuses on practical decisions like using a neural network to guess outcomes, having stable learning targets, and a careful exploration strategy. The training process starts from offline data and is later refined during online interaction with the simulated system. Because truly "critical" moments happen less often than calm periods, the loss function and sampling strategy give more focus on rare situations that are important for safety. This helps ensure the agent pays attention to them and doesn't overlook them.

3.1 Adaptive Control Problem

In this thesis, the hybrid CoAP–MQTT stack adapts transfer settings to context (urgency, link quality, battery) to balance timeliness, reliability, and energy. This section formalizes the problem as an MDP, defines the state and action spaces, and outlines the learning and safety choices that make the agent practical and reproducible.

3.1.2 Critical Events and Class Imbalance

The main idea of this work is the concept of a "critical" operating window. A step is considered "critical" when the system needs to prioritize being on time and reliable rather than just saving energy. This usually happens when the sensor data is changing quickly, when there's a lot of unsent information piling up, when there is a lot of packet loss or delay, or when something unusual is noticed. Outside those periods, the agent should use simpler settings to save battery life.

To find such windows, each decision step checks at a small set of discrete indicators. One indicator shows how much the sensor value has changed recently. It uses three levels such as "no or small change", "moderate change", and "large jump". A second indicator indicates how full the outgoing buffer is, again using a few simple levels to show how much unsent information has accumulated. A third indicator describes the remaining battery as low, medium, or high. Two further indicators classify recent packet loss and recent end-to-end delay into "low", "moderate", or "high" categories. Finally, a binary anomaly flag marks obvious out-of-range or abrupt changes in the signal.

These indicators are kept simple to make the policy compact and suitable for constrained devices. Even with such simple values, they capture the main situations in which the application needs to provide better service by improving fidelity, reporting more often, or increasing the MQTT QoS, and the situations where it should reduce performance to save energy. A step is considered as "critical" in two ways. Some conditions act as immediate triggers: for example, a sudden big change in signals, a nearly full buffer, severe loss, or very high delay can mark the step as "critical" without further computation. In all other situations, the system combines the indicators into one severity score. Larger changes, higher backlog, bigger loss and delay, and lower battery all make this score go up. If the score goes over a certain limit, the step is also labelled "critical". The exact details of this score and the schedule

for thresholds will be explained later, but the main idea is that they follow the PASA principle: **escalate when priority is high, and conserve otherwise.**

In typical IoT telemetry, most steps are calm and only a minority are truly "critical". This pattern is easy to see in the training examples of this work. In general, during the training episodes and steps, the observed share of critical steps was:

Class	**Count (# steps)**	**Share (%)**
Critical	38,879	22.2
Non-critical	136,121	77.8
Total	**175,000**	**100.0**

Table 3. 1 Empirical prevalence: critical vs non-critical steps and count

Based on the above Table 3.1, the critical proportion, $\hat{p}_{\text{crit}} = N_{\text{crit}}/N = 0.222$ where N represents total steps count and N_{crit} as "critical" steps count. Across all episodes, about 22% of decision steps were marked as "critical", while around 78% as non-critical. The positive class is not extremely rare but still a minority, and critical steps tend to occur in short bursts with some borderline cases near the decision threshold. In this work, the idea of a "critical" step follows the same rule-based definition described earlier. A step is marked as "critical" when the system needs to prioritize being on time and reliable rather than saving energy. This can happen in two ways. First, some conditions act as hard triggers: a very large signal change, a nearly full buffer, very high loss, very high delay, or an anomaly flag immediately mark the step as "critical". Next, in all other cases, the indicators are combined into a single severity score. Larger change, higher backlog, worse loss and delay, and lower battery all push this score up. If that score crosses a threshold, the step is also treated as "critical". This means that every step gets labelled as a critical/non-critical label based on this rule; no external hand-labelled dataset is used [51].

This rule is applied to every decision step during the training process. The counts in Table 3.1 come from this automatic labelling: only about 22% of the steps are marked as "critical", and the remaining 78% are non-critical. This means that critical steps form a minority class, even though they are very important for safety. This is what is meant here by "class imbalance": the system generates many more non-critical examples than critical ones [52].

To use this information inside the controller, the network includes an extra output that estimates how likely it is that the next step will be critical based on the same guideline. During training, the model learns to match the rules for labelling and uses a special loss function called focal loss. This means that rare and difficult critical examples matter more than easy non-critical ones. The goal is not to change what "critical" means, but to learn a smooth estimate of it that can guide the safety mechanisms. This learned criticality estimate is then used to steer the action choices. When the estimated probability of being in a critical window is high, the control logic favours "escalate" actions such as higher QoS, shorter reporting intervals, and higher fidelity, with the "critical" flag raised. When the estimate is low, the logic favours frugal actions that save battery. In this way, the critical versus non-critical split, defined by a clear rule and supported by the auxiliary head. It puts the PASA idea into action: provide more service when priority is high and save resources when the system is calm.

3.2 MDP Definition

The interaction between the agent and the IoT environment can be explained using a finite MDP, defined by the tuple as follows [20]:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma) \qquad (2)$$

In equation (2), $\mathcal{S}$ is the set of all possible states, $\mathcal{A}$ is the set of all possible actions, $\mathcal{P}$ is the transition probability function, $\mathcal{R}$ is the reward function, and $\gamma \in [0,1]$ is the discount factor. The discount factor γ controls how much future rewards matter compared to rewards received immediately.

At each discrete decision time step t, the system is in a state $s_t \in \mathcal{S}$. This state s_t is a brief overview of the current situation, created using discretized indicators derived from real-time or simulated telemetry. The agent then picks an action $a_t \in \mathcal{A}$. The environment changes to a new state s_{t+1} according to the transition probability function $\mathcal{P}$. The probability is shown as $P(s' \mid s, a)$ and gives the probability of moving in a next state s' when action a is taken in state s. At the same time, the reward function $\mathcal{R}$ yielding a scalar reward:

$$R_t = \mathcal{R}(s_t, a_t, s_{t+1})$$

which measures how good that particular transition was. The long-term goal of the agent is to maximize the discounted sum of all future rewards. This long-term quantity is called the return and is written as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \qquad (3)$$

This return G_t combines immediate and future rewards into one number, and the agent learns a policy that makes decisions which, on average, lead to high return. The design of the reward function $\mathcal{R}$ is therefore a key element in encoding the desired behaviour of the communication system [20].

3.2.1 State Space

The state space ($\mathcal{S}$) represents the set of all possible observations the agent can see and use to make decisions. The state representation is designed to be small but informative. At each decision step, the state s_t is built from nine indicators that together describe data urgency, congestion and timeliness, and resource and policy context.

- Data Urgency:
  - Rate of change, $\Delta T \in \{0,1,2\}$: This indicates the rate of change of the sensed signal. This variable ΔT shows how much the signal has changed recently. A value of $\Delta T = 0$ represents little or no change, $\Delta T = 1$ represents a moderate change, and $\Delta T = 2$ represents a large jump. Large values of ΔT indicate that new information has appeared and that prompt or detailed reporting may be needed.
  - Anomaly flag, $anom \in \{0,1\}$: This second urgency-related indicator is the anomaly flag. This flag $anom$ is set to 1 when simple rule-based checks detect an obvious anomaly, such as measurements going out of an acceptable range or changing very abruptly. When $anom = 1$, the situation is treated as clearly urgent.

- Congestion and Timeliness:
  - Backlog level, $bufT \in \{0,1,2\}$: This measures how full the bridge's outbound queue is. Larger values mean that more unsent data is waiting, which increases the risk of packet drops and long delays.

- Packet loss, $L \in \{0,1,2\}$: This reflects the recent reliability class based on transmission and acknowledgment statistics. low values indicate almost no loss, and high values indicate frequent loss of packets or acknowledgments.
- Delay, $D \in \{0,1,2\}$: This indicates recent end-to-end latency classifications; low values mean low delay, and high values mean that data is arriving very late.

- Resource and Policy Context:

  - Battery level, $B \in \{0,1,2\}$: This variable B is an indicator of remaining energy, with low, medium, and high levels. When B is low, the policy is pushed toward more economical/frugal actions.
  - Current Configuration, (I_{cur} , Q_{cur} , F_{cur}): Here $I_{\text{cur}} \in \{0,1,2\}$ is the current reporting interval level, $Q_{\text{cur}} \in \{0,1,2\}$ is the current MQTT QoS level, and $F_{\text{cur}} \in \{0,1,2\}$ is the current payload fidelity level. These three variables capture what configuration was chosen in the previous decision step.

Putting all of these together, the state at time t is:

$$s_t = (\Delta T, bufT, B, L, D, I_{\text{cur}}, Q_{\text{cur}}, F_{\text{cur}}, anom) \qquad (4)$$

Each of the eight multi-level indicators takes one of three values (0, 1, or 2), and the anomaly flag $anom$ is binary. For use with the neural network, each indicator is discretized and then one-hot encoded, meaning that each level is turned into a small binary vector. This conversion yields a 25-dimensional numerical input vector (eight indicators with three levels each and one binary flag). This 25-dimensional vector is the direct input to the policy network. Including the current configuration ($I_{\text{cur}}, Q_{\text{cur}}, F_{\text{cur}}$) helps to avoid situations where two states look identical from sensor and network values but actually lead to different futures because different settings are active; this keeps the process closer to the Markov property.

3.2.2 Action Space

The action space $\mathcal{A}$ contains all application-layer control choices available to the agent. Each action is a 4-tuple:

$$a_t = (q, \tau, f, c) \qquad (5)$$

The first component, $q \in \{0,1,2\}$, is the MQTT QoS level. Higher values of q represent stronger delivery guarantees but also more overhead due to acknowledgments and retransmissions. The second component, $\tau \in \{5,30,60\}$ seconds, is the sensing and transmission interval. Smaller values of τmean more frequent transmissions and fresher data, but also higher energy and airtime consumption. Larger values of τ save energy and airtime but risk sending outdated information.

The third component, $f \in \{0,1,2\}$, is the payload fidelity. This variable f controls how detailed the transmitted data is, for example low, medium, or high information content. Higher fidelity provides richer information but increases payload size and network usage. The fourth component, $c \in \{0,1\}$, is the critical flag. When $c = 1$, the step is explicitly marked as "critical" and the system can treat it as urgent, for example by publishing immediately or flushing backlog. When $c = 0$, no special critical handling is requested.

The combination of these four elements produces a discrete action space of size $3 \times 3 \times 3 \times 2 = 54$. Each action corresponds to a specific configuration of reliability (via q), reporting rate (via τ), information richness (via f), and critical signalling (via c). This gives the agent fine-grained control over the trade-offs between energy, timeliness, and reliability. The following figure 2.2 shows the interaction process between the agent and environment in DDQN can be defined as an MDP.

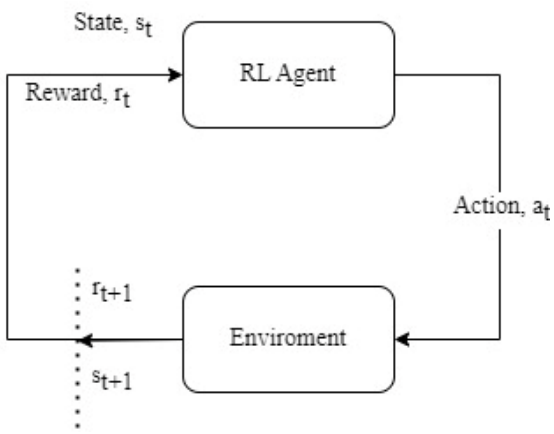

Figure 2. 2 The agent-environment interaction in MDP

3.2.3 Reward Function

This work uses a shaped, single scalar reward at each step. The reward combines several utility terms, which are higher when behaviour is desirable, and cost terms, which are higher

when behaviour is wasteful or risky. Safety-related shaping terms are also included so that certain unsafe outcomes are heavily penalized. Based on the equation (4), utility components depend on the next-state indicators$(\Delta T, bufT, B, L, D, anom)$. By making the utilities depend on the state reached after the action, the reward reflects both what the agent did and what actually happened in the system.

- **Utility components (higher is better):** Utilities act as positive reinforcement for actions that meet high-performance standards, with the reward amount adjusted based on the situation's priority.

  - **Information utility, U_{info}:** This term measures how much useful information is delivered by one transmission step. The idea is to ask two simple question capacity and demand. "Capacity" is used for how much detail can this packet carry and "demand" is for how much information is really needed right now. "Capacity" is proportional to the chosen fidelity $f \in \{0,1,2\}$, with higher f meaning higher information capacity. The "demand" depends on how much the environment is changing and how much unsent data is waiting in the queue. Both the rate-of-change bin $\Delta T \in \{0,1,2\}$ and the backlog bin $bufT \in \{0,1,2\}$ are used. The information utility multiplies these two terms and clips the result to $[0,1]$:

$$U_{\text{info}}(s,a,s') = \min\{1, \underbrace{(f+1)/3}_{\text{capacity}} \cdot \underbrace{\min(1,(\Delta T + bufT)/3)}_{\text{demand}(s,s')}\} \tag{6}$$

    In equation (6), the factor $(f+1)/3$ represents capacity, and the term $\min(1,(\Delta T + bufT)/3)$ represents demand, where ΔT and $bufT$ are the bins for rate of change and backlog derived from the state. The utility U_{info} is kept between 0 and 1, with 1 meaning that information demand is fully met. A weight w_{I} (for example $w_{\text{I}} = 0.30$) gives this term about 30% of the total reward before penalties. High fidelity is only rewarded when demand is high.

  - **Timeliness utility, U_{time}:** This term rewards fresh data delivery. It is higher when the reporting interval τ is short and when network delay D is low, especially during periods that require fresh observations. It is defined as:

$$U_{\text{time}}(s,a,s') = \frac{1}{1+\tau/5+D/2} \cdot \underbrace{\min(1, \Delta T/2 + anom)}_{\text{demand}(s,s')} \tag{7}$$

In equation (7), τ is the chosen interval (normalized by 5 seconds) and D is the delay class. The fraction becomes close to 1 when τ is small and D is low. The demand term $\min(1, \Delta T/2 + anom)$ becomes high when there is a big change or an anomaly. This means that short intervals matter most when fresh data is needed. A separate idle reward R_{idle} is used to give a small bonus when the system is calm and the most economical settings are in place. A weight w_{T} (for example $w_{\text{T}} = 0.20$) controls the importance of timeliness.

- **Reliability utility, U_{rel}:** This term rewards actions for choosing a QoS level that matches the current link conditions. The link conditions are represented by the loss and delay bins (L, D) observed in the next state s'. It goes up when the chosen QoS is appropriate and goes down when QoS is clearly too low or unnecessarily high. The equation can be represented as follows,

$$U_{\text{rel}}(s, a, s') \;=\; 1 \;-\; \kappa \; error\,(q, L, D), \quad (8)$$

In above equation (8), $q \in \{0,1,2\}$ is the QoS level chosen in the current action a_t. The same value is stored in the state as Q_{cur}, but to make it easier the formulas use $q, L, D \in \{0,1,2\}$ are the loss and delay bins in the next state s' and $error\,(q, L, D) \in \{0,1,2\}$. In this thesis, the mismatch penalty is computed in two parts. First, a link severity score is calculated based on the loss and delay bins:

$$link_sev(s') = 0.5\left(\frac{L}{2} + \frac{D}{2}\right) \in [0,1]$$

These averages normalized loss and delay, giving 0 for a very clean link and 1 for a very bad link. From this severity, a required QoS level $q^*(s') \in \{0,1,2\}$ is derived using simple thresholds (for example: $q^* = 0$ for low severity, $q^* = 1$ for moderate, and $q^* = 2$ for high severity). The normalized QoS mismatch is then,

$$\Delta_{\text{QoS}}(s, a, s') = \frac{|q - q^*(s')|}{2} \in [0,1]$$

This quantity is 0 when the chosen QoS equals the required level and reaches 1 only when they differ by two levels (for example, choosing QoS 0 when the link really requires QoS 2). The concrete reliability utility used in all experiments is

$$U_{\text{rel}}(s,a,s') = 1.0 - \left(\Delta_{\text{QoS}}(s,a,s')\right)^2 \times (0.6 + 0.8 \times link_sev(s')) \qquad (9)$$

In other words, in equation (8) the penalty term is

$$error(q,L,D) = \left(\Delta_{\text{QoS}}(\text{s},\text{a},\text{s}')\right)^2 \times (0.6 + 0.8 \times link_sev(s'))$$

which lies in a bounded range, starting at 0 when QoS is perfectly matched and increasing as the mismatch and link severity grow.

- If $q = q^*(s')$, then $\Delta_{\text{QoS}} = 0$, the penalty is 0, and $U_{\text{rel}} = 1$: the action is fully rewarded from a reliability point of view.
- If QoS is too low on a bad link (for example, $q = 0$ when $q^* = 2$), then Δ_{QoS} is large, the penalty increases, and U_{rel} can become significantly smaller or even negative.
- If QoS is much higher than needed on a very good link, the penalty also grows, reflecting wasted reliability overhead.

Later, in the overall scalar reward, this utility is multiplied by a weight w_{R} (for example $w_{\text{R}} \approx 0.28$). This makes reliability one of the main positive terms whenever the link is impaired, while still allowing other utilities and costs to shape the final reward.

- **Critical event handling utility or Critical correctness, U_{crit}:** This term rewards correct use of the critical flag c. First, a true criticality label for the new state s_{t+1} is computed by checking "red flag" conditions such as $anom = 1$ or extreme backlog or delay. If any of these conditions or a high combined score is present, the state is treated as truly critical. The utility is then,

$$U_{\text{crit}}(s,a,s') = \begin{cases} +\sigma(s'), & if\ c = true\ critical, \\ -\sigma(s'), & otherwise \end{cases} \qquad (10)$$

Where the severity $\sigma(s')$ is defined as, $\sigma(s') = \min\{2,\ 1 + 0.5(\Delta T/2) + 0.5(bufT/2) + 0.6 * anom\}$. If the chosen flag c agrees with the true critical label, U_{crit} is positive; otherwise, it is negative. The magnitude depends on σ, so mistakes in more severe situations are punished more strongly.

- **Cost components (lower is better):** Costs serve a crucial part in providing negative feedback for actions that are inefficient or resources intensive.

  - **Energy Cost (C_E):** It penalizes actions according to the estimated energy consumption expressed in milliampere-hours (mAh) for that decision interval. Higher transmission frequency τ^{-1}, higher fidelity f, or higher QoS q all increase power use. The cost is normalized to lie between 0 and 1:

$$C_{\mathrm{E}}(s,a,s') = min\{1, \frac{mAh\ (\tau,\ q,\ f,\ c)}{e_{\mathrm{ref}}}\} \cdot \left(1 + \frac{q}{2}\right) \text{ with } e_{\mathrm{ref}} = 0.010 \quad (11)$$

    which $mAh(\tau, q, f, c)$ is the estimated energy used in that step based on the action, and e_{ref} is a reference high-drain level. The factor $(1 + q/2)$ slightly increases the cost for higher QoS levels, reflecting extra acknowledgments. A context multiplier μ_{E} (for example 0.6 when battery B is low and 0.2 when B is high) scales this cost, making energy more valuable as the battery depletes.

  - **Network Strain (C_X):** penalizes sending large payloads over stressed links. It is defined as,

$$C_{\mathrm{X}}(s,a,s') = \underbrace{(f+1)/3}_{\text{payload proxy}} \cdot \underbrace{(L+D)/4}_{\text{link load}} \quad (12)$$

    The first factor approximates relative message size, from 0 for the lowest fidelity up to 1 for the highest. The second factor $(L + D)/4$ measures how stressed the network is; it is highest when both loss and delay are in their worst bins. Thus, sending large data when the link is already bad incurs a high penalty. A multiplier μ_{X} in the range 0 to 1 controls how strongly this term affects the total reward.

  - **Backlog Cost (C_{backlog}):** This cost term penalizes high queue occupancy. Recall that the backlog bin $bufT \in \{0,1,2\}$ in the next state s' summarizes how full the queue is (0 = low, 1 = medium, 2 = high). The backlog cost is defined simply as half of this bin value:

$$C_{\mathrm{backlog}}(s,a,s') = bufT(s')/2 \in \{0, \frac{1}{2}, 1\} \quad (13)$$

When the queue is almost empty ($bufT = 0$), the cost is 0; for a medium backlog ($bufT = 1$) the cost is 0.5; for a high backlog ($bufT = 2$) the cost reaches 1. This linear mapping keeps the backlog penalty bounded between 0 and 1 and makes it easy to combine with the other reward terms mentioned earlier.

Although $bufT$ is already part of the state, a separate backlog cost is still needed: the state tells the agent what the queue level is, but the cost tells the agent how undesirable it is to keep the queue at medium or high levels. In the total reward, this term is multiplied by a backlog weight μ_{B}, which controls how strongly the agent is discouraged from letting the queue stay in the higher bins. Together with the hard overflow penalty, this encourages policies that keep the backlog low and avoid persistent congestion.

- **Shaping and safety:** To further enhance the agent's behaviour, three specific shaping terms are introduced.

  - **Idle bonus, ($\boldsymbol{R_{\mathrm{idle}}}$):** It gives a small positive reward when the system is in a completely calm state, and the most economical configuration is used. It is defined as,

$$R_{\mathrm{idle}} = \begin{cases} \rho, \text{if } (\Delta T = 0, bufT = 0, B = 2, q = 0, \tau = 60, f = 0, c = 0), \\ 0, \text{otherwise}, \end{cases}$$

. (14)

where $\rho \approx 0.7$ in the simulator. This encourages staying in a low-power baseline during quiet periods but is kept small so that it does not override the need to react in urgent conditions.

  - **Safety penalty, ($\boldsymbol{safety_pen}$):** punishes under-reaction when the situation is clearly urgent. It is applied when urgency indicators such as $\Delta T = 2$, $bufT = 2$, or $anom = 1$ are present and the action still uses the lowest fidelity $f = 0$ or the slowest interval $\tau = 60$. The penalty is:

$$safety_pen = \begin{cases} \rho, \text{if } (\Delta T = 2 \ \vee \ bufT = 2 \ \vee \ anom = 1) \ \wedge \ (f = 0 \ or \ \tau = 60); \\ 0, \ otherwise, \end{cases} \quad (15)$$

with $\rho \approx 2.0$. This term is larger than the idle bonus so that urgent situations are not ignored.

- **Hard violations, (Ω):** This shaping term represents severe penalties for outcomes that are never acceptable, such as buffer overflow or missing a truly "critical" alert. In these cases, very negative values $\Omega_{\text{overflow}} \ll 0$ or $\Omega_{\text{miss_crit}} \ll 0$ are used. These penalties are chosen so that a single violation outweighs several perfect steps, making such behaviours clearly unattractive to the agent.

The immediate reward $R(s, a, s')$ for taking action $a$in state $s$and moving to $s' = s_{t+1}$ combines all these pieces. With fixed weights $(w_{\text{I}}, w_{\text{T}}, w_{\text{R}}, w_{\text{C}})$ and context multipliers $\mu_{\text{E}}, \mu_{\text{X}}$, the per-step reward is:

$$\boxed{\left\{\begin{array}{c} R(s,a,s') = w_{\text{I}}\, U_{\text{info}}(s,a,s') + w_{\text{T}} U_{\text{time}}(s,a,s') + w_{\text{R}}\, U_{\text{rel}}(s,a,s') + w_{\text{C}}\, U_{\text{crit}}(s,a,s') \\ - \mu_{\text{E}}\, C_{\text{E}}(s,a,s') \\ - \mu_{\text{X}}\, C_{\text{X}}(s,a,s') - \mu_{B}\, C_{\text{backlog}}(s) + R_{\text{idle}} - safety_pen + \Omega \end{array}\right\}} \quad (16)$$

In equation (16), $\Omega \in \{0, \Omega_{\text{overflow}}, \Omega_{\text{miss_crit}}\} \ll 0$ and is strongly negative when a hard violation occurs. Using a single scalar reward to combine several objectives has a known drawback: positive and negative terms can cancel each other, so a high score in one component might hide a problem in another. For this reason, the utilities and costs in equation (16) have been designed to use a straightforward, similar scale (mostly within [0,1], adding big penalties for unsafe actions). This way, the total reward remains interpretable, and the main trade-offs remain visible. This scalar design should be viewed as a pragmatic compromise for this thesis, allowing to keep the learning setup compatible with standard DDQN training and limited computational resources. In the evaluation, information, timeliness, reliability, energy, and backlog indicators are reported separately to assess how well this simple reward scale guides the agent, and to highlight that a more formal vector-valued or multi-objective reward formulation remains an important direction for future work beyond the scope of this thesis.

Typical values used in this thesis are $w_{\text{I}} = 0.30$, $w_{\text{T}} = 0.20$, $w_{\text{R}} = 0.28$, and $w_{\text{C}} = 0.30$. The multiplier μ_{E} is higher when the battery bin B is low, making energy more precious. The multiplier μ_{X} is kept modest so as not to discourage transmission too strongly unless the link is severely constrained. The idle bonus $idle_{\text{bonus}}$ is around 0.7, and ρ is around 2.0. For numerical stability during learning, the combined scalar reward r is clipped to the range

$[-1,1]$: $r \leftarrow \text{clip}(r/4, -1,1)$, so that rare very large penalties or bonuses do not destabilise the neural-network training.

3.3 The PASA Principle and Design Rationale

The controller in this thesis follows the PASA by Kavitha and Suseendran which was proposed for IoT sensor systems dealing with different types of data traffic. The main idea of PASA is simple: increase service when priority is high; decrease service when priority is low. In PASA, A base station allocates collision-free time slots to each node and adaptively sets that node's duty cycle (denoted as *ST*), which controls how long the node stays active and how long it sleeps, based on four things: traffic priority (*PR*), remaining buffer size (*RBS*), remaining energy (*RE*), and required transmit power (*TP*). When traffic is urgent or there are serious risks (for example, high buffer occupancy, low remaining energy, or high transmit power), PASA increases the duty cycle so that it makes the node work for a longer time and send more data. When there aren't much traffic and resources are in good shape, PASA reduces the duty cycle to save energy [21].

This thesis does not implement the full PASA scheduling algorithm (no time-slot allocation or explicit ST computation). Instead, the controller adopts PASA's qualitative principle and incorporates it inside a MDP and a DDQN agent. The MDP state sends signals that show priority levels. The actions are about providing either high or low service. The rewards are given for high service only when the priority is high, and the safety mechanisms enforce "must-serve" conditions for "critical" situations.

- **State as PASA-style priority signs:** The state in this thesis acts as a set of "priority signs" inspired by PASA's inputs (*PR, RBS, RE, TP*). The indicators ΔT, $anom$, $bufT$, B, L, and D together describe how urgent and risky the current situation is. Here, ΔT is the rate-of-change bin for the sensed signal (data activity, analogous to traffic class and data rate), $anom$ is the anomaly flag (emergency-like events), $bufT$ is the backlog bin (similar to *RBS*), B is the battery bin (similar to *RE*), and L and D are the loss and delay bins (capturing link quality and transmit difficulty, similar in spirit to *TP*). When these indicators show high values, the state represents a high-priority condition: information is changing quickly, queues are filling, the link is unreliable, or energy is tight. When they take low values, the state reflects a calm, low-priority condition. There is no special numeric score given for priority; instead, the agent learns how to connect these signs to

the right actions. This follows PASA's idea of using different status indicators to help scheduling but leaves the combination to be learning rather than fixed rules. [22].

- **Actions as Escalation vs. Conservation:** The action in this thesis is $a_t = (q, \tau, f, c)$ as mentioned in equation (5). The current configuration is also stored in the state as $Q_{\text{cur}}, I_{\text{cur}}, F_{\text{cur}}$. These actions are grouped conceptually into two PASA-style categories [23]:

  - "Escalate actions" use short intervals τ (so I_{cur} is in a low bin), high fidelity f (so F_{cur} is in a high bin), higher QoS levels q (reflected in Q_{cur}), and set the critical flag $c = 1$. Such actions correspond to a high duty-cycle mode in PASA: the system spends more airtime and energy to deliver more reliable and detailed data when risk is high.
  - "Conserve actions" use long intervals τ (high I_{cur} bin), low fidelity f (low F_{cur} bin), lower QoS levels q (e.g., $Q_{\text{cur}} = 0$), and keep the critical flag unset $c = 0$. These actions correspond to a low duty-cycle mode: the system saves energy and airtime when priority is low.

  Thus, the discrete action space provides both sides of PASA's trade-off ("escalate" versus "conserve") in terms of QoS, interval, fidelity, and critical flag. These labels are purely conceptual; within this action space the DDQN still has to learn when to favour escalate-type versus conserve-type actions based on the scalar reward and safety constraints.

- **Reward as a dynamic PASA-style incentive:** The reward function turns this PASA principle into a numerical incentive. Utilities for information, timeliness, reliability, and critical correctness are all designed to become larger when the urgency signs (ΔT, $anom$, $bufT$, L, D) indicate a high-priority situation. In those cases, actions that escalate service (shorter τ, higher f, higher q, $c = 1$) receive a larger positive contribution. During calm periods, the same actions receive little extra benefit and are penalized by energy, network strain, and backlog costs. In this way, the scalar reward approximates PASA's adaptive rule: aggressive service is attractive only when justified by priority and risk.
- **Guardrails for Selective Escalation:** Selective escalation is reinforced by the safety guardrails described in the next subsection. Hard masks remove clearly unsafe actions,

such as very long intervals during severe backlog or QoS 0 on very bad links. Soft gating uses a learned criticality estimate $p_{\text{crit}}(s)$ to bias decisions toward escalate-type actions when the state looks critical, and toward conserve-type actions when the state looks calm. Finally, non-bypassable penalties make queue overflow and missed critical events always worse than several perfect steps. Together, these elements use PASA's qualitative rule—escalate under high priority; conserve otherwise—inside an RL controller with a scalar reward, rather than re-implementing PASA's original NS-2 scheduling algorithm.

3.4 Safety Constraints

The reward in this thesis is a single scalar value per step. This design is easy for learning, but it could cause a potential problem: positive contributions from some terms could hide serious penalties from others. For example, a high information or timeliness reward might cancel out a cost related to safety when all terms are simply added together. To prevent such "term cancellation" from producing unsafe behaviour, the framework uses a three-layer safety system. These three layers help the agent to make safe and responsible decisions while preserving the simplicity of a scalar reward.

- **Layer 1 — Hard Action Masks (rule-based feasibility):** The first safety layer consists of hard action masks. These masks make sure of rule-based feasibility. For every state s, a set of feasible actions $\mathcal{A}_{\text{feas}}(s)$ is defined using a mask function $\mathcal{M}(s,a)$. The function $\mathcal{M}(s,a)$ takes the value 1 if action a is allowed in state s, and 0 if the action is forbidden. Formally,

$$\mathcal{A}_{\text{feas}}(s) = \{a \in \mathcal{A} | \mathcal{M}(s,a) = 1\},$$

$$\mathcal{M}(s,a) \in \{0,1\} \qquad (17)$$

  Hard masks implement strict rules. These masks forbid very long delays when the backlog $bufT$ is already high or disallowing extremely low fidelity in clearly "critical" situations. These limitations act before the policy chooses an action, so the agent never encounters actions that are clearly unsafe.

- **Layer 2 — Soft Gating (criticality bands):** The second layer uses soft gating to determine how likely the state is to be critical. An auxiliary network head estimates the

probability that the current state is critical, written as $p_{\text{crit}}(s) \in [0,1]$. This probability works like the priority level in PASA: high values indicate states where escalation should be favoured, and low values indicate states where conservation is preferred. A gating operator $\mathcal{G}$ then refines the feasible set $\mathcal{A}_{\text{feas}}(s)$ using two thresholds θ_{lo} and θ_{hi}.

- When the estimated criticality is high, $p_{\text{crit}}(s) > \theta_{\text{hi}}$, the allowed actions are restricted to those with the critical flag set to $c = 1$.
- When the system appears calm, $p_{\text{crit}}(s) < \theta_{\text{lo}}$, the allowed actions are restricted to those with the critical flag unset, $c = 0$.
- Otherwise, a hysteresis rule keeps the current mode (urgent or calm) to avoid rapid switching. The resulting set of actions after gating is denoted by $\mathcal{A}_{\text{allow}}(s)$.

The policy then picks actions only from $\mathcal{A}_{\text{allow}}(s)$, both during greedy exploitation (for example, $arg\ max$ over Q-values) and during exploration (for example, ε-greedy or NoisyNet noise). Disallowed actions are treated as having very low value at selection time. If $\mathcal{A}_{\text{allow}}(s)$ ever becomes empty, a predefined safe fallback action $a_{\text{safe}}(s)$ is used. This layer does not tell the agent which action is best. It only enforces simple consistency rules between the estimated criticality and the use of the critical flag, so that learning takes place over actions that respect these safety and design constraints. Even though there are limits, the agent still learns from the scalar reward which actions work better or worse over time.

- **Layer 3 — Non-Bypassable Penalties (catastrophic failures):** The final layer consists of non-bypassable penalties for major failures. These are the "deal-breaker" terms in the reward function, represented by Ω. If the buffer overflows, a strong penalty Ω_{overflow} is applied. If a truly "critical" alert is missed, an even stronger penalty $\Omega_{\text{miss_crit}}$ is applied. Both Ω_{overflow} and $\Omega_{\text{miss_crit}}$ are large negative numbers. They are picked so that no combination of positive utility from other terms can make these outcomes profitable to the agent. In other words, a single catastrophic failure is worse than several perfect steps. This makes the learned policy on preventing these failures above everything else.
- **Selection order:** At each time step, the selection process follows a specific order. First, hard masks define $\mathcal{A}_{\text{feas}}(s)$. Second, soft gating based on $p_{\text{crit}}(s)$ produces $\mathcal{A}_{\text{allow}}(s)$. Third, the policy and exploration mechanism pick an action from this final safe set. If

the set becomes empty, the safe fallback action $a_{\text{safe}}(s)$is executed. This step-by-step procedure makes sure that safety is enforced before and after looking at how much the policy is worth.

3.5 Transition Assumptions and Simulator Abstractions

The transition kernel $P(s' \mid s, a)$ in this thesis is implemented using a lightweight simulator that tries to be realistic while also being easy to analyse and control. The simulator captures the main effects that m-atter for the communication problem—such as sensor dynamics, backlog accumulation, link quality, and battery drain. It ignores lower-level details that would add complexity without changing the main findings.

Configuration and Validation:

- **The simulator follows the Markov property:** the next state s' depends only on the current state s and the chosen action a, not on the full past history. Random disturbances in the environment, such as noise on the signal or fluctuations in latency, are modelled as independent draws at each step. Environment parameters such as baseline loss rates or battery coefficients stay fixed during a single run, so the system is stable while it runs.
- **Signal model ($\boldsymbol{\Delta T}$):** The temperature signal used to compute ΔT follows a smooth daily pattern combined with random noise. A typical model is a diurnal waveform with a certain period, amplitude, and Gaussian noise level. The quantity ΔT is then derived as the residual of an Exponential Moving Average (EMA) and is binned into the discrete values $\{0,1,2\}$ used in the state. Parameters such as the waveform period, the amplitude, the noise standard deviation, and the EMA factor α control how quickly the signal varies.
- **Anomaly flag ($\boldsymbol{anom}$)**: The anomaly flag $anom$ is generated by a simple threshold on a standardized residual. When the deviation of the measured value from its expected level exceeds a chosen threshold, the simulator sets $anom = 1$. When the deviation is within normal bounds, $anom = 0$. The threshold can be tuned so that anomalies are rare but not impossible.
- **Backlog dynamics ($\boldsymbol{bufT}$):** Backlog dynamics for $bufT$ are modelled by a FIFO (first-in–first-out) queue of "bytes to send". Each step, the queue grows by a production term that depends on activity and fidelity: higher fidelity f produces more bytes. At the

same time, the queue drains at a rate that depends on the interval τ and the link capacity. Shorter intervals and favourable conditions drain faster, while longer intervals and poor conditions drain more slowly. The queue is capped at a fixed maximum capacity. After updating, the queue occupancy is mapped into the backlog bin $bufT \in \{0,1,2\}$.

- **Link impairment model (L, D):** The link impairment model for the loss bin L and the delay bin D captures how network conditions vary over time. For each transmitted message, latency is drawn from a distribution whose baseline depends on QoS level q and is further influenced by the interval τ and fidelity f. Loss probability increases when QoS is low, latency is high, or fidelity is high. Messages that arrive later than a fixed deadline (for example, 300 ms) are considered "late" even if they are eventually delivered. Recent loss and delay outcomes are tracked over a sliding window of fixed size, such as the last 20 transmissions, and then summarized into the ternary bins $L \in \{0,1,2\}$ and $D \in \{0,1,2\}$.
- **Battery model (B):** The battery model for B tracks remaining energy in milliampere-hours. At each step, the state of charge decreases by a base transmit current plus extra overheads that depend on QoS q and fidelity f, along with duty-cycle terms linked to the interval τ. Higher QoS and higher fidelity cost more, and shorter intervals mean more frequent consumption. The continuous state-of-charge value is then discretized into the battery bin $B \in \{0,1,2\}$.
- **Criticality label (ground truth):** A ground-truth criticality label is also generated inside the simulator. A step is marked "critical" if any hard trigger fires, for example if $\text{anom} = 1$, $bufT = 2$, $L = 2$, or $D = 2$. If none of these hard triggers are active, a composite severity score is computed as:

$$\Delta T + bufT + (2 - B) + L + D + anom.$$

  If this score exceeds a threshold called CRIT_THRESH (in the implementation), the step is also labelled "critical". This internal label drives both safety masks and the training of the criticality head $p_{\text{crit}}(s)$.

- **Action application:** Based on the equation (5), the action $a = (q, \tau, f, c)$ affects transitions in several ways. The QoS level q changes both reliability and overhead by influencing the loss and delay distributions and the energy spent per transmission. The interval τ changes how quickly the backlog drains and how fresh the samples are. The

fidelity level f changes the payload size, which in turn affects backlog, link strain, and information utility. The critical flag c acts as a short-lived command for urgent publication or backlog flushing, temporarily prioritizing some transmissions. In an online deployment, these levers would be applied by the bridge and server at publish or command points to control the end-to-end behaviour of the communication pipeline.

4.

RL-based Framework

This chapter describes how the RL framework is turned into a working agent or a controller for the hybrid CoAP–MQTT IoT system. The main goal is to create a context-aware transfer policy that changes MQTT QoS q, reporting interval τ, payload fidelity f, and the critical flag c based on what is happening in the system real-time. The chapter explains that how raw measurements are converted into a state vector s_t, how the agent picks an action a_t, how the environment reacts with a new state s_{t+1} and a reward $R(s_t, a_t, s_{t+1})$. It helps to make better decisions over time using this information.

This chapter describes one cycle of making a decision. First, the system architecture and the abstractions for state $\mathcal{S}$ and action $\mathcal{A}$ are connected to real data such as queue length, battery level, loss, delay, and anomalies. Next, the design of the reward function $\mathcal{R}$ is shown in practice, including how the shaped scalar reward balances utilities (information, timeliness, reliability, correct critical handling) against costs (energy, network strain, backlog) and how safety mechanisms such as hard masks and gating sets restrict the allowed actions in risky conditions. Finally, the chapter describes how a dueling DDQN is used as the function approximator: this network gives an estimate for each discrete action, is trained with n-step targets to look at several steps ahead and uses prioritized replay to sample the most informative past transitions. These parts work together to implement a PASA-guided controller that adapts its decisions to the current priority level while keeping energy usage and safety in mind, making it suitable for constrained IoT devices.

4.1 System Architecture Overview

The RL controller in this thesis runs on top of a hybrid CoAP–MQTT stack. At the edge, a virtual sensor exposes a CoAP resource and behaves like a small battery-powered device. In the core, an MQTT broker distributes messages between components. A bridge service sits between them and translates CoAP requests and responses into MQTT publications and control commands. All services run locally as separate processes so that the same setup can be used both for experiments and for reproducible evaluation without interference from external networks.

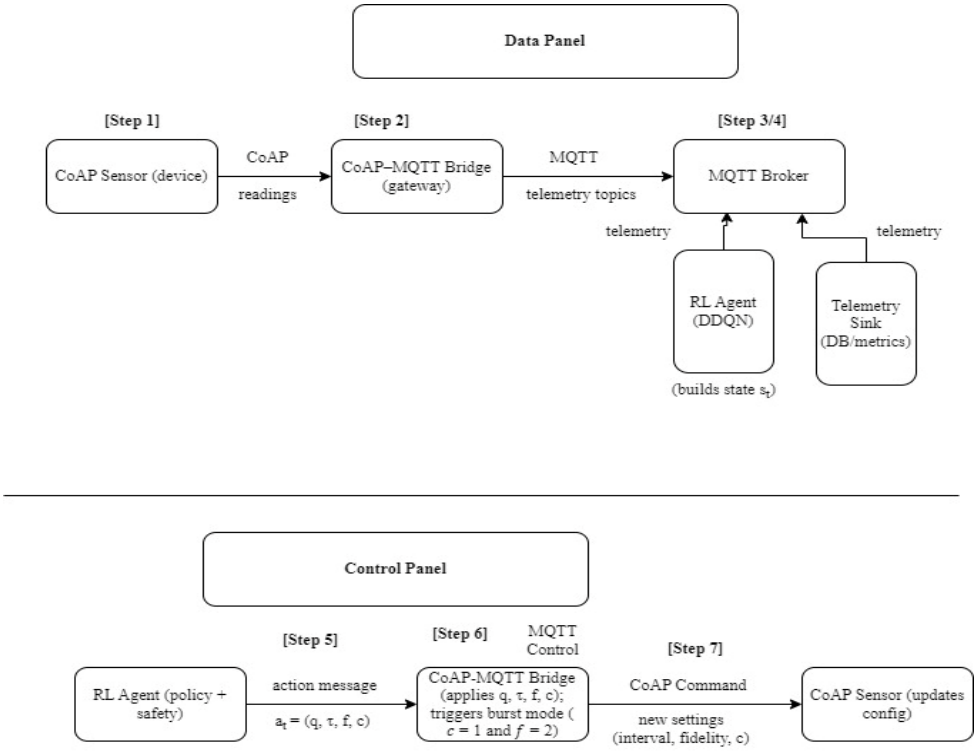

Figure 4. 1 Hybrid CoAP–MQTT system architecture with data plane, control plane, and RL (MDP) loop

Figure 4.1 shows the overall architecture that can be understood as a repeated loop. This loop connects sensing, decision, and actuation. One decision cycle proceeds as follows

- **Step 1 – Sensing at the CoAP sensor device (data creation):** The CoAP sensor acts as a digital twin of a physical node. It maintains an internal configuration (reporting interval, fidelity, and critical flag) and generates temperature readings according to this configuration. Each new reading is made available through a CoAP resource. At the same time, the sensor tracks local quantities such as remaining battery and backlog of unsent work.
- **Step 2 – Bridge polling and telemetry publish (data plane):** The CoAP–MQTT bridge periodically polls the CoAP resource to fetch the latest reading. For each poll, it:
  - reads the sensor value and local indicators (for example backlog and battery),
  - shapes the payload according to the current fidelity setting f (more or less detail per message), and
  - publishes a telemetry message to the MQTT broker with the selected QoS level q.

This step moves information from the CoAP edge into the MQTT core. The bridge is therefore the place where the abstract action $a_t = (q, \tau, f, c)$ starts to affect real messages on the network.

- **Step 3 – Logging at the telemetry sink (data recording):** A telemetry sink subscribes to the MQTT topics carrying sensor data and control decisions. It stores raw measurements, state indicators (such as bins for ΔT, backlog, loss, delay, and battery), and the chosen actions in a time-series database. These records provide the basis for later analysis, plotting, and comparison between different training runs or policies.
- **Step 4 – State reconstruction at the RL agent (from messages to s_t):** The RL agent subscribes to the same telemetry topics as the sink. From the incoming messages it reconstructs the current state s_t. This state is built from the nine indicators defined in the problem formulation chapter: rate of change and anomaly (urgency), backlog, loss, and delay (path pressure), and battery plus current configuration $(I_{\text{cur}}, Q_{\text{cur}}, F_{\text{cur}})$ (resource and policy context). These indicators are binned and encoded into the 25-dimensional state vector that feeds the neural network.
- **Step 5 – Policy evaluation and action selection (decision):** Given s_t, the neural network estimates a value for each of the 54 possible actions. Safety masks and gating are then applied to remove actions that are infeasible or inconsistent with the estimated criticality. Inside the remaining safe set, the agent selects an action $a_t = (q, \tau, f, c)$ according to the learned policy: mostly the best-valued action, with occasional exploration. At the same time, the auxiliary head updates its estimate of how likely the current step is critical, based on the same indicators.
- **Step 6 – Control publication via MQTT (control plane):** The selected action a_t is encoded into a small control message (for example, fields for QoS level, reporting interval in seconds, payload fidelity, and critical flag) and published to a dedicated MQTT control topic. From the broker's perspective this message is just another publication, but it belongs to the control plane rather than the data plane.
- **Step 7 – Bridge actuation and CoAP command (closing the loop):** The bridge subscribes to the control topic. When a new control message arrives, the bridge:
  - updates its local view of q, τ, f, c,
  - uses τ and c to adjust how often the CoAP resource is polled and whether urgent flushing is needed, and
  - forwards any configuration changes to the CoAP sensor using a CoAP request.

The sensor then updates its internal configuration and uses the new settings when generating readings for the next cycle.

4.2 State Encoding and Feature Pipeline

For the RL agent, the IoT system is not seen as raw temperatures, voltages, or packet traces. Instead, each decision is based on a small summary of the current situation called the state s_t. This state is built from nine indicators that were defined earlier in the problem formulation. Together, these indicators answer three practical questions: how urgent the data is, how loaded the communication path is, and how much resource is left along with the current configuration. Data urgency is captured by the rate of change ΔT and the anomaly flag $anom$. Path load and timeliness are captured by the backlog bin $bufT$ and the link-quality bins L (packet loss) and D (delay). Resource and context are captured by the battery bin B and the current settings $(I_{\text{cur}}, Q_{\text{cur}}, F_{\text{cur}})$, which describe the active interval level, QoS level, and payload fidelity level.

These nine indicators are calculated at each step using live measurements and internal counters. The CoAP sensor and the bridge keep track of delay and loss, count how many messages are waiting in the queue, check the battery level in milliampere-hours, and monitor how quickly the temperature changes. Each raw quantity is then mapped into a small number of bins that represent low, medium, or high levels, for example $bufT \in \{0,1,2\}$ for low/medium/high queue occupancy. Similarly, ΔT, L, D, B, I_{cur}, Q_{cur}, and F_{cur} each take one of three values $\{0,1,2\}$, and the anomaly indicator $anom \in \{0,1\}$ shows whether a simple detector has flagged an abnormal situation. This step changes messy, continuous signals into clear, separate categories that are easier to understand and learn from.

After the nine indicators are calculated, they are changed into a fixed 25-dimensional numerical vector suitable for the neural network. The eight three-level indicators ΔT, $bufT$, B, L, D, I_{cur}, Q_{cur}, and F_{cur} are each converted using one-hot encoding. One-hot encoding means that each indicator is represented by a small vector of length three, with exactly one entry equal to 1 and the other two equal to 0. For example, if $bufT = 1$, its encoded form is $[0,1,0]$, and if $bufT = 2$, its encoded form is $[0,0,1]$. This avoids inventing artificial distances between categories and tells the network clearly which bin is active. The anomaly flag $anom$ is already binary, so it is kept as a single feature with value 0 or 1. Combining the eight one-hot vectors ($8 \times 3 = 24$ features) with the single anomaly bit produces a 25-dimensional state vector. This

vector used directly in the DDQN and works the same way in both the offline simulator and the online system. This means that both training and deployment work with the same kind of information.

For the worked example used in this chapter, the bins take the values $\Delta T = 2$, $bufT = 1$, $B = 1$, $L = 1$, $D = 2$, $I_{\text{cur}} = 1$, $Q_{\text{cur}} = 1$, $F_{\text{cur}} = 1$, and $anom = 1$. Here ΔT is the bin index for the rate-of-change indicator ΔT. Each ternary indicator activates one position in its three-element one-hot block, and the anomaly flag sets the final bit. The result is a 25-dimensional vector with eight isolated "1" entries (one per ternary indicator) and one extra "1" for the anomaly. From the network's point of view, this vector is the complete description of the current world state s_t on which all further decisions and value estimates are based.

4.3 Action Space Realization

Once the state vector s_t is constructed, the RL agent responds by choosing an action at time, t as mentioned in equation (5):

$$a_t = (q, \tau, f, c)$$

In the running system, this abstract tuple is encoded as a small JSON control message and sent through MQTT so that the rest of the pipeline can apply the chosen settings in a consistent way.

A typical control message contains four fields:

```
1. {
2.   "qos": 1,
3.   "interval_s": 30,
4.   "fidelity": 2,
5.   "critical": 1
6. }
```

Here, qos carries the value of q, $interval_s$ carries τ, $fidelity$ carries f, and $critical$ carries c. The CoAP sensor interprets $interval_s$ as the new sampling and publishing period, so smaller values make the sensor send data more frequently and larger values slow it down. The bridge interprets fidelity as a choice of how much detail to include in each message: low f selects compact, coarse summaries, while high f allows richer payloads. The bridge also uses qos when publishing telemetry to the MQTT broker, so higher q increases delivery assurance at the cost of extra acknowledgements and overhead. Finally, the critical flag tells the bridge

that the current situation is urgent when $c = 1$. In that case, the bridge treats the next messages as high priority and may flush queued data more aggressively; when $c = 0$, the system behaves in a normal, frugal mode.

In practice, all actions belong to the same discrete set built from the allowed values of each component. The QoS level q can take values $\{0,1,2\}$, the interval τ can take values $\{5,30,60\}$ seconds, the fidelity level f can take values $\{0,1,2\}$, and the critical flag c can take values $\{0,1\}$. Combining these choices yields 54 possible actions in total. Each action is one specific combination of reliability, sending rate, information richness, and urgency, and the agent's task is to pick the one that best fits the current state.

To make these options easier to understand, this chapter focuses on three representative actions that appear often in the learned policy:

- **Action A ("frugal")**: $q = 0, \tau = 60, f = 0, c = 0$. This action uses the lowest QoS, the slowest reporting interval, minimum payload detail, and no critical flag. It represents an economical mode suitable for calm conditions, with low energy use and minimal network load, but weaker delivery guarantees and slower reaction times.
- **Action B ("balanced")**: $q = 1, \tau = 30, f = 1, c = 0$. This action takes a middle-ground approach, with moderate QoS, a medium reporting rate, and medium payload fidelity. It trades off energy, reliability, and timeliness in a balanced way, without explicitly marking the state as "critical".
- **Action C ("escalate")**: $q = 2, \tau = 5, f = 2, c = 1$ .This action pushes the system into a highly responsive mode: highest QoS, fastest reporting, highest fidelity, and the critical flag set. It is designed for situations where data is changing quickly or where the risk of missing important information is high, even though it consumes more energy and uses more airtime.

In the live system, each time the agent outputs an action a_t, the corresponding JSON message is published once, and the CoAP sensor and bridge update their internal settings accordingly. If the same tuple is issued again, the effective configuration remains unchanged, so repeated commands do not create extra side effects. Every applied action is also recorded together with the surrounding state and outcomes, so that later analysis can trace how different choices of q, τ, f, and c affected backlog, delay, loss, and energy consumption. In this way,

the abstract action space $\mathcal{A}$ becomes a concrete set of levers that shape the behaviour of the CoAP–MQTT pipeline step by step.

4.4 Practical Example: One Decision Step from Signal to Reward

The previous sections described how the system represents the world as a state $s_t$and how the agent chooses an action $a_t = (q, \tau, f, c)$ as mentioned in equation (5). This section now follows one concrete decision step all the way from the sensed conditions to the numerical effects on backlog, battery, delay, loss, and, later, reward. The same equations that drive the simulator and the live system are used here, but with small, rounded numbers so each step can be followed easily. The goal is to show, in a single snapshot, how three different actions shape the queue, the energy cost, and the network behaviour under exactly the same starting conditions.

4.4.1 Setup of the Example

Consider a single decision step at time t, with the system already running for some time. The backlog queue currently holds $q_t = 120$ units of work, out of a maximum capacity $C = 300$. This corresponds to a medium backlog bin $bufT_t = 1$ (the first third of the queue capacity represents low pressure, the second third medium pressure, and the last third high pressure).

The battery has delivered part of its charge and now sits at 600 mAh out of a nominal capacity $B_{cap} = 1000$ mAh. This corresponds to 60% state-of-charge, which lies in the medium battery bin $B_t = 1$.

Recent communication history shows moderate packet loss and high delay. The loss bin is $L_t = 1$ (medium), and the delay bin is $D_t = 2$ (high). This indicates that messages are still getting through, but the link is somewhat stressed, and responses may be slow.

On the sensing side, the temperature has just undergone a sharp change. The deviation between the current temperature and its smoothed EMA is large, giving a high change bin $\Delta T_t = 2$. In addition, the anomaly detector has fired, so the anomaly flag is $anom_t = 1$. Together, these two indicators say that something unusual and potentially important is happening at the sensor. Based on the equation (4), The state vector at the moment, t can therefore be summarized as

$$s_t = (\Delta T_t, bufT_t, B_t, L_t, D_t, I_{\text{cur},t}, Q_{\text{cur},t}, F_{\text{cur},t}, anom_t)$$

with the urgency-related entries $\Delta T_t = 2$ and $anom_t = 1$, the congestion-related entries $bufT_t = 1$, $L_t = 1$, $D_t = 2$, and the battery and configuration entries $B_t = 1$, $I_{\text{cur},t}$, $Q_{\text{cur},t}$, $F_{\text{cur},t}$ inherited from the previous action. From this state, three candidate actions are considered as representative choices:

- **Action A ("frugal")**: $q = 0, \tau = 60, f = 0, c = 0$.
- **Action B ("balanced")**: $q = 1, \tau = 30, f = 1, c = 0$.
- **Action C ("escalate"):** $q = 2, \tau = 5, f = 2, c = 1$.

These three actions are examples drawn from the full set of 54 possible combinations of QoS, interval, fidelity, and critical flag. In the rest of this section, the environment model is applied once to each of these actions to show how backlog, battery, delay, and loss evolve within a single step.

4.4.2 Backlog Update (Step 1)

The backlog represents how much unsent information hasn't been sent yet and is waiting to be sent. At each step, the queue gets bigger when new information comes in and shrinks because the system sends some of that information out. The backlog update follows a simple balance:

$$q_{t+1} = clip(q_t + a(\Delta T_t) - d_{\text{cap}}(f_t, \tau_t, burst_t), 0, C) \qquad (18)$$

where:

- $a(\Delta T_t)$is the production term: how much new "work" is added due to content change. For temperature, this is chosen as

  $$a(\Delta T) \in \{0.2, 0.6, 1.2\}$$

  for low, medium, and high change. In this example, $\Delta T = 2$ (high), so $a(\Delta T) = 1.2$ units are added to the queue.

- $d_{\text{cap}}(f_t, \tau_t, \text{burst}_t)$ is the drain capacity: how much can be shipped in this step. It depends on the interval τ_t, the fidelity f_t, and a "burst" mode:

$$d_{\text{cap}} = \kappa_{\text{F}}(f_t) \cdot \frac{5}{\tau_t} \cdot (1 + \text{burst}_t) \qquad (19)$$

  where $\kappa_{\text{F}}(0) = 0.5, \kappa_{\text{F}}(1) = 1.0, \kappa_{\text{F}}(2) = 2.0]$ scales with fidelity, and $\text{burst}_t = 1$ only if the action asks for critical and max fidelity $c_t = 1, f_t = 2$; otherwise $\text{burst}_t = 0$.

- $\text{clip}(\cdot, 0, C)$ keeps the backlog between 0 and capacity $C = 300$. After updating the raw backlog q_{t+1}, the queue is mapped back to a bin:

$$Q_{t+1} = \begin{cases} 0, q_{t+1} < C/3 \\ 1, C/3 \leq q_{t+1} < 2C/3 \\ 2, q_{t+1} \geq 2C/3 \end{cases} \qquad (20)$$

  For the three actions, the calculation proceeds as follows (starting from $q_t = 120$, $\Delta T = 2$):

- **Action A:** $q = 0$, $\tau = 60$, $f = 0$, $c = 0$. Here $f = 0$ and $c = 0$, so $\text{burst} = 0$. The drain capacity is based on the equation (19),

$$d_{\text{cap}}^{\text{A}} = 0.5 \cdot \frac{5}{60} \cdot 1 \approx 0.0417$$

  The next backlog is then obtained from the balance based on the equation (18)

$$q_{t+1}^{\text{A}} = q_t + a(\Delta T_t) - d_{\text{cap}}^{\text{A}} = 120 + 1.2 - 0.0417 \approx 121.16$$

  The physical queue capacity is $C = 300$, which is split into three equal ranges for the backlog bin based on equation (20):

  - **Bin 0 (low):** $0 \leq q_{t+1} < C/3 = 100$
  - **Bin 1 (medium):** $100 \leq q_{t+1} < 2C/3 = 200$
  - **Bin 2 (high):** $q_{t+1} \geq 200$

Since $q_{t+1}^{\mathrm{A}} \approx 121.16$ lies between 100 and 200, the backlog bin after Action A is so $Q_{t+1}^{\mathrm{A}} = 1$ (moderate backlog).

- **Action B:** $q = 1, \tau = 30, f = 1, c = 0$. Here $f = 1$, burst $= 0$. The drain capacity from the equation (19) is

$$d_{\mathrm{cap}}^{\mathrm{B}} = 1.0 \cdot \frac{5}{30} \cdot 1 \approx 0.1667$$

So according to equation (20), it gives,

$$q_{t+1}^{\mathrm{B}} = 120 + 1.2 - 0.1667 \approx 121.03$$

Again, in the same medium bin, $Q_{t+1}^{\mathrm{B}} = 1$.

- **Action C:** $q = 2, \tau = 5, f = 2, c = 1$. Here the critical flag is set and fidelity is max, so burst $= 1$. So based on the equation (19), The drain capacity becomes

$$d_{\mathrm{cap}}^{\mathrm{C}} = 2.0 \cdot \frac{5}{5} \cdot (1 + 1) = 2.0 \cdot 1 \cdot 2 = 4.0$$

and the updated backlog is

$$q_{t+1}^{\mathrm{C}} = 120 + 1.2 - 4.0 = 117.20$$

This is still in the medium range, so $Q_{t+1}^{\mathrm{C}} = 1$, but the queue is clearly moving down.

In summary, the high-intensity action C actively drains the queue (about 3 units less than before), whereas the frugal action A allows the backlog to edge upwards. The balanced action B lies in between but, in this particular numeric step, still leaves the queue slightly larger than C does.

4.4.3 Energy per Step (Step 2)

Energy cost in this system is based on how long the radio stays active within each interval, and how much current it draws while transmitting. This is captured by the duty factor u_t, the transmit current $I_{\mathrm{tx}}(a_t)$, the average current I_{avg}, and the resulting battery drain ΔmAh_t. The duty factor expresses what fraction of the interval is spent sending data. It depends on the reporting interval τ_t, the payload fidelity f_t, and whether the burst mode is active.

The burst mode is represented by a binary flag [2]

$$\beta_t = \chi\{c_t = 1 \wedge f_t = 2\} \qquad (21)$$

which is 1 only when the action marks the situation as "critical" and requests maximum fidelity. When burst mode is on, the node is allowed to send more in that step, at the cost of higher airtime. The duty factor u_t expresses what fraction of the reporting interval is spent sending data. It depends on the reporting interval τ_t, the payload fidelity f_t, and whether burst mode is active

$$u_t = min\left(1, \frac{t_{\mathrm{tx}}^{\mathrm{base}} \phi_{\mathrm{f}}[f_t]}{\tau_t}(1 + \beta_t)\right) \qquad (22)$$

with $t_{\mathrm{tx}}^{\mathrm{base}} = 0.1$ as a base transmit time per message and $\phi_{\mathrm{f}}(f_t)$ is a dimensionless airtime multiplier that depends on fidelity:

$$\phi_{\mathrm{f}}(0) = 0.6, \qquad \phi_{\mathrm{f}}(1) = 1.0, \qquad \phi_{\mathrm{f}}(2) = 1.6.$$

Higher fidelity or burst mode increases u_t, and a longer interval τ_t decreases it. The min (1,·) ensures that the duty factor never exceeds 100% of the interval. Given u_t, the transmit current is

$$I_{\mathrm{tx}}(a_t) = I_{\mathrm{tx}}^{\mathrm{base}} \omega_{\mathrm{q}}(q_t) u_t \qquad (23)$$

with $I_{\mathrm{tx}}^{\mathrm{base}} = 120$ and $\omega_{\mathrm{q}}(0) = 1.00, \omega_{\mathrm{q}}(1) = 1.15, \omega_{\mathrm{q}}(2) = 1.35\}$ as small overhead factors for QoS levels $q = \{0,1,2\}$. The total average current for the interval combines a baseline current and sensing current with this transmit current:

$$I_{\mathrm{avg}} = I_{\mathrm{base}} + I_{\mathrm{sense}} + I_{\mathrm{tx}}(a_t) \qquad (24)$$

where $I_{\mathrm{base}} = 5$ mA and $I_{\mathrm{sense}} = 2$ mA.

To reflect that higher currents drain the battery slightly more than proportionally, a mild Peukert-like exponent is used. The per-step battery loss in milliampere-hours is

[2] Here $\chi\{\cdot\}$denotes the indicator function of a_t: for any condition (or event) E, $\chi\{E\} = 1$ if E is true and $\chi\{E\} = 0$ otherwise.

$$\Delta mAh_t = \frac{I_{\text{avg}}^{\text{k}}}{3600}, k = 1.07 \qquad (25)$$

Using these formulas, the three example actions give the following values:

- **Action A ($q = 0, \tau = 60, f = 0, c = 0$):** Here $f = 0$ and $\beta_t = 0$, so, based on the equation (22),

$$u = \frac{0.1 \cdot 0.6}{60} = 0.001.$$

With $q = 0$ and $\omega_{\text{q}}(0) = 1.00$, the equation (23) and (24) give,

$$I_{\text{tx}} = 120 \cdot 1.00 \cdot 0.001 = 0.12 \text{ mA}$$

$$I_{\text{avg}} = 5 + 2 + 0.12 = 7.12 \text{ mA}$$

These yield from the equation (25)

$$\Delta mAh \approx \frac{7.12^{1.07}}{3600} \approx 0.00227 \text{ mAh}$$

- **Action B ($q = 1, \tau = 30, f = 1, c = 0$):** Here $f = 1$ and $\beta_t = 0$, so

$$u = \frac{0.1 \cdot 1.0}{30} \approx 0.00333.$$

With $q = 1$ and $\omega_{\text{q}}(1) = 1.15$,

$$I_{\text{tx}} \approx 120 \cdot 1.15 \cdot 0.00333 \approx 0.46 \text{ mAh}.$$

$$I_{\text{avg}} \approx 5 + 2 + 0.46 = 7.46 \text{ mAh}.$$

This gives from the equation (25)

$$\Delta mAh \approx \frac{7.46^{1.07}}{3600} \approx 0.00239 \text{ mAh}.$$

- **Action C (q=2, $\tau = 5$, f=2, c=1):** Here $f = 2$ and $\beta_t = 1$, so burst mode is active, the same way from the equation (22) as follows:

$$u = \frac{0.1 \cdot 1.6}{5} \cdot 2 = 0.064.$$

With $q = 2$ and $\omega_{\mathrm{q}}(2) = 1.35$,

$$I_{\mathrm{tx}} = 120 \cdot 1.35 \cdot 0.064 \approx 10.37 \text{ mAh},$$

$$I_{\mathrm{avg}} \approx 5 + 2 + 10.37 = 17.37 \text{ mAh}$$

Based on the equation (25), this leads to

$$\Delta mAh \approx \frac{17.37^{1.07}}{3600} \approx 0.00589 \text{ mAh}$$

These values show the intended trade-off: in this step, Action C uses roughly $0.00589/0.00227 \approx 2.6$ times more energy than Action A. The escalated action therefore pays a higher energy price in exchange for faster, richer communication, while the frugal action conserves battery at the cost of slower updates and weaker reliability.

4.4.4 Delay, Loss, and On-Time Delivery (Step 3)

Alongside energy and backlog, the communication quality is characterized by end-to-end delay and packet loss. The delay δ_t in milliseconds is composed of a QoS-dependent base component and extra overhead terms for interval and fidelity. The base delay mean is modelled as

$$\mu(q_t) = \mu_0 + \Delta_{\mu}(2 - q_t), \quad (26)$$

where $\mu_0 = 100$ ms is the base delay for the highest QoS level ($q_t = 2$), and $\Delta_{\mu} = 50$ ms is the extra delay per step down in QoS. These values were chosen so that base delays fall in a realistic 100–200 ms range for low-power IoT links and so that QoS levels differ enough to affect the delay bins and the 300 ms "on-time" threshold. So higher QoS (larger q_t) reduces the baseline delay. The total delay is then

$$\delta_t = clip\left(\mu(q_t) + \frac{1000}{\tau_t} + 12\, f_t, 0, 800\right) \quad (27)$$

where the term $1000/\tau_t$ captures the effect of the reporting interval on effective latency (shorter τ_t lowers this component) and $12f_t$ represents the extra time for larger payloads at higher fidelity. The clip function limits the result to a reasonable range $[0,800]$ ms.

The probability of packet loss p_{loss} increases when QoS is low, delay is high, or fidelity is high. It is approximated by

$$p_{\text{loss}} = clip\left(0.18(2 - q_t) + 0.008\frac{\delta_t}{100} + 0.012f_t, 0,1\right) \quad (28)$$

A delivered packet is considered "on-time" only if it arrives and $\delta_t < 300$ms. For the three example actions, the numbers become:

- **Action A (q=0, $\boldsymbol{\tau} = 60$, f=0):** With $q = 0$, based on the equation (26)

  $$\mu = 100 + 50(2 - 0) = 200\text{ms}$$

  The total delay is based on the equation (27)

  $$\delta = 200 + \frac{1000}{60} + 12 \cdot 0 \approx 200 + 16.7 + 0 = 216.7\text{ms}$$

  which lies below the 300 ms on-time threshold. The loss probability is based on the equation (28)

  $$p_{\text{loss}} \approx 0.18 \cdot 2 + 0.008 \cdot \frac{216.7}{100} + 0.012 \cdot 0 \approx 0.36 + 0.017 \approx 0.377$$

  The probability of a packet being delivered and on time is therefore approximately $1 - p_{\text{loss}} \approx 0.623$.

- **Action B (q=1, $\boldsymbol{\tau} = 30$, f=1):** With $q = 1$, the same way,

  $$\mu = 100 + 50(2 - 1) = 150 \text{ ms}$$

  The total delay is

  $$\delta = 150 + \frac{1000}{30} + 12 \cdot 1 \approx 150 + 33.3 + 12 = 195.3 \text{ ms}$$

again below 300 ms. The loss probability becomes

$$p_{\text{loss}} \approx 0.18 \cdot 1 + 0.008 \cdot \frac{195.3}{100} + 0.012 \cdot 1 \approx 0.18 + 0.016 + 0.012 \approx 0.208$$

Hence the on-time delivery probability is approximately $1 - 0.208 = 0.792$.

- **Action C (q=2, $\tau = 5$, f=2):** With $q = 2$, the same ways

$$\mu = 100 + 50(2 - 2) = 100\text{ms}$$

The total delay is now

$$\delta = 100 + \frac{1000}{5} + 12 \cdot 2 = 100 + 200 + 24 = 324\text{ms}$$

which exceeds the 300 ms threshold. The loss probability is

$$p_{\text{loss}} \approx 0.18 \cdot 0 + 0.008 \cdot \frac{324}{100} + 0.012 \cdot 2 \approx 0 + 0.026 + 0.024 \approx 0.050$$

This means the packet is delivered with high probability (about 95%), but those deliveries are typically late according to the 300 ms criterion. In this simplified step, the "on-time" probability is therefore treated as zero, because δ is above the threshold even when delivery succeeds.

These results highlight the basic timing and reliability trade-offs built into the model. Actions A and B often deliver messages on time, but suffer from higher loss rates, especially for action A with QoS 0 and a long interval. Action C, in contrast, almost never loses a packet due to its high QoS and very frequent sending, but in this configuration the additional per-message delay and processing overhead push the total delay above 300 ms, so messages tend to arrive late. Later sections combine these effects with backlog and energy into a single scalar reward, so that the agent can learn when an aggressive action like C is justified and when a more frugal action is preferable.

4.5 Reward Computation in Using the Example

The previous subsections showed how the three candidate actions A, B, and C change backlog, battery usage, and link quality for the same starting state ($\Delta T = 2$, $bufT = 1$, $B = 1$, $L = 1$, $D = 1$, $anom = 1$). This state clearly represents an urgent situation: the signal just spiked ($\Delta T = 2$), an anomaly was detected ($anom = 1$), the backlog is growing ($bufT = 1$), and the link delay is high ($D = 2$).

The RL agent does not see all these effects separately in its update rule. Instead, the environment combines them into a single scalar reward r_t for each executed action $a_t = (q_t, \tau_t, f_t, c_t)$. This reward is built from four "utility" terms that capture desirable behaviour and from cost and penalty terms that discourage wasteful or unsafe choices. This section shows, using the running example, how these already defined terms are actually evaluated for actions A, B, and C and then combined into a final score.

4.5.1 Utilities (What is Rewarded)

The reward contains four utility components that measure how helpful the action is in this step:

- **Information utility, U_{info}:** This term measures how much useful information is delivered. It increases with payload fidelity f and with demand for new information. In this example, demand is already high because the temperature change is large ($\Delta T = 2$) and the backlog is non-empty ($bufT = 1$), so the demand part can be treated as "maxed out". The illustrative formula used here is, from the equation (6),

$$U_{\text{info}} = \frac{f+1}{3} \quad (29)$$

  so higher fidelity maps to a higher information score.

- **Timeliness utility, U_{time}:** This term reflects how fresh the data is. It decreases when the interval τ is long and when the delay bin D is high. For this example, a simple decreasing function is used from the equation (7):

$$U_{\text{time}} = \frac{1}{1+\tau/5+D/2}, \quad (30)$$

with $D = 2$. Shorter intervals and lower delay bins give higher values.

- **Reliability utility, U_{rel}:** This term rewards appropriate reliability when the link is stressed. In this example, high delay $D = 2$ is treated as a "severe link" condition, where QoS 2 is preferred. The utility is set to

$$U_{\text{rel}} = \begin{cases} 1, & \text{if } q \text{ matches the required level (here } q = 2), \\ 0, & \text{otherwise.} \end{cases} \quad (31)$$

- **Critical handling utility, U_{crit}:** This term checks whether the critical flag $c$matches the true situation. In this step, the state is considered "critical" because $\Delta T = 2$ and $anom = 1$. The utility is based on the equation (10)

$$U_{\text{crit}} = \begin{cases} +0.5, & \text{if } c \text{ correctly signals a critical step,} \\ -0.5, & \text{if } c \text{ fails to signal it (or signals it unnecessarily).} \end{cases} \quad (32)$$

Using these simple formulas, the utilities for the three actions become:

- **Action A (q=0, $\tau = 60$, f=0, c=0)**
  - $U_{\text{info}}(A) = (0 + 1)/3 \approx 0.333$, based on the equation (29).
  - $U_{\text{time}}(A) = 1/(1 + 60/5 + 2/2) = 1/(1 + 12 + 1) = 1/14 \approx 0.071$, based on the equation (30)
  - $U_{\text{rel}}(A) = 0$, because the severe link condition prefers $q = 2$, not $q = 0$, based on the equation (31)
  - $U_{\text{crit}}(A) = -0.5$, because the state is critical but $c = 0$, based on the equation (32)

- **Action B (q=1, $\tau = 30$, f=1, c=0),** based on the same as above:
  - $U_{\text{info}}(B) = (1 + 1)/3 \approx 0.667$.
  - $U_{\text{time}}(B) = 1/(1 + 30/5 + 2/2) = 1/(1 + 6 + 1) = 1/8 = 0.125$.
  - $U_{\text{rel}}(B) = 0$. because the QoS (1) still does not match the preferred level (2).
  - $U_{\text{crit}}(B) = -0.5$, because $c = 0$ in a critical situation.

- **Action C (q=2, $\tau = 5$, f=2, c=1),** based on the same as above:
  - $U_{\text{info}}(C) = (2 + 1)/3 = 1.0$; this is the maximum value because fidelity is highest.
  - $U_{\text{time}}(C) = 1/(1 + 5/5 + 2/2) = 1/(1 + 1 + 1) = 1/3 \approx 0.333$.

- $U_{\text{rel}}(C) = 1$, because $q = 2$ matches the required reliability for the severe link.
- $U_{\text{crit}}(C) = +0.5$, because the action correctly marks the state as "critical".

These values already show the intended pattern: action C scores highest on all utility terms except energy, action B sits in the middle, and action A behaves poorly for a truly urgent situation.

4.5.2 Costs and Shaping (What is Penalized)

The reward also includes terms that discourage resource waste and unsafe behaviour. Two main elements appear in this example: an energy cost C_{E} and an "event-lazy" penalty when the agent acts too conservatively during an urgent event.

The energy cost C_{E} converts the per-step battery drain ΔmAh from the previous subsection into a normalized quantity in $[0,1]$. A reference high-drain level $e_{\text{ref}} = 0.010$ mAh per step is used, and higher QoS is given a slightly higher weight. The illustrative formula is

$$C_{\text{E}} = \min\left(1, \frac{\Delta mAh}{e_{\text{ref}}}\right) \cdot \left(1 + \frac{q}{2}\right) \qquad (33)$$

Using the energy values already computed:

- **Action A:** $\Delta mAh \approx 0.00227$, based on the equation (33)

$$C_{\text{E}}(A) \approx 0.227 \cdot \left(1 + \frac{0}{2}\right) = 0.227$$

- **Action B:** $\Delta mAh \approx 0.00239$. First, $\Delta mAh/e_{\text{ref}} \approx 0.239$. With $q = 1$, from the equation (33)

$$C_{\text{E}}(B) \approx 0.239 \cdot \left(1 + \frac{1}{2}\right) \approx 0.239 \cdot 1.5 \approx 0.358$$

- **Action C:** $\Delta mAh \approx 0.00589$**.** Here $\Delta mAh/e_{\text{ref}} \approx 0.589$. Similarly, With $q = 2$,

$$0.589 \cdot \left(1 + \frac{2}{2}\right) = 0.589 \cdot 2 \approx 1.178$$

which is clipped at 1, so $C_{\mathrm{E}}(C) = 1.0$. This reflects that action C behaves like a "maximum drain" step.

On top of this, a simple event-lazy penalty is applied. When the environment is clearly urgent (here $\Delta T = 2$ or $anom = 1$), slow and low-fidelity actions are discouraged. Two levels are used in this example:

- A strong penalty of -0.6 is applied if the action uses a long interval and very low fidelity during an event.
- A smaller penalty of -0.2 is applied if the action is only mildly under-reacting.

For the three actions:

- Action A (long $\tau = 60$, $f = 0$ in a critical state) is clearly too lazy ⇒ penalty, $safety_pen \approx -0.6$.
- Action B (moderate $\tau = 30$, $f = 1$) responds somewhat but not fully ⇒ penalty, $safety_pen \approx -0.2$.
- Action C is already escalating hard ⇒ penalty, $safety_pen = 0$.

All these pieces are then combined into a single scalar reward using fixed weights. In this illustrative example, the overall reward is, based on the equation (16)

$$\text{Reward} \approx 0.4\, U_{\text{info}} + 0.2\, U_{\text{time}} + 0.2\, U_{\text{rel}} + 0.2\, U_{\text{crit}} - 0.4\, C_{\text{E}} + safety_pen.$$

	Action A	**Action B**	**Action C**
$\boldsymbol{U_{\text{info}}}$	0.333	≈ 0.667	1.0
$\boldsymbol{U_{\text{time}}}$	≈ 0.071	0.125	≈ 0.333
$\boldsymbol{U_{\text{rel}}}$	0	0	+0.5
$\boldsymbol{U_{\text{crit}}}$	−0.5	−0.5	1.0
$\boldsymbol{C_{\text{E}}}$	≈ 0.227	$C_E \approx 0.358$	0
$\boldsymbol{safety_pen}$	≈ −0.6	≈ −0.2	≈ 0.333
Reward	$R(A) \approx -0.64$	$R(B) \approx -0.15$	$R(C) \approx +0.37$

Table 4.5. 1 Overall reward for each action

In this particular critical state, the escalate action C obtains the highest reward, the balanced action B stays close to neutral, and the frugal action A is strongly discouraged. This matches the intended design: when the sensor detects a strong event and the link delay is high,

the reward function pushes the policy toward short intervals, high QoS, rich payloads, and a correct critical flag, even at the cost of extra energy. In calmer states (not shown here), the same formula reverses the preference and makes frugal actions more attractive, allowing the agent to learn a priority-aware scheduling strategy over time.

4.6 Safety Enforcement in Runtime (using the Same State)

The previous subsection showed that, for the urgent state

$$\Delta T = 2,\ bufT = 1,\ B = 1,\ L = 1,\ D = 2,\ anom = 1$$

the scalar reward ranks the three actions as based on the Table 4.5.1,

$$R(A) \approx -0.64, R(B) \approx -0.15, R(C) \approx +0.37$$

In this example calculation, the "escalate" action C is best by reward, the balanced action B is acceptable but not ideal, and the frugal action A is clearly undesirable in this situation. At runtime, the agent does not rely on the reward alone. Before the selected action is executed, a safety layer modifies the set of candidate actions and their scores. This layer uses hard masks and soft gating to remove obviously unsafe actions and to encourage a consistent choice of the critical flag, while still leaving the DDQN free to choose among the remaining options. The explanation below keeps the same state and the same three actions:

- A (frugal): $q = 0, \tau = 60\ s, f = 0, c = 0$
- B (balanced): $q = 1, \tau = 30\ s, f = 1, c = 0$
- C (escalate): $q = 2, \tau = 5\ s, f = 2, c = 1$

This subsection shows how the safety layer constrains what the agent is allowed to do in this example, rather than fully determining the action.

4.6.1 Hard Masks

The first safety step is a set of hard masks. These masks implement simple rules of the form "never sleep through a crisis". In urgent states with high temperature change ΔT, high delay D, or an anomaly flag $anom = 1$, very lazy actions are removed before the agent can select them.

One typical rule says that, in an urgent state, actions with all of the following properties are forbidden:

- lowest QoS: $q = 0$
- longest interval: $\tau = 60$ s
- lowest fidelity: $f = 0$
- no critical flag: $c = 0$

This pattern corresponds exactly to action A in the example. It sends very little information, very slowly, with the weakest reliability, and does not mark the step as "critical", even though the environment has just signalled a strong event. Such a choice would risk late or missing alerts without any strong benefit.

For the concrete state used in this chapter ($\Delta T = 2, D = 2, anom = 1$), this rule is active. The masking logic checks the state, detects high change, high delay, and an anomaly, and then applies the rule that forbids very lazy actions. As a result:

- Action A is hard-masked and removed from the candidate set.

The neural network may still produce an internal Q-value for A, but the mask marks it as "not allowed". When the agent later searches for the best action, A is simply invisible and can never be picked, no matter how large or small its Q-value might be. The remaining candidate set for this step therefore contains only actions of type B and C. These hard masks encode coarse safety constraints; they do not specify the precise QoS or interval choices within the remaining action set.

4.6.2 Soft Gating with Criticality Probability

After the hard masks, a second safety mechanism reshapes the scores of the remaining actions using a learned criticality estimate. Alongside the Q-values for the 54 actions, the network also produces a scalar

$$p_{\text{crit}}(s) \in [0,1],$$

which represents the estimated probability that the current state is critical. Two thresholds, θ_{lo} and θ_{hi}, divide this range into three regions:

- if $p_{\text{crit}}(s) < \theta_{\text{lo}}$, the state is treated as clearly non-critical, and the layer prefers actions with $c = 0$;
- if $p_{\text{crit}}(s) > \theta_{\text{hi}}$, the state is treated as clearly critical, and the layer prefers actions with $c = 1$;
- if $\theta_{\text{lo}} \leq p_{\text{crit}}(s) \leq \theta_{\text{hi}}$, both critical and non-critical actions remain possible.

The important point is that this gating step does not change the reward itself. Instead, it adjusts the Q-values produced by the network before the final selection:

- preferred actions (whose $c$matches the criticality estimate) keep their Q-values.
- discouraged actions (whose $c$does not match) have their Q-values reduced.

This adjustment nudges the policy towards a consistent choice of the critical flag, while still allowing some flexibility for exploration and learning, especially when $p_{\text{crit}}(s)$ falls between the thresholds.

In the example state, the features $(\Delta T = 2, anom = 1, D = 2)$ strongly indicate a critical situation. The criticality head therefore produces a high value, for instance $p_{\text{crit}}(s) \approx 0.8$, which lies above θ_{hi}. This places the state in the "clearly critical" regime, where actions with $c = 1$ are preferred. Applying this logic to A, B, and C gives the following effects:

- Action A: already removed by the hard mask, so the gating step no longer considers it.
- Action B: remains a feasible action $(q = 1, \tau = 30, f = 1)$ but keeps $c = 0$ despite the high estimated criticality. The gating logic treats this as an under-reaction. Its Q-value is therefore reduced. In the toy numbers used around this example, a raw value such as $Q(B) = 0.00$ may be shifted down to something like $Q_{\text{gated}}(B) = -0.20$. This makes B less attractive than C, even if both had similar raw scores before gating.
- Action C: matches the critical flag $c = 1$ and also uses high QoS and high fidelity. The gating logic treats this as consistent with a critical state and leaves its Q-value unchanged (or possibly adds a very small bonus). In the toy values, C remains at about $Q_{\text{gated}}(C) = 0.25$ and becomes the clear best option among the unmasked actions.

With this combination of hard masks and soft gating, the final candidate set in this step contains B and C, but with different effective scores. A small exploration rate (for example $\varepsilon = 0.05$) means that, most of the time, the agent selects C as the runtime action for this state,

and only rarely explores B. In this sense, the safety layer constrains the choice to safety-consistent actions, while the DDQN still learns the detailed trade-offs between QoS, interval, fidelity, and energy across the wider state space.

4.6.3 Non-Bypassable Penalties

The last safety element concerns truly unacceptable outcomes, such as a buffer overflow or a missed critical alert. When such an event occurs, the system adds a large negative term Ω to the reward at that step. The magnitude of Ω is chosen so that it dominates the return of the whole episode.

In practical terms, this means that no amount of good behaviour before or after the violation can compensate for a single overflow or missed critical event. Any sequence of actions that leads to such an outcome receives a much lower total return than sequences that avoid it, even if the other stepwise rewards are similar. During learning, the agent therefore comes to treat these violations as hard constraints and learns policies that keep the queue within capacity and deliver critical messages, rather than "gambling" on dangerous behaviour and trying to repair the average reward later.

4.7 Action Selection and ε-Greedy Choice

The previous subsections described how the reward is computed and how safety masks and gating modify the available actions. The final step in one decision is the actual choice of an action. This choice is based on the DDQN's Q-values, but only after the safety layer has removed or down-weighted unsafe options. This subsection shows how this works for the running example with actions A, B, and C in the urgent state

$$\Delta T = 2,\ bufT = 1,\ B = 1,\ L = 1,\ D = 2,\ anom = 1$$

4.7.1 Q-values Before and After Gating

For this state, the DDQN network first outputs a Q-value for each of the 54 possible actions, without yet considering masks or gating. For the three actions under discussion, the raw predictions are assumed to be:

- $Q(A) = -0.20$

- $Q(B) = 0.00$
- $Q(C) = 0.25$

At this stage, these numbers only encode how good each action appears according to the learned value function. They do not yet enforce any safety rules and do not yet prefer "critical" or non-critical actions based on the current risk.

The safety layer then modifies this picture in two steps, as described earlier:

1. **Hard mask:** In this urgent state, the very lazy pattern ($q = 0, \tau = 60$ s, $f = 0, c = 0$) is not allowed. This pattern matches action A. The hard mask therefore removes A from the candidate set completely. After masking, A is no longer considered at all, regardless of its Q-value.
2. **Soft gating using criticality probability:** The criticality head produces a high probability that the state is critical, for example $p_{\text{crit}}(s) \approx 0.8$. This value lies above the high threshold, so the gating logic prefers actions with $c = 1$.
    - Action B keeps $c = 0$ in a state that appears "critical". Its Q-value is therefore reduced as a soft penalty, for example from $Q(B) = 0.00$ to a gated value around $\tilde{Q}(B) = -0.20$.
    - Action C uses $c = 1$ and matches the critical nature of the state. Its Q-value remains unchanged at $\tilde{Q}(C) = 0.25$.

After masking and gating, the effective Q-values over the safe set become:

- A: removed by the mask
- B: $\tilde{Q}(B) \approx -0.20$
- C: $\tilde{Q}(C) \approx 0.25$

The argmax operation is then applied only over this safe set. In this example, the argmax of the gated Q-values selects C, because 0.25 is higher than -0.20. If ε-greedy exploration were not used, the runtime policy in this state would always choose the escalate action C

4.7.2 ε-greedy Exploration

Purely greedy behavior (always taking the highest Q-value action) can cause the agent to get stuck in a local pattern and stop trying alternatives. To keep some exploration, the policy

uses a ε-greedy rule. This rule works as follows, always over the safe set after masking and gating:

- With probability $(1 - \varepsilon)$, the agent chooses the action with the highest gated Q-value in the safe set (the "best guess" according to current knowledge).
- With probability ε, the agent ignores the ranking and samples an action at random from the safe set (to explore other feasible options).

In the running example, the safe set consists of:

- B with $\tilde{Q}(B) \approx -0.20$
- C with $\tilde{Q}(C) \approx 0.25$

The greedy choice among these two is C. With a typical late-training value $\varepsilon = 0.05$:

- About 95% of the time, the policy chooses C, because C has the highest gated Q-value.
- About 5% of the time, the policy explores and randomly picks B or C from the safe set, which means B is occasionally tried even though it is not currently the best.

In this state, the reward calculation already showed that C has the highest scalar reward $R(C) \approx +0.37$, while A and B have lower rewards. The ε-greedy policy, after masks and gating, therefore, aligns the learned choice with the ideal reward-maximising action: most of the time, the system escalates using C, and only rarely tries B as a safe but suboptimal alternative.

4.8 Training Loop and Losses (Offline and Online)

The previous subsections described what happens in a single decision step: the environment produces a state, the agent computes Q-values, the safety layer filters actions, and the system receives a reward. Learning requires many such steps. This subsection summarises how these steps are organised into episodes for training, and how the DDQN updates its parameters from stored transitions, both in the offline simulator and in the online Docker-based system.

4.8.1 Offline Training Loop

Offline training uses the simulator to generate long sequences of experience under different conditions. The training is organised into episodes. In this work, a typical configuration uses 700 episodes, each with up to 250 steps. One episode corresponds to one simulated lifetime fragment of a device: the battery starts at a given level, the backlog starts near empty, and the episode stops when the maximum number of steps is reached or when the battery is exhausted.

Each episode follows the same pattern:

1. **Initialisation:** The simulator resets the battery (for example to 1000 mAh or 5 mAh), clears the backlog, and initialises the sliding windows for loss and delay. The agent observes the first state s_0, which includes the bins for change ΔT, backlog $bufT$, battery B, link quality (L, D), the anomaly flag $anom$, and the current action choice $(I_{\text{cur}}, Q_{\text{cur}}, F_{\text{cur}})$.
2. **Interaction and storage:** For each step t in the episode:

- The agent receives the current state s_t.
- The safety layer masks forbidden actions and gates the Q-values, as described earlier.
- The ε-greedy rule selects one safe action a_t from the 54 possible (q, τ, f, c) combinations.

The simulator applies this action: it updates the backlog, battery, delay and loss, computes the utilities and costs, and returns the next state s_{t+1}, the scalar reward r_t, a terminal flag, $done \in \{0,1\}$ indicating whether the episode has ended (battery empty or step limit reached), and a criticality label, $y_{\text{crit}} \in \{0,1\}$ indicating whether this step is critical according to the rule-based definition. The transition $(s_t, a_t, r_t, s_{t+1}, done, y_{\text{crit}})$ is stored in a replay buffer. The buffer keeps a large collection of such experiences from many episodes. Priorities are assigned so that rare or surprising transitions, especially those involving critical states, are sampled more often.

3. **Network updates:** At regular intervals (for example, every step once the buffer is warm), the trainer samples a mini-batch of transitions from the buffer. For this batch, the DDQN update is carried out:

- The Q-head is updated using a temporal-difference target from a delayed target network (Double-DQN).
- The criticality head is updated using a classification loss that focuses more strongly on rare critical states.
- The gradients from both heads are combined and used to adjust the shared network weights.
- The target network is slowly updated towards the main network using Polyak averaging, which stabilises learning.

4. **Exploration and threshold adaptation:** At the end of certain episodes, the exploration rate ε is reduced towards a small floor value, so that training gradually moves from frequent exploration to more stable exploitation. In addition, the threshold used to interpret $p_{\text{crit}}(s)$ as "critical or not" is periodically re-evaluated using validation statistics, so that the gating logic balances recall and precision for critical events.

Online training follows the same logical structure. The main difference is the source of transitions: instead of the simulator, the experience now comes from the live CoAP–MQTT pipeline running in Docker, with real-time temperature traces, real RTTs, and real loss patterns. The replay buffer and loss calculations remain the same.

4.8.2 Losses on the Example Transition

For each sampled transition $(s_t, a_t, r_t, s_{t+1}, y_{\text{crit}})$, the network computes two losses:

- a value loss for the Q-head, which measures how close the predicted Q-value is to the temporal-difference target.
- a criticality loss for the auxiliary head, which measures how well the predicted probability $p_{\text{crit}}(s_t)$ matches the true label of the state.

Both losses are then added and used together to update the shared parameters of the network. The running example provides a concrete illustration. Consider the transition where the agent chose action C in the urgent state and received the reward $r \approx +0.37$. Suppose:

- the discount factor is $\gamma = 0.99$;
- the maximum target Q-value for the next state s' over all actions is $\max_{a'} Q_{\text{target}}(s', a') \approx 0.20$; the current Q-head predicts $Q(s,\text{C}) = 0.25$. The Double-DQN target for this sample is then

$$y = r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a') \approx 0.37 + 0.99 \cdot 0.20 \approx 0.5646.$$

The temporal-difference error is

$$e = Q(s,\text{C}) - y \approx 0.25 - 0.5646 = -0.3146.$$

This work uses the Huber (Smooth L1) loss for the Q-head. For errors with magnitude smaller than 1, the Huber loss is essentially half the squared error:

$$L_Q^{\text{raw}} = \frac{1}{2} e^2 \approx 0.5 \times 0.3146^2 \approx 0.0495.$$

Because prioritised replay is used, each sample carries an importance weight. If the importance weight for this transition is $w \approx 0.8$, the weighted Q-loss becomes

$$L_{\text{Q}} \approx 0.8 \times 0.0495 \approx 0.0396.$$

In parallel, the criticality head is trained to predict whether the state is critical. For this example, assume the true label is "critical" (label = 1) and the criticality head currently outputs $p_{\text{crit}}(s) = 0.60$. A focal loss is used here, which down-weights easy examples and focuses more on harder or rare ones. With parameters $\alpha = 0.9$ and $\gamma = 2$, the focal loss for this positive sample evaluates to

$$L_{\text{crit}} \approx 0.074.$$

The total loss for this transition is then

$$L_{\text{total}} = L_{\text{Q}} + L_{\text{crit}} \approx 0.0396 + 0.074 \approx 0.1136.$$

This combined loss is backpropagated through the network. The Q-head becomes better at matching the observed returns, and the criticality head becomes better at recognising critical states. Because both heads share the same feature extractor, improvements in one head also

help the other. Over many episodes and many such updates, the DDQN agent learns a policy that escalates aggressively when the state is genuinely urgent and conserves energy in quiet periods, while respecting the safety constraints built into the masks and penalties.

4.8.3 Network Impairment Model

The During online deployment, the same decision pipeline is used as in offline training. At each step, the device observes a state vector

$$s_t = (\Delta T_t, bufT_t, B_t, L_t, D_t, I_{\text{cur},t}, Q_{\text{cur},t}, F_{\text{cur},t}, anom_t)$$

selects an action

$$a_t = (q_t, \tau_t, f_t, c_t)$$

and receives a single scalar reward. The only difference is the source of the next state s_{t+1} and the observed outcomes: these now come from the live CoAP–MQTT stack, with real sensor readings, real delay, and real loss, instead of from the simulator. Because the interfaces are kept identical (same state encoding, same 54-action space, same reward definition), a policy trained offline can be deployed online without structural changes, and additional online experience can continue to refine the same DDQN parameters.

4.9 One-Glance Summary Table

The following Table 4.9 brings together the entire pipeline for the worked example state with backlog $q_t = 120$ (bin $bufT = 1$), medium battery (bin $B = 1$), medium loss, high delay, and a strong temperature event ($\Delta T = 2, anom = 1$). The three candidate actions A, B, and C differ in QoS, interval, fidelity, and critical flag. The table shows, for each action: how the backlog evolves, how much battery is consumed, the induced delay and drop probability, the resulting reward, the Q-values before and after safety gating, and the final choice.

Action (q, τ, f, c)	A: (0, 60 s, 0, 0)	B: (1, 30 s, 1, 0)	C: (2, 5 s, 2, 1)
Next backlog (q_{t+1})	121.16	121.03	117.2
ΔmAh per step	0.00227	0.00239	0.00589

Delay δ (ms)	216.7	195.3	324
Drop probability (p_{drop})	0.377	0.208	0.05
On-time probability	0.623	0.792	0
Reward (R)	−0.64	−0.15	0.37
Q before gating	−0.20	0	0.25
Q after gating	masked	−0.20	0.25
Chosen?	✗	✗	✓

Table 4.9. 1 Entire pipeline parameters based on each example actions

Based on the Table 4.9, in this state, action C aggressively drains the backlog, greatly reduces drop probability, and earns the highest scalar reward, at the cost of higher energy and slightly late delivery. The safety layer removes A as an unsafe "event-lazy" action and down-weights B for ignoring the critical flag, so the gated argmax and the ε-greedy policy both favour C as the preferred behaviour.

4.10 Behavioural Summary

This example illustrates how the designed system behaves under pressure and how this behaviour addresses the original problem of intelligent and efficient IoT communication. The backlog dynamics show how a slow, frugal action lets the queue grow, while an escalated action can actively drain it. The energy model exposes the extra battery cost of escalation and the savings of conservative choices. The delay and loss models highlight the reliability gains from higher QoS and higher fidelity, balanced against timeliness. The reward brings these aspects together so that escalation (short interval, high QoS, high fidelity, critical flag on) is encouraged only when the state truly indicates urgency, while quiet periods favour long intervals and low-cost settings. Safety masks and non-bypassable penalties further protect the device from obviously unsafe decisions, such as very lazy actions during anomalies or buffer overflows. As a result, the learned DDQN policy tends to increase service only when needed, conserve energy when possible, and maintain acceptable reliability and latency across a wide range of network and battery conditions.

5.

Evaluation and Results

This chapter evaluates the PASA-driven RL controller from simulation to a live CoAP–MQTT testbed. Results are organized to first establish experimental conditions and baselines, then report offline (sim) outcomes, online (runtime) outcomes, and finally ablations, sensitivity, and limitations.

5.1 Experimental Setup

Chapter 3 described the decision problem as a MDP with clear objectives: save battery, keep latency low, and deliver important messages reliably. Chapter 4 then showed how this decision process was implemented in practice and walked through one complete example step, from sensor reading to chosen action. This chapter now evaluates the same controller with numbers, using both a simulated environment and a live CoAP–MQTT testbed. This subsection explains how these experiments are set up so that later results are easier to interpret.

Both offline and online experiments use the same kind of workload. Temperature readings are generated with a simple "day–night" pattern plus small random noise and occasional spikes. A short moving average tracks the recent level, and the difference between the current reading and this average is used to detect change. This difference is converted into bins (low, medium, high) and an anomaly flag is raised when the change is unusually large. In parallel, the communication link is exposed to different regimes of delay and loss so that the controller is forced to adapt its behaviour under easy and difficult conditions. In the online runs, the bridge reconstructs the same summary information (loss, delay, backlog, battery) using sliding windows, so that the state fed to the controller has the same meaning as in simulation.

The offline simulator is run for 700 episodes. Each episode lasts up to 250 decision steps. A decision step corresponds to one control interval (for example, 5, 30, or 60 seconds, depending on the chosen action). Fixed random seeds are used so that a given configuration can be repeated and compared fairly with others. The online testbed currently contains 139 episodes. These episodes follow the same decision cadence: at each interval, the agent observes the state built from the live CoAP server and bridge, chooses an action, and applies it to the running system. This makes the offline and online timelines directly comparable: both are sequences of "observe–decide–act" steps driven by the same logic.

All experiments run on a modest laptop using a Docker-based microservice architecture. One container emulates the CoAP sensor and its queue; another container acts as the CoAP–MQTT bridge; a local MQTT broker forwards messages; the RL agent runs in its own container; and a telemetry sink records time-series data in a database for later analysis. Every component exchange compact JSON message with stable field names so that the same dashboards and analysis scripts can be used for both modes.

Safety mechanisms are kept consistent between simulation and the live system. The controller uses hard action masks to rule out clearly unsafe choices, soft gating driven by a criticality estimate to favour either "escalate" or "conserve" actions, and strong penalties inside the reward when serious violations occur, such as buffer overflow or missed critical alerts. In the runtime environment, a watchdog is also present: if the agent stops sending control messages for some time, the system falls back to a conservative default action profile so that the device remains in a safe operating mode. This common safety design ensures that behaviour seen in simulation has the same structure when the controller is deployed on the real CoAP–MQTT stack.

5.2 Baselines (Static QoS/Interval, Heuristic)

To judge whether the learned controller is useful, this thesis compares it against several simple strategies that do not learn. These baselines represent intuitive extremes: always cheap, always safe but expensive, and a hand-written rule that tries to copy the PASA principle.

- **Static-Frugal baseline:** This baseline always chooses the lowest QoS level, the longest reporting interval (60 seconds), the smallest payload (lowest fidelity), and never marks data as "critical". This pattern strongly protects battery lifetime and reduces network load, but risks delayed or lost delivery when conditions are poor or when important events occur.
- **Static-Balanced baseline:** This baseline always uses a medium QoS level, a medium reporting interval (30 seconds), and a medium payload size. The critical flag is kept off. This setting represents a "reasonable default" that many might pick not too aggressive, not too conservative, but also unable to react to changing conditions.
- **Static-Aggressive baseline:** This baseline always uses the highest QoS level, the shortest interval (5 seconds), the richest payload, and always marks data as "critical". This behaviour is very safe in terms of reliability, because messages are sent often and

with strong delivery guarantees. However, it consumes the most energy and network airtime and can shorten device lifetime significantly.

- **PASA-rule baseline (hand-crafted heuristic):** This baseline follows a simple rule inspired by the PASA idea. When the temperature change bin is high, when an anomaly flag is raised, or when the backlog bin indicates strong queue pressure, the heuristic switches to an "escalate" profile: higher QoS, shorter interval, higher fidelity, and the critical flag set. When the state looks calm (low change, no anomaly, low backlog), the heuristic returns to a "conserve" profile similar to the frugal setting. This baseline acts as a manual version of the desired policy: escalate only when signals indicate priority, otherwise save energy.

Together, these baselines cover the main trade-offs of the problem. The static-frugal policy represents "always cheap but sometimes unsafe", the static-aggressive policy represents "always safe but costly", the static-balanced policy sits in between, and the PASA-rule baseline represents what a human designer might implement by hand. The learned controller is expected to match or exceed the safety of the aggressive and heuristic baselines while moving closer to the energy efficiency of the frugal baseline.

5.3 Scenarios (Cases) Table

The evaluation considers two closely related scenarios: a controlled offline simulator and a live online testbed. Both scenarios share the same decision logic, state variables, and action space, so that policies trained in simulation can be transferred directly to the live system.

In both scenarios, the temperature signal follows a daily-like wave with added noise. Short-lived spikes are injected to create events. A simple moving average estimates the recent level, and the absolute difference between the current and average values is used to measure change. This change is converted into three levels (low, medium, high), and an anomaly flag is raised when the change exceeds a threshold. Under typical settings, this produces a natural "critical" fraction of about 10–25% of the steps, meaning that most of the time the system is calm but occasional periods require escalation.

Backlog, loss, delay, and battery are treated in the same way in both cases. The queue can hold up to 300 backlog units. Each step, new "work" enters the queue depending on how much the signal changes, and work leaves the queue depending on the chosen interval, payload

fidelity, and burst flag. The occupancy ratio of this queue is then converted into low, medium, or high backlog bins. Link quality is monitored with sliding windows over recent packets: the fraction of dropped packets and the average latency are turned into loss and delay bins. A simple battery model maps current draw into a remaining capacity estimate and then into three battery bins (low, medium, high). These bins, together with the anomaly flag and the currently applied action settings, form the state fed into the controller.

The main differences between the two scenarios lie in the physical budget and in how delay and loss are generated. In the offline simulator, the battery capacity is set to 1000 mAh to represent a typical long-lived IoT node, and delay and loss are generated with simple formulas that depend on QoS, interval, and fidelity. In the online testbed, the battery proxy uses a much smaller nominal capacity (5 mAh) to make changes visible over a manageable time span, and delay and loss are derived from real round-trip times and observed delivery outcomes on the local network. The same three interval options (5, 30, and 60 seconds), three QoS levels (0, 1, 2), three fidelity levels, and the binary critical flag are active in both scenarios, and both start from the same initial profile (medium QoS, medium interval, medium fidelity). Episode length and step budget are also aligned, so that one episode in simulation corresponds roughly to one episode in the live experiment.

Id	Offline	Online
$\boldsymbol{\Delta T}$ **pattern**	Daily sinusoid (period=288, amplitude=10) + noise σ=0.2; EMA ΔT binning [1,3]	Daily sinusoid (period=288, amplitude=10) + noise σ=0.2; EMA ΔT binning [1,3]
Anomaly rule	$anom$=1 if ΔT > 2.5 (auto-calibrated if needed)	$anom$=1 if ΔT > 2.5 (auto-calibrated if needed)
Natural critical prevalence (target)	10–25% (auto-calibrator adjusts $CRIT_THRESH$ and $\{anom_threshold, a_scale\}$)	Same
Base loss model $\boldsymbol{p(loss)}$	Action-dependent: $0.18 \cdot (2 - QoS) + 0.008 \cdot (delay/100) + 0.012 \cdot fidelity$	$p = 0.18 \cdot (2 - QoS) + 0.00008 \cdot RTT_ms + 0.012 \cdot fidelity$
RTT / delay model $\boldsymbol{(\mu \pm \sigma)}$	Action-dependent: $base = N(100 + 50 \cdot (2 - QoS), 20)$ $then\ delay = base + 1000/\tau + 12 \cdot fidelity$ $(clipped \leq 800\, ms)$	Delay ~ Normal ($\mu = 100 + 50 \cdot (2 - QoS) + 1000/\tau + 12 \cdot Fidelity, \sigma = 20$), clipped to [0, 800] ms
Battery $\boldsymbol{(mAh)}$	1000	5
Start frequency (s)	30	Same

Start QoS	1	Same
Start fidelity	1	Same
$\boldsymbol{bufT}$ **cap**	300	Same
Seeds	1337 (train), 2024 (val sweep)	Same
$\boldsymbol{Episodes} \times \boldsymbol{max_steps}$	700 × 250	Same

Table 5. 1 Online vs Offline scenario comparison

Table 5.1 summarises these offline and online cases side by side. In short, both scenarios expose the controller to the same structure of information (state bins and anomaly signals) and the same menu of actions but differ in battery scale and in how the network behaves underneath. This design allows a clear comparison: if the learned policy performs well in both scenarios, then the approach is not only effective in a synthetic environment but also robust when faced with real delays, losses, and timing effects in the CoAP–MQTT stack.

5.4 KPIs and Evaluation Method

To judge whether the controller is actually useful in practice, this chapter follows a small, fixed set of performance measures. Each measure focuses on one aspect of behaviour and is always computed in the same way for both the simulator and the live testbed. The six Key Performance Indicators (KPI) are:

- **Return per episode:** For each episode, all rewards over time are added with a discount factor. This gives one number per episode. A higher value means that, on average, the controller made a better sequence of decisions over that episode.
- **Reliability (on-time, late, drop):** Each packet is marked as on-time (arrived before the deadline), late (arrived but too slow), or dropped (never arrived). For each episode, the on-time rate and the late/drop rates are computed as fractions between 0 and 1.
- **Latency (p50 / p95):** For all packets in an episode, the median delay (p50) and a tail delay (p95) are measured in milliseconds. These two numbers show typical delay and "worst-case" delay, without needing the full distribution.
- **Energy and lifetime factor:** Total battery use per episode is measured in mAh. From this, a fractional saving $s$between two periods (early vs late episodes) is computed, and this saving is translated into an ideal lifetime multiplier $F = 1/(1 - s)$. This shows how much longer a battery could last if the same behaviour continued.

- **Policy footprint:** The frequency of each action in the grid (QoS level × interval) is counted. This produces a "fingerprint" of the policy: whether it prefers slow and frugal settings, fast and reliable settings, or a mix.
- **Safety events:** The number of times hard masks are triggered, safety penalties are applied, and hard violations occur is counted. Ideally, masked and penalised events remain low and hard violations remain close to zero.

To compare early and late behaviour in a stable way, all KPIs use a simple two-window method. For a given series (for example, energy per episode), the average over the first 100 episodes is compared with the average over the last 100 episodes. The difference between these two averages is reported as an absolute change and, when useful, also as a relative percentage.

In the simulator, there are 700 episodes, so the "first-100" window covers episodes 1–100 and the "last-100" window covers episodes 601–700. In the online testbed, there are 139 episodes so far. The same method is applied there by using the first 100 episodes as the "early" window and the last 100 available episodes as the "late" window. This evaluation method gives a clear and consistent view of how the controller behaves at the beginning and near the end of training, even when individual episodes are noisy.

5.5 KPIs for Offline Results (Sim)

The simulator experiments provide a controlled environment where the controller can interact with a synthetic sensor, battery model, and link model for 700 episodes. This section summarises how the learned policy behaves at the end of this offline training with respect to the main goals: saving energy, keeping packets reliable and timely, and reacting correctly to "critical" events.

- **Energy Efficiency and Device Lifetime:** The first KPI is energy use per episode. Over the 700 simulated episodes, the total energy consumed in each episode shows a gentle downward trend. This means that the controller gradually shifts towards combinations of QoS, interval, and fidelity that achieve similar usefulness with less battery drain.

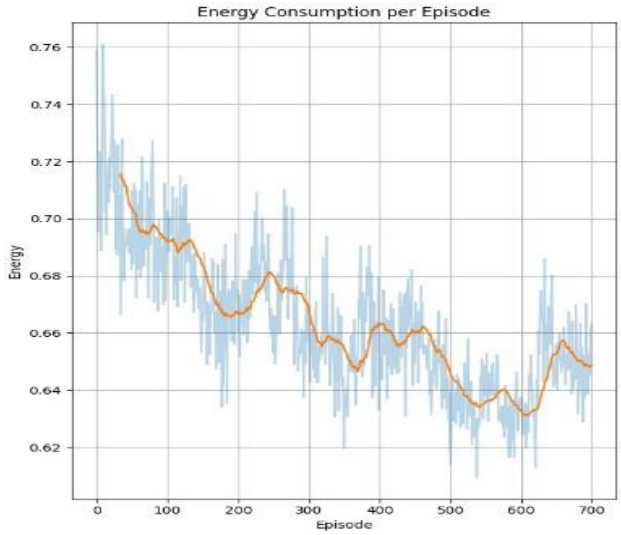

Figure 5. 1 Energy consumption per episode

Figure 5.1 plot is showing Energy Consumption per Episode. The plot tracks the total simulated energy consumed during each episode. A downward trend indicates the policy is becoming more energy efficient. Average latency remains within a narrow, stable band ~ $\boxed{\text{380-395 ms}}$ while on-time increases—i.e., better deadline adherence without latency inflation. Thus, to calculate energy reduction and conversion to lifetime, From the Energy per Episode plot:

$$\overline{E}_{[1:100]} \approx 0.72, \qquad \overline{E}_{[601:700]} \approx 0.695.$$

Per-episode fractional energy reduction:

$$s \equiv \frac{\overline{E}_{[1:100]} - \overline{E}_{[601:700]}}{\overline{E}_{[1:100]}} = \frac{0.72 - 0.695}{0.72} \approx \boxed{0.035\ \ (3.5\%)}.$$

Battery-lifetime translation: If average consumption drops by a fraction s, ideal lifetime scales as

$$\text{gain factor } F = \frac{1}{1-s}, \qquad \%\text{ extra lifetime} = 100 \cdot \frac{s}{1-s}.$$

With $s \in [0.035,\ 0.040]$:

$$F \in \left[\frac{1}{0.965}, \frac{1}{0.96}\right] \Rightarrow \boxed{+3.6\text{-}4.2\ \%\text{ longer lifetime}}.$$

Calculation readability example based on the above data: For a $Capacity = 1000\,\text{mAh}$ node drawing $I_0 = 10\,\text{mA}$ on average:

$$T_0 = \frac{Capacity}{I_0} = 100\,\text{h} \quad \Rightarrow \quad T_1 = F \cdot T_0 \approx \boxed{103.6\text{-}104.2\,\text{h}}$$

i.e., $\boxed{+3.6\text{-}4.2 \text{ extra hours per full charge}}$.

Equivalently, mAh saved per charge $= s \cdot Capacity \approx \boxed{35\text{-}40\,\text{mAh}}$. In summary, this energy saving can be translated into a direct extension of the device's operational lifetime per battery cycle. Assuming ideal battery behaviour, a 3.5% reduction in average consumption yields a projected lifetime gain factor ($F = 1 - 0.0351$) of approximately $+3.6\%$. For a hypothetical 1000 mAh battery with a baseline lifetime of 100 hours, this translates to an additional 3.6 hours of operation per charge. The scatter plot of reward versus energy further revealed that the agent learned to avoid high-energy actions that yielded diminishing returns, concentrating its behavior in an optimal efficiency zone.

- **Average delivery latency:** The following 5.2 is the average delivery latency plot which is showing average latency curve per episode. This latency curve stays in a narrow band (≈380–395 ms) without any long-term upward trend, suggesting that the controller does not trade reliability for excessive delay. Taken together, these panels indicate that the learned policy improves both reliability and energy efficiency while keeping latency under control.

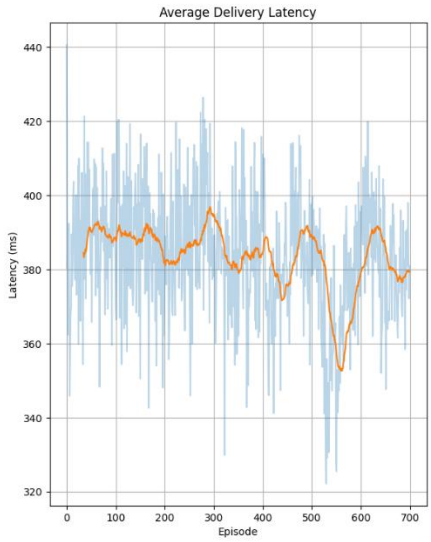

Figure 5. 2 Average delivery latency

- **Reward vs Energy:** The following 5.3 scatter plot is showing "Reward vs Energy" that provides a direct diagnostic of how the scalar reward treats energy. Each point corresponds to one episode. The cloud has a clear negative slope: episodes with higher reward are concentrated in the lower-energy region, whereas high-energy episodes tend to have lower reward. This shows that the hand-crafted scalarisation does not ignore energy or allow positive terms to cancel the energy cost; instead, it promotes policies that achieve good reliability at moderate energy consumption.

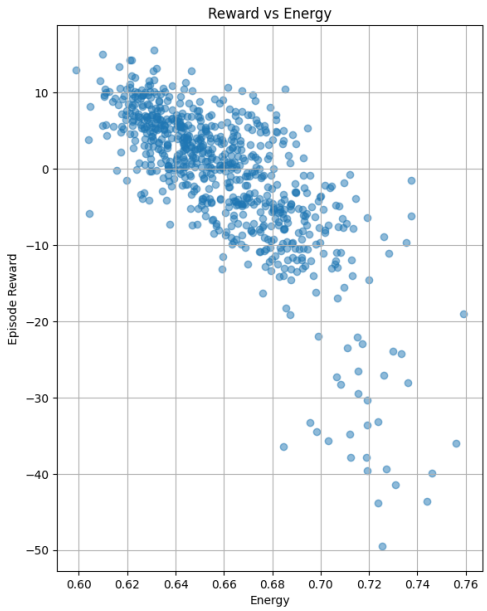

Figure 5. 3 Reward vs Energy scatter plot

- **Network Reliability and Latency:** Energy savings would not be acceptable if they came from simply sending fewer packets and ignoring the deadline. The second set of KPIs therefore focuses on reliability and timeliness.

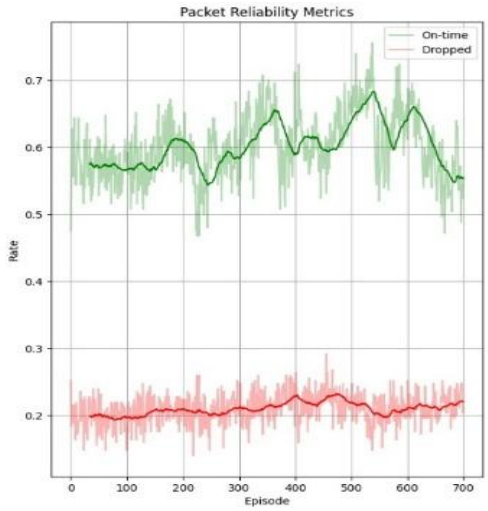

Figure 5. 4 Packet reliability metrics per episode

Figure 5.4 plot is Packet Reliability and Timeliness Metrics showing the evolution of the on-time delivery rate (percentage of packets arriving within a 300 ms deadline) over the training episodes. Populated as on_time_rate and $drop_rate$. This subplot shows that the rate of on-time packet delivery increased from approximately 56% to 66% (+10 percentage points). From the Packet Reliability panel (using the same windows):

$$\overline{\text{OnTime}}_{[1:100]} \approx 0.56, \qquad \overline{\text{OnTime}}_{[601:700]} \approx 0.66.$$

Absolute (percentage-point) improvement:

$$\Delta_{\text{pp}} = (0.66 - 0.56) \times 100 = \boxed{+10\ \text{pp}}.$$

Relative improvement (optional):

$$\frac{0.66 - 0.56}{0.56} \times 100 \approx \boxed{+17.9\ \%}.$$

During the same period, average latency stays in a narrow band around 380–395 ms. This means that the controller does not achieve higher on-time rates by pushing delays

up for other packets. Instead, it learns to adjust QoS and interval so that more packets meet the deadline while the overall delay distribution remains stable. In summary, the simulation suggests that offline learning reduces energy use and improves on-time delivery, without introducing systematic extra delay.

- **Critical Event Handling:** A key part of this work is the ability to treat critical events differently from normal behaviour. The controller includes an auxiliary head that predicts whether the next state is likely to be "critical". This head is trained using a classification loss alongside the main value function.

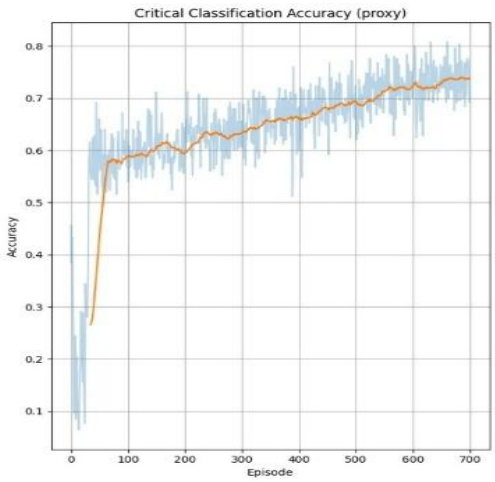

Figure 5. 5 Critical classification accuracy (proxy)

Figure 5.5 subplot represents critical state classification accuracy. Accuracy of the auxiliary network head in predicting whether an upcoming state is critical, measured on training batches. At the start of training, the classifier accuracy is around 27%, which is close to guessing. Over time, the accuracy rises and stabilises in the range of 66–68% on the training batches. This means the controller becomes much better at recognising which situations deserve urgent treatment (for example, strong temperature changes, anomalies, or growing backlogs). This improved awareness is reflected in the actions: early in training, many transmissions are marked as "critical", which wastes resources. Later, critical flags are used more selectively, and bursts of high QoS and short intervals are applied mainly when the state actually looks dangerous.

- **Qualitative Analysis of the Converged Policy:** Looking only at averages can hide how the controller behaves in specific situations. To address this, the final offline policy is inspected in two complementary ways: through its overall action footprint and through

a small set of “probe” states. Learned Bimodal Strategy Distribution is an analysis of the agent's action selection frequency across a representative set of states that reveals a clear, bimodal operational strategy.

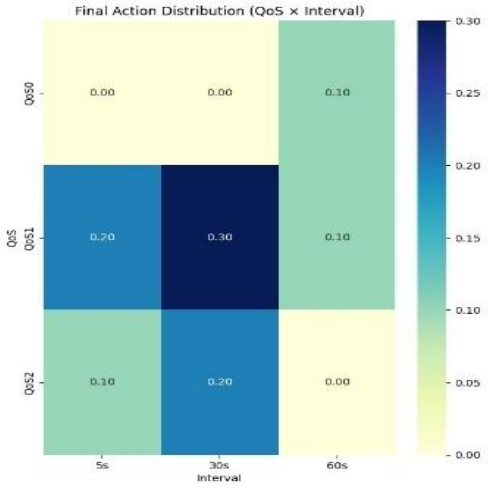

Figure 5. 6 Final action distribution (QoS x interval)

Figure 5.6 heatmap is Final Action Distribution (QoS × Interval). The heatmap shows the selection frequency of different QoS and reporting interval combinations for the converged policy. As seen in Figure 4.4, the policy is dominated by two distinct modes:

- Frugal Mode: The most frequently chosen action (~30% of the time) is (QoS 1, Interval 30s). This represents a balanced, energy-saving "cruising" state for periods of normal system operation.
- High-Performance Mode: A secondary concentration around (QoS 1, Interval 5s) represents an "alert" mode. This action is selected to increase data freshness when the agent perceives a change in the environment or an increase in urgency.

This bimodal pattern is exactly the kind of behaviour that the PASA principle encourages conserve energy most of the time and escalate when priority is high. To zoom in further, a few representative probe states are analysed:

- State-Action Policy Probes: To examine the policy at a granular level, the agent’s decision needs a closer look in specific, representative states. The output result below in Appendix B Policy probes (state → learned vs. “ideal”) compares the agent's learned action to an "ideal" action that maximizes only the immediate reward, showcasing the agent's learned foresight.

- “Probe-state” referencing Appendix B:

Scenario (signature)	Learned action vs. ideal	Critical flag	Comment
Calm, clean link ($\Delta T = 0, bufT = 0$)	Match: (QoS 0, 60 s, F0)	0 ✓	Energy-saving baseline.
Moderate change, clean link ($\Delta T = 1, bufT = 0$)	Shortens interval; minor QoS/F trade vs. ideal	0 ✓	Long-term value over myopic reward.
Backlog high / anomaly ($bufT = 2\ or\ anom = 1$)	Escalates to 5 s, higher QoS/F	1 ✓	Gating works: some negative rewards reflect costly states, not policy failure.
Poor link + low battery ($L/D = 2, B = 0$)	QoS2 with frugal F/τ	1 ✓	Reliability preserved while controlling cost.

Table 5.5. 1 “probe-state” mini-table (for Appendix B cross-reference)

Based on the result shown in Appendix B: Analysis on the printed Double DQN Policy Analysis (T-only) results, to look inside the trained policy beyond aggregate curves and to examine the policy at a granular level, a set of representative probe states was analysed. This compares the agent's learned action against an "ideal" action that maximizes immediate reward.

- In calm states (small temperature changes, empty backlog, healthy link, comfortable battery), the learned policy chooses a very frugal action, such as low QoS, long interval, low fidelity, and no critical flag. This matches the ideal energy-saving behaviour.
- In states with moderate change but still healthy link, the controller shortens the interval and sometimes raises QoS or fidelity. These actions may not always match the single step “best reward” action, but they reflect a compromise that considers long-term value.
- When backlog is high or an anomaly is present, the controller shifts to short intervals and higher QoS/fidelity and usually sets the critical flag. This shows

that the safety and gating logic is active and pushes the policy into more aggressive modes when needed.

- Under poor link and low battery, the policy often chooses high QoS combined with more frugal interval and fidelity settings. This combination keeps reliability high while still controlling energy and airtime.

In the simulator, the PASA-driven Double DQN controller achieves all of the main design goals at the same time. Per-episode energy use drops by a few percent, which translates into several extra hours of battery life for a typical IoT node. On-time delivery improves by about 10 percentage points without inflating latency. The critical-state classifier becomes substantially more accurate, allowing the system to reserve expensive actions for truly urgent situations. The final policy footprint and probe states both show a clear PASA-style pattern: a frugal cruising mode for normal operation and an alert mode for high-priority states. These offline results provide a strong motivation to deploy the controller in the live CoAP–MQTT testbed and to observe how much of this behaviour carries over to real runtime conditions.

The offline diagnostic plots further clarify how the scalar reward relates to the individual objectives. These diagnostics support the interpretation of the proposed controller as a scalarised multi-objective heuristic with constraints: one scalar reward combines information utility, timeliness, reliability, and energy, but the resulting policy also improves the individual KPIs when they are plotted separately. At the same time, the figure also illustrates a limitation. The scalar reward and its fixed weights select one particular compromise on the underlying energy–delay–reliability trade-off surface, rather than explicitly exploring a full Pareto front. A more principled vector-valued or constrained RL formulation remains an important direction for future work.

5.6 KPIs for Online Results (Testbed)

The online experiments run the same controller inside the real CoAP–MQTT stack, using the same state, action, and reward definitions as in the simulator. At the time of analysis, the live system had produced 139 episodes. This section presents a first snapshot of learning in this setting, using the same KPIs and two-window method as in the offline case.

- **Learning and convergence:** The cumulative return per episode remains the main indicator of whether the controller is learning useful behaviour online. Using the first-100 vs last-100 episodes:
    - The mean return in the first 100 episodes is about −22.8.
    - The mean return in the last 100 episodes is about 0.0.

  The gain in return is therefore +22.8, which corresponds to a 100% relative improvement from the initial level. This shows that the controller is indeed learning and improving its decisions in the live system, even though the total training time online is much shorter than in the simulator (139 vs 700 episodes).

- **Reliability (on-time) and latency:** For the online testbed, reliability is again measured as the on-time delivery rate per episode. Applying the same two-window method:
    - The mean on-time rate in the first 100 episodes is about 0.603.
    - The mean on-time rate in the last 100 episodes is about 0.520.

  This corresponds to a change of about $(0.520 - 0.603) \times 100 = -8.28$ percentage points, or roughly $\frac{0.520-0.603}{0.603} \times 100 = \boxed{-13.7\%}$ in relative terms. In other words, in this early snapshot, the fraction of packets delivered before the deadline has slightly decreased. At the same time, the median latency (p50) remains close to 400 ms, and the p95 latency stays well below 1 second. These values are consistent with the simulator, where the central latency band was around 380-395 ms. This indicates that the live system does not suffer from general delay inflation; instead, the on-time decrease is mainly due to the choice of actions, not a collapse of the link itself.
  The most plausible explanation is that the controller is still in a relatively exploratory phase online. The action footprint confirms this: choices of QoS and interval are spread almost uniformly across the grid, without the clear bimodal pattern seen offline. As a result, the agent has not yet locked into the most reliable combinations for urgent situations.

- **Energy consumption and device lifetime:** Energy per episode in the online testbed is computed in the same way as in the simulator and then used to estimate a fractional saving and lifetime factor. However, in the current run, the resulting numbers are not physically meaningful. Using the two-window method:

- Average energy per episode in the first 100 episodes is about 0.262 mAh.
- Average energy per episode in the last 100 episodes is about 0.00306 mAh.

Plugging these values into the same formula for fractional saving yields an apparent saving of about $\frac{0.262-0.00306}{0.262} \times 100 \approx 98.8\%$ and a lifetime factor of around $\frac{1}{1-0.988} \approx 85.5\times$, which would suggest almost no battery use in later episodes. This is clearly unrealistic and is caused by several experimental artefacts:

- Use of a very small 5 mAh test battery to shorten episodes.
- Interruptions and restarts during online runs.
- A mismatch between episode normalisation in the simulator and in the live data logging.

Because of these factors, the online energy and lifetime KPIs are marked as provisional. A longer, uninterrupted online run with a more realistic battery model and consistent episode definition is required before a meaningful comparison with the offline energy savings can be made.

- **Evolved policy structure (QoS × interval):** The next step is to look at the policy footprint in the live system. For each interval (5 s, 30 s, 60 s), the number of times each QoS level was chosen is counted. For the 139 online episodes, the distribution is:
  - **5 s interval:** each QoS level (0, 1, 2) is chosen about one third of the time.
  - **30 s interval:** each QoS level again has a share close to one third.
  - **60 s interval:** shares are slightly tilted towards QoS 0 but still close to uniform.

Overall, about 35.6% of actions use 60 s, 30.8% use 30 s, and 33.7% use 5 s. Within each interval, QoS choices are almost evenly split across the three levels. This nearly uniform footprint is typical of a controller that is still exploring the action space and has not yet formed a strong preference for particular combinations. In contrast, the offline converged policy has a clear bimodal structure, with one cluster around QoS 1 / 30 s and another around QoS 1 / 5 s. The absence of this pattern online explains why on-time reliability has not yet reached the offline target: the agent is still trying many options instead of specialising on the most effective ones.

In the live CoAP–MQTT testbed, the Double DQN controller already shows clear signs of learning: cumulative return per episode improves from −22.8 to 0.0, and latency remains within a healthy range. However, on-time delivery has not yet improved and currently drops by about 8 percentage points in the last 100 episodes. Energy and lifetime metrics are distorted by the small battery and run interruptions and therefore cannot yet be trusted. The nearly uniform action footprint suggests that the policy is still exploring and has not yet settled into the PASA-style bimodal strategy seen in the simulator. For these reasons, the current online results are best understood as an early snapshot, not as a final performance bound.

5.7 Comparison Offline vs Online

With both offline and online KPIs in place, it becomes possible to compare the two regimes side by side. The same two-window method is used for each KPI so that the early and late behaviour of the controller can be contrasted under simulation and under live conditions.

For cumulative return, the simulator shows an improvement of about +36 points, from roughly −28 to +8, which corresponds to a relative gain of about 129%. Online, the improvement is about +22.8 points, from −22.8 to 0.0, a relative gain of 100%. Both regimes therefore show a clear positive learning signal, with the offline run reaching a higher final level simply because it covers a much longer training horizon (700 episodes vs 139).

For on-time delivery, the simulator achieves an increase of about 10 percentage points, from 0.56 to 0.66, while keeping average latency stable. In contrast, the online system currently shows a decrease of about 8.28 percentage points, from 0.603 to 0.520. This mismatch aligns with the policy footprints: the simulator has a well-formed bimodal pattern, whereas the online policy is still close to uniform.

For latency, both regimes are broadly consistent. Offline, average latency remains around 380-395 ms. Online, the median latency is around 400 ms, with tail values (p95) well below 1 s. This suggests that neither regime gains performance by allowing systematic delay inflation; the main differences arise from how often particular actions are chosen, not from a fundamental limit of the link.

For energy, the simulator shows a modest but meaningful saving of about 3.5% per episode, which translates into a lifetime factor of about 1.036× in the simple battery example.

Online, the raw numbers suggest a saving of about 98.8% and a lifetime multiplier of 85.6×, but this is clearly an artefact of the current setup (small 5 mAh battery, irregular runs, and episode normalisation issues). At present, only the offline energy result can be treated as physically meaningful. The online energy metrics will need to be recomputed after longer, standardised runs.

For critical event handling, the simulator reports that the classifier accuracy rises from roughly 27% to 66–68%, and the probe-state analysis confirms that critical flags and bursts are applied in the right kinds of states. No quantitative online KPI for the critical classifier is reported yet, so this comparison is still one-sided.

The battery-lifetime translation gives another way to read the offline energy saving. For a 1000 mAh battery at 10 mA, a 3.47% saving extends lifetime from about 100 h to about 103.6 h, adding roughly 3.6 h and saving about 35 mAh per cycle. For a tiny 5 mAh battery at the same current, the same relative saving would add only about 1 minute of lifetime and save about 0.17 mAh, which is consistent with the idea that relative gains are easier to interpret than absolute mAh in very small batteries.

Finally, the policy structure completes the comparison. Online, the counts over QoS × interval show roughly one-third share for each QoS at each interval and a balanced split over the three intervals. Offline, the converged policy footprint shows:

- a dominant frugal mode around QoS 1 / 30 s, and
- a secondary high-performance mode around QoS 1 / 5 s.

This context-aware bimodality explains why offline on-time delivery improves while energy use drops. The online footprint has not yet collapsed into this pattern, which explains why on-time delivery has not reached the offline level despite evidence of learning in the return.

5.7.1 Robustness Tests: Removing Safety Features and Variants

To evaluate the system's robustness, several tests are performed in the simulator. Each test turns off or changed just one component of the system to see what would happen. The main qualitative observations are as follows:

- Without hard masks, the controller is allowed to pick unsafe actions (for example, very slow, low-QoS actions during clear congestion or critical events). In these runs, unsafe situations become more frequent, on-time delivery in urgent states degrades, and the overall return is less stable. This proves that hard masks are important to keep behaviour inside reasonable safety bounds.
- Without soft gating and the criticality head, the controller no longer receives a clear signal about which states are critical. In this case, more false alarms and missed critical events occur, and the system takes longer to recover after bursts of bad conditions.
- Using 1-step targets instead of 3-step targets for learning makes credit assignment more short-sighted. The learning curve becomes flatter, and improvements in tail latency are weaker, because the controller sees less of the delayed effect of its actions.
- Without Prioritized Experience Replay, all experiences are treated equally when sampling batches. This reduces sample efficiency: return grows more slowly, and hard or rare cases are revisited less often.
- Without focal loss for the critical classifier, a standard class-balanced loss is used. In this case, rare critical events are detected less reliably, the gating head becomes less informative, and the overall behaviour becomes more conservative, sending fewer urgent bursts than desired.
- With energy and network cost multipliers set to zero, the reward no longer penalises energy or link strain. The policy then drifts towards more aggressive, high-QoS and short-interval actions, which increase battery usage and network load without giving proportional gains in return.

Overall, these tests prove that the masks, gating, multi-step targets, replay prioritisation, focal loss, and cost terms are all active components of the implementation. The safety mechanisms, in particular, act as constraints and guardrails: they do not solve the control problem by themselves, but they focus learning on behaviour that respects basic safety and resource limits. Removing any of them does not completely break the system, but it moves the policy away from the intended PASA-style balance and highlights where the design currently relies on hand-crafted structure.

5.7.2 Sensitivity: Weights, Multipliers, and Thresholds

In addition to ablations, a small sensitivity study explores how the system reacts when key weights and thresholds are shifted within reasonable ranges. The aim is to check whether the design behaves smoothly when tuning changes, rather than in a brittle way. The main findings are:

- **Utility weights** for information, timeliness, reliability, and criticality form a balanced set in the main experiments. Increasing the reliability weight too much encourages higher QoS even when the link is already healthy, which wastes energy. Increasing the timeliness weight too much pushes intervals towards shorter values more often than needed. Moderate changes keep behaviour reasonable; extreme changes skew the policy in predictable directions.
- **Cost multipliers** for energy and link strain behave as expected. Tying the energy weight to the battery bin makes the trade-off between return and energy transparent: when the multiplier is raised, the policy becomes more conservative, and energy is saved at the cost of slightly lower return. If the network cost multiplier is set too high, the controller becomes reluctant to increase fidelity even when the link is clearly impaired.
- **Safety shaping terms**, such as the idle bonus and safety penalty, need to sit in a sensible range. If the idle bonus is too large, the controller over-uses low-QoS, long-interval, low-fidelity actions and becomes excessively frugal. If the safety penalty is too small, slow and low-fidelity actions are allowed to persist even in urgent states, which harms on-time delivery.
- **Gating thresholds** define a band between “definitely not critical” and “definitely critical”. A moderate gap between the lower and upper thresholds avoids constant switching. If the band is too narrow, the system flips frequently between modes; if too wide, the guidance becomes weak, and the critical flag has less influence.
- **Exploration settings** (epsilon decay and NoisyNet) also show the expected pattern. Higher exploration early on helps the controller cover the state-action space. Later, a lower epsilon or a more deterministic NoisyNet behaviour is needed to specialise the policy footprint.

- **The n-step parameter** works best around three steps in this setting. Larger values increase variance in the updates without bringing clear performance gains; smaller values make learning slower and more myopic.

Taken together, these sensitivity results indicate that the design has a robust operating region: within broad ranges, changing weights and thresholds leads to smooth and interpretable shifts in behaviour rather than sudden collapse.

6.

Conclusion

This thesis implemented and evaluated an intelligent IoT communication system that adapts MQTT/CoAP transmissions with a Double DQN controller. At each step, the controller selects four parameters—QoS level, reporting interval, payload fidelity, and a critical flag. These choices help to balance energy use, reliability, and delay. A complete testbed was implemented, including a CoAP sensor service, a CoAP–MQTT bridge, a local MQTT broker, an RL agent, and a telemetry pipeline with dashboards. The same state, action space, and reward were used in both offline simulation and the online Docker deployment.

In offline simulation, the controller achieved the expected. Over 700 episodes, average return rose from about −28 to +8, on-time delivery increased from roughly 56% to 66%, and average delay stayed around 380–395 ms. Energy used each episode went down by about 3.5%. This means a 1000 mAh battery using 10 mA can last about 3 to 4 extra hours. The learned policy showed a clear two-mode pattern: a frugal cruising mode with moderate QoS and a 30 s interval in calm conditions, and a more aggressive mode with shorter intervals and richer payloads when backlog, anomalies, or link problems signalled higher risk. This fits with the PASA idea of escalating service only when necessary.

The online experiments on the live CoAP–MQTT setup confirmed some of these trends but not all of them. Cumulative return improved over the first 139 episodes and latency remained stable, but the on-time delivery rate in the latest window decreased and the actions did not yet concentrate into the same two-mode pattern. This means that the online controller is still exploring and has not fully converged. In addition, the online energy results are not physically meaningful because a very small virtual battery (5 mAh), interrupted runs, and inconsistent episode lengths produced unrealistically high lifetime gains. These online energy figures should therefore be treated as provisional.

The study has several important limitations. The reward is a scalarised multi-objective heuristic: a single hand-crafted score that combines information value, timeliness, reliability, energy cost, backlog pressure, and safety terms. Fixed weights and penalties only represent one specific point on the trade-off between energy, delay, and reliability, instead of providing a complete solution that considers all these factors together. The three-layer safety structure (hard masks, soft gating, and non-bypassable penalties) effectively prevents unsafe actions, but it also injects strong rule-based structure and limits the ability to find completely new ways to act on its own. The environment is simplified: synthetic temperature traces, simple link models, a single sensor stream, and a small discrete action set.

In general, the results indicate that using safe and priority-aware RL for IoT communication is feasible in this setting. The offline experiments demonstrate simultaneous gains in energy, reliability, and delay under the chosen scalarisation and guardrails, while the online experiments reveal the challenges of small batteries, limited horizons, and imperfect reward calibration due to computational resource constraints. Future work should aim for better realistic online evaluation, larger and more stable battery models, richer multi-device scenarios, and more principled multi-objective or limits in RL formulations instead of using a single scalar reward.

References

1. Victor Seoane, C. G.-R. (2021). Performance evaluation of CoAP and MQTT with security support for IoT environments. 197. DOI: https://doi.org/10.1016/j.comnet.2021.108338
2. Zanella, N. B. (2014, February). Internet of Things for Smart Cities. 1. IEEE IoT Journal. DOI: https://doi.org/10.1109/JIOT.2014.230632
3. Shelby, Z. H. (2014). The Constrained Application Protocol (CoAP). RFC 7252. Internet Engineering Task Force (IETF) RFC-7252. DOI: http://dx.doi.org/10.17487/RFC7252
4. Andrew Banks, R. G. (2014). MQTT Version 3.1.1 Errata 01. OASIS Standard. Retrieved from https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html
5. Bandyopadhyay, D. a. (2011). Internet of Things—Applications and Challenges in Technology and Standardization. 58. Wireless Personal Communications. DOI: https://doi.org/10.1007/s11277-011-0288-5
6. Muhammad Adeel Mahmood, W. K. (2015, January). Reliability in wireless sensor networks: A survey and challenges ahead. Computer Networks, 79. DOI: https://doi.org/10.1016/j.comnet.2014.12.016
7. Mnih, V. K. (2015). Human-level control through deep reinforcement learning. 518 (7540). DOI: https://doi.org/10.1038/nature14236
8. O. Bouhamed, O. B. (2021). Lightweight IDS For UAV Networks: A Periodic Deep Reinforcement Learning-based Approach. IFIP/IEEE International Symposium on Integrated Network Management (IM). https://ieeexplore.ieee.org/document/9463947
9. S. M. Aslam, A. K. (2021). Feature Evaluation of Emerging E-Learning Systems Using Machine Learning: An Extensive Survey. 9, pp. 69573-69587. DOI: https://doi.org/10.1109/ACCESS.2021.3077663
10. Erdal ÖZDOĞAN, O. A. (2024). Adaptive Hybrid Application Protocol for IoT. 21. Acta Polytechnica Hungarica. doi: https://doi.org/10.12700/APH.21.2.2024.2.15
11. M.Seetharama Prasad, 2025. Efficient Power Management in IoT Communication Protocols.Volume 7, Issue 3, ISSN - 2456-1142. DOI: https://doi.org/10.21590/ijhit.07.03.02
12. Cenk Gündoğan, Peter Kietzmann, Martine Lenders, Hauke Petersen, Thomas C. Schmidt, Matthias Wählisch,21 September 2018, NDN, CoAP, and MQTT: a comparative measurement study in the IoT, ICN '18: Proceedings of the 5th ACM Conference on Information-Centric Networking, Pages 159 - 171, DOI: https://doi.org/10.1145/3267955.3267967
13. E. Palavras, K. Fysarakis, I. Papaefstathiou and I. Askoxylakis, "SeMIBIoT: Secure Multi-Protocol Integration Bridge for the IoT," 2018 IEEE International Conference on

Communications (ICC), Kansas City, MO, USA, 2018, pp. 1-7, DOI: - https://doi.org/10.1109/ICC.2018.8422486

14. C. Innamorati, A. E. C. Redondi and M. Cesana, Designing a Broker Extension for Seamless CoAP and MQTT Interoperability, 2024 IEEE 8th Forum on Research and Technologies for Society and Industry Innovation (RTSI), Milano, Italy, 2024, pp. 437-442, DOI: https://doi.org/10.1109/RTSI61910.2024.10761826 .
15. N. Naik, Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP, 2017 IEEE International Systems Engineering Symposium (ISSE), Vienna, Austria, 2017, pp. 1-7, DOI: https://doi.org/10.1109/SysEng.2017.8088251 .
16. Chanhyuk Lee, Jisoo Kim, Heedong Ko, Byounghyun Yoo, Addressing IoT storage constraints: A hybrid architecture for decentralized data storage and centralized management,Volume 25, April 2024,Internet of Things, ISSN 2542-6605, DOI: https://doi.org/10.1016/j.iot.2023.101014
17. Zhang H, Zhang H, Wang Z, et al. Delay-reliability-aware protocol adaption and quality of service guarantee for message queuing telemetry transport-empowered electric Internet of things. International Journal of Distributed Sensor Networks. 2022;18(5). DOI: https://doi.org/10.1177/15501329221097815
18. Anbazhagan, S., Mugelan, R.K. Adaptive power-saving mode control in NB-IoT networks using soft actor-critic reinforcement learning for optimal power management. Sci Rep 15, 34618 (2025). DOI: https://doi.org/10.1038/s41598-025-18214-4
19. Akshatha, P., Divyashree, S. & Dilip Kumar, S. Priority-enabled MQTT: a robust approach to emergency event messaging. J. Eng. Appl. Sci. 71, 67 (2024). DOI: https://doi.org/10.1186/s44147-024-00400-2
20. Harm van Seijen, Mehdi Fatemi, Joshua Romoff, Romain Laroche, Tavian Barnes, Jeffrey Tsang, Hybrid Reward Architecture for Reinforcement Learning, 2017, arxiv, DOI: https://doi.org/10.48550/arXiv.1706.04208
21. Suseendran, K. K. (2019). Priority based Adaptive Scheduling Algorithm for IoT Sensor Systems. DOI: https://doi.org/10.1109/ICACTM.2019.8776691 .
22. M. C. Pognon, A. Quintero and S. Pierre, "Adaptive Priority Scheduling of Internet of Things Data for Disaster Management in Smart Cities," in IEEE Access, vol. 12, pp. 83285-83298, 2024, DOI: https://doi.org/10.1109/ACCESS.2024.3407672 .
23. Kumar, S., Sharma, S.C. (2022). Two-Level Priority Task Scheduling Algorithm for Real-Time IoT Based Storage Condition Assessment System. In: Rathore, V.S., Sharma, S.C., Tavares, J.M.R., Moreira, C., Surendiran, B. (eds) Rising Threats in Expert Applications and Solutions. Lecture Notes in Networks and Systems, vol 434. Springer, Singapore. DOI: https://doi.org/10.1007/978-981-19-1122-4_17

24. Mohammed Alshiekh, Roderick Bloem, Ruediger Ehlers, Bettina Könighofer, Scott Niekum, Ufuk Topcu, Safe Reinforcement Learning via Shielding, September 2017, arxiv, DOI: https://doi.org/10.48550/arXiv.1708.08611
25. Haonan Yu, Wei Xu, Haichao Zhang,Towards Safe Reinforcement Learning with a Safety Editor Policy, January 2022, DOI: https://doi.org/10.48550/arXiv.2201.12427
26. Joshua Achiam, David Held, Aviv Tamar, Pieter Abbeel, Constrained Policy Optimization, May 2017, arxiv, DOI: https://doi.org/10.48550/arXiv.1705.10528
27. Geraud Nangue Tasse, Tamlin Love, Mark Nemecek, Steven James, Benjamin Rosman, ROSARL: Reward-only Safe Reinforcement Learning, May 2023, arxiv, DOI: https://doi.org/10.48550/arXiv.2306.00035
28. Derhamy, H. (2016). Towards Interoperable Industrial Internet of Things: An On-Demand Multi-Protocol Translator Service. Luleå University of Technology. https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A971142&dswid=8426
29. Lee, C.-H., Chang, Y.-W., Chuang, C.-C., & Lai, Y. H. (2016). Interoperability enhancement for Internet of Things protocols based on software-defined network. IEEE 5th Global Conference on Consumer Electronics. DOI: https://doi.org/10.1109/GCCE.2016.7800510 .
30. Donta, P. S. (2020). iCoCoA: intelligent congestion control algorithm for CoAP using deep reinforcement learning. 14. J Ambient Intell Human Comput. DOI: https://doi.org/10.1007/s12652-023-04534-8
31. Sun, G. W. (2025, April). Context-Aware Adaptive Sampling for Intelligent Data Acquisition Systems Using DQN. arXiv. Retrieved from https://arxiv.org/pdf/2504.09344
32. M. J. Gaber, A. F.-A. (2025, April). Reinforcement Learning for Adaptive Sampling and Filtering in Real-Time Big Data Streams. ResearchGate . Retrieved from https://www.researchgate.net/publication/390520144
33. González, S. &.-L. (2023). Comparative Analysis of Power Consumption between MQTT and HTTP Protocols in an IoT Platform Designed and Implemented for Remote Real-Time Monitoring of Long-Term Cold Chain Transport Operations. 23(10). DOI: https://doi.org/10.3390/s23104896
34. D. Thangavel, X. M.-X.-Y. (2014). Performance evaluation of MQTT and CoAP via a common middleware. (pp. 1-6). 2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP). DOI: https://doi.org/10.1109/ISSNIP.2014.6827678
35. Suseendran, K. K. (2019). Priority based Adaptive Scheduling Algorithm for IoT Sensor Systems. DOI: https://doi.org/10.1109/ICACTM.2019.8776691 .
36. Parisa Rezaei, N. D. (2025). Reinforcement learning based routing in delay tolerant networks. 31. doi: https://doi.org/10.1007/s11276-025-03911-9

37. W. Huang, J. Z. (2025, April). Context-Aware Adaptive Sampling for Intelligent Data Acquisition Systems Using Dqn. Shenzhen, China: IEEE 6th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT). DOI: https://doi.org/10.1109/AINIT65432.2025.11035407 .
38. S. Shresthamali, M. K. (2021). Multi-objective Reinforcement Learning for Energy Harvesting Wireless Sensor Nodes. (pp. 98-105). 2021 IEEE 14th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC). DOI: https://doi.org/10.1109/MCSoC51149.2021.00022
39. Matteo Hessel, J. M. (2017). Rainbow: Combining Improvements in Deep Reinforcement Learning. arxiv. DOI: https://doi.org/10.48550/arXiv.1710.02298
40. Can Jin, Y. Z.-W. (2025). Your Reward Function for RL is Your Best PRM for Search: Unifying RL and Search-Based TTS. DOI: https://doi.org/10.48550/arXiv.2508.14313
41. T. C. Jermin Jeaunita, V. S. (2021). A Multi-Agent Reinforcement Learning-Based Optimized Routing for QoS in IoT. 21(4). DOI: https://doi.org/10.2478/cait-2021-0042
42. Ziyu Wang, T. S. (2016). Dueling Network Architectures for Deep Reinforcement Learning. DOI: https://doi.org/10.48550/arXiv.1511.06581
43. Shengyi Huang, Santiago Ontañón, A Closer Look at Invalid Action Masking in Policy Gradient Algorithms, Jun 2020, Flairs vol.35, DOI: https://doi.org/10.32473/flairs.v35i.130584
44. Hongli Dai, Yu Xia, Jing Mao, Cheng Xu, Wei Liu, Peukert's Law-Based State-of-Charge Estimation for Primary Battery Powered Sensor Nodes, Sensors 2023, 23(2), 1036; DOI: https://doi.org/10.3390/s23021036
45. Kovtun, V. Decentralized queue control with delay shifting in edge-IoT using reinforcement learning. Sci Rep 15, 30845 (2025).DOI: https://doi.org/10.1038/s41598-025-13983-4
46. Shivang Shrivastav, DDPG: DQN Re-engineered for Continuous Action Spaces, March 2025, Medium, URL: https://shivang-ahd.medium.com/ddpg-dqn-re-engineered-for-continuous-action-spaces-3e536ffd287b
47. Das, R. (2025). Enhancing Health and Environmental Monitoring Open-Source IoT-Enabled SCADA System with Node-RED, InfluxDB, Grafana, and Raspberry Pi. researchGate. Retrieved from https://www.researchgate.net/publication/389041469_Enhancing_Health_and_Environmental_Monitoring_Open-Source_IoT-Enabled_SCADA_System_with_Node-RED_InfluxDB_Grafana_and_Raspberry_Pi
48. Y. Wan, K. Xu, F. Wang and G. Xue, "IoTMosaic: Inferring User Activities from IoT Network Traffic in Smart Homes," IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, London, United Kingdom, 2022, pp. 370-379, DOI: https://doi.org/10.1109/INFOCOM48880.2022.9796908 .

49. Zach Shelby, Constrained RESTful Environments (CoRE) Link Format, August 2012, Internet Engineering Task Force (IETF), ISSN: 2070-1721, URL: https://scispace.com/pdf/constrained-restful-environments-core-link-format-12r32q3fcq.pdf
50. P. Parsch and A. Masrur, "A Reliable MAC for Energy-Efficient WSNs in the Era of IoT," 2016 Euromicro Conference on Digital System Design (DSD), Limassol, Cyprus, 2016, pp. 309-317, DOI: https://doi.org/10.1109/DSD.2016.55 .
51. Dominik Baumann, Jia-Jie Zhu, Georg Martius, and Sebastian Trimpe," Deep Reinforcement Learning for Event-Triggered Control", September 2018, Arxiv, DOI: https://doi.org/10.48550/arXiv.1809.05152
52. Manuel Lopez-Martin, Belen Carro, Antonio Sanchez-Esguevillas, "Application of deep reinforcement learning to intrusion detection for supervised problems", Volume 141, March 2020, 112963, ISSN 0957-4174, DOI: https://doi.org/10.1016/j.eswa.2019.112963

Appendix A — Algorithms and Pseudocode

1. **Code reference:**

   GitHub link= https://github.com/ronyahammad/coap_mqtt_integration

2. **Safe Action Selection (mask → gate → fallback → selection)**

Masks encode the hard safety rules; soft gating uses the learned criticality to bias the feasible set; if nothing remains, pick a context-safe default; finally, select via masked ε-greedy (or NoisyNet noise when enabled).

```
function SAFE_SELECT_ACTION(state s, Q_net, Crit_net, A_all, ε, training):
 # ----- Hard masks (state-only, reward-independent)
 M = []                          # indices of feasible actions
 for i, a=(q, τ, f, c) in enumerate(A_all):
    urgent = (s.bufT==2) or (s.dT==2) or (s.anom==1)
    very_bad_link = max(s.L, s.D)==2
    if urgent and τ==60:          continue          # forbid slow when urgent
    if very_bad_link and q==0:     continue          # forbid QoS0 on bad link
    if s.anom==1 and c==0:         continue          # require c=1 on emergency
     M.append(i)

 # ----- Soft gating (model-based, confidence-aware)
 p = sigmoid(Crit_net(s))                          # p_crit(s)
 θ = current_threshold()                           # schedule/adaptive (§3.6)
 sev = severity_proxy(s)                           # same as trainer
 (θ_hi, θ_lo) = band_around(θ, sev)                # tighter at high severity
 G = M.copy()
 if p >= θ_hi:
    G = [i for i in M if A_all[i].c == 1]          # force c=1 band
 elif p <= θ_lo:
    G = [i for i in M if A_all[i].c == 0]          # force c=0 band
 if not G: G = M                                   # never empty due to gating

 # ----- Safe fallback if masks+gating empty (shouldn't happen, but defensive)
 if not G:
    if urgent or very_bad_link:
       return index_of_action((q=1, τ=5,  f=1, c=1))   # urgent default (safe-aggressive)
    else:
       return index_of_action((q=1, τ=30, f=0, c=0))   # calm default (safe-conservative)

 # ----- Masked selection (ε-greedy over G; NoisyNet still helps ranking)
 if training and rand() < ε:
    return random_choice(G)                        # explore safely
 else:
    # exploit among G only
    q_values = Q_net(s)                            # dueling head
    return argmax_over_subset(q_values, G)

```

3. **Training Loop (experience tuple, PER, 3-step targets, Double-DQN, target update)**

The trainer and the online agent both use DDQN with n-step (n=3), Huber TD loss, a supervised auxiliary head with Focal loss, PER-style sampling with importance weights, gradient clipping, and Polyak target updates. (The trainer adds stratified positives for class imbalance.)

Pseudocode (trainer/agent common core):

```
# ----- data structures -----
Transition = (state s, action_idx a, reward r, next_state s', done d)
Replay.push(Transition, td_error≈seed)        # PER priority on insert

# ----- n-step accumulator (n=3) -----
def store_transition(s, a, r, s', d, true_crit):
    Nbuf.append((s, a, r, s', d))
    if len(Nbuf) < 3: return
    R, sN, dN = 0.0, Nbuf[-1].s', Nbuf[-1].d
    for k, (_, _, rk, _, dk) in enumerate(Nbuf):
        R += (γ**k) * rk
        if dk: dN=True; break
    s0, a0 = Nbuf[0].s, Nbuf[0].a
    seed = 1.0 + 4.0*true_crit                # nudge PER toward positives
    Replay.push((s0, a0, R, sN, dN), td_error=seed)
```

Code reference: online agent `push/flush_nstep` (n=3).

```
def learn_step():
    if len(Replay) < batch_size: return None
    β = anneal_beta(step, β0=0.4 → βT=1.0, warmup=WARMUP_STEPS)   # PER weights

    batch, idx, w = Replay.sample(batch_size, beta=β)           # PER-simple (online)
    # (trainer uses stratified sampling over current-state positives)

    S  = one_hot(batch.s)            # 25-D input (8×3 one-hots + anom)
    S' = one_hot(batch.s')
    A  = batch.a; R = batch.r; D = batch.d

    # Q online / target
    q_pred, c_logit = Qθ(S)          # dueling trunk + criticality head
    q_sa = gather(q_pred, A)
    with no_grad():
        a*   = argmax(Qθ(S'))        # Double-DQN: next action from online
        q_t' = Qθ-(S')               # target net
        q_n  = gather(q_t', a*)
        y    = R + (1-D) * (γ**3) * q_n

    # losses
    L_TD   = Huber(q_sa, y) weighted by PER w
    y_crit = true_critical(S)        # label from rule in code
    L_crit = FocalLoss(c_logit, y_crit)

    loss = L_TD + λ * L_crit         # λ=1.0 in code

    # update
    opt.zero_grad()
```

```
    loss.backward()
    clip_grad_norm_(Qθ.params, 1.0)
    opt.step()

    # PER priorities + soft target update
    td = |q_sa - y|
    Replay.update_priorities(idx, td)
    θ- ← (1-τ)·θ- + τ·θ            # Polyak; τ=0.01

    update_epsilon()
    return loss

```

4. Reward Computation Service (call signature, scaling/clipping, edge cases)

Call signature (offline and online share the same interface):

```
r = AdaptiveRewardCalculator.calculate_reward(
      state: Tuple[int,int,int,int,int,int,int,int,int],  # 9 indicators (§3.1)
      action: Tuple[int,int,int,int],                    # (q, τ, f, c)
      energy_consumed: float,                            # mAh (from telemetry)
      true_crit: int)                                    # {0,1} rule-based label

```

5. Minimal training loop (episodes/steps) and online loop (for completeness)

```
# ----- Offline trainer (700 episodes × up to 250 steps) -----
for ep in range(700):                                  # §B states 700 offline
    s = (0,0,2,0,0,1,1,1,0)
    for t in range(250):
        a_idx = SAFE_SELECT_ACTION(s, ...)
        a = index_to_action(a_idx)
        s', pkt, true_crit, e_used, latency = env.step(s, a)  # simulator
        r = Reward.calculate_reward(s, a, e_used, true_crit)
        done = env.battery_empty()
        store_transition(s, a_idx, r, s', done, true_crit)    # n-step buffer
        learn_step()
        s = s'
        if done: break
    flush_nstep()

```

Code reference: *train_dqn* loop mirrors this structure;

```
# ----- Online agent loop (139 episodes to date; continuous) -----
while True:
    d = read_telemetry()                               # MQTT ddqn topic
    s' = build_state_from_payload(d)
    e_used = d.energy_mAh_used; true_crit = rule_label(s')    # same rubric
    if have_prev:
        r = Reward.calculate_reward(s_prev, a_prev, e_used, true_crit)
        store_transition(s_prev, a_prev_idx, r, s', done=low_battery(d), true_crit=true_crit)
        learn_step()
    a_idx = SAFE_SELECT_ACTION(s', ...)
```

```
    publish_control(A[a_idx])                    # MQTT control topic
    s_prev, a_prev_idx = s', a_idx

```

Code reference: *PolicyRunner.run()* and surrounding helpers.

6. Pseudocode of Learning Algorithm

```
# Inputs: Q(s,·), pcrit = p_crit(s), band_prev, thresholds (θ_lo, θ_hi)
# Outputs: action a_t, band_curr

band_curr = HIGH if pcrit >= θ_hi else LOW if pcrit <= θ_lo else band_prev

A_allow = []
for a in A:
   if M_hard(s,a) == 0:
      continue
   if band_curr == HIGH and c(a) != 1:
      continue
   if band_curr == LOW  and c(a) != 0:
      continue
   A_allow.append(a)

if len(A_allow) == 0:
   a_t = a_safe(s)
else:
   # ε-greedy within A_allow (or masked-argmax)
   if rand() < ε:
      a_t = random_choice(A_allow)
   else:
      a_t = argmax_a_in(A_allow) Q(s,a)

```

Appendix B —Policy Probes

- **Policy probes (state → learned vs. "ideal")**

This appendix includes a set of test states and simple printed output format from the offline simulation.

Probe states for the test (excerpt, fixed list in code):

```
1. (0,0,2,0,0,1,1,1,0),
2. (1,0,2,0,0,1,0,0,0),
3. (1,1,2,1,0,2,1,1,0),
4. (2,0,2,0,0,0,2,2,0),
5. (0,2,2,1,1,1,1,0,0),
6. (1,1,0,2,2,1,2,1,0),
7. (2,2,2,0,0,0,2,2,1),
```

(These are exactly the tuples used by $_evaluate_policy(\cdot)$ and will reproduce the printed comparison and Q-tops). Each state represents:

- the action that the agent actually learned,
- the ideal action based on the current reward,
- the reward point for that state, and
- the top three Q-values from the network, with a clear indication when the learned policy is the best choice.

All of this is based on the offline simulation. It is a quick way to check if the agent is making good choices based on the reward it gets. To use it, the evaluator is run after training. It looks at the stored replay data, re-calculates the criticality threshold (using the best F-score or similar tuning), and then prints the table for these probe states. This provides a quick, easy-to-understand report showing that learning matches the intended objective.

The Below is printed Double DQN Policy Analysis (T-only) results:

```
1. Double DQN Policy Analysis (T-only):
2.
3. State (ΔT=0, buf_T=0, B=2, L=0, D=0, anomT=0):
4.   Learned action: QoS=0, Interval=60s, Fidelity=0, Critical=0
5.   Ideal action:   QoS=0, Interval=60s, Fidelity=0, Critical=0
6.   Reward: 0.25, Critical match: ✓ (True: 0)
7.   Top 1: QoS=0, τ=60s, Fidelity=0, Critical=0 (IDEAL) (Q=4.78)
8.   Top 2: QoS=2, τ=60s, Fidelity=1, Critical=0 (Q=4.67)
```

```
 9.   Top 3: QoS=0, τ=5s, Fidelity=2, Critical=0 (Q=4.64)
10.
11. State (ΔT=1, buf_T=0, B=2, L=0, D=0, anomT=0):
12.   Learned action: QoS=1, Interval=5s, Fidelity=0, Critical=0
13.   Ideal action:   QoS=0, Interval=5s, Fidelity=2, Critical=0
14.   Reward: 0.11, Critical match: ✓ (True: 0)
15.   Top 1: QoS=0, τ=60s, Fidelity=0, Critical=0 (Q=3.98)
16.   Top 2: QoS=1, τ=5s, Fidelity=0, Critical=0 (Q=3.97)
17.   Top 3: QoS=2, τ=60s, Fidelity=1, Critical=0 (Q=3.94)
18.
19. State (ΔT=1, buf_T=1, B=2, L=1, D=0, anomT=0):
20.   Learned action: QoS=1, Interval=5s, Fidelity=1, Critical=0
21.   Ideal action:   QoS=1, Interval=5s, Fidelity=2, Critical=0
22.   Reward: 0.14, Critical match: ✓ (True: 0)
23.   Top 1: QoS=1, τ=5s, Fidelity=1, Critical=0 (Q=3.90)
24.   Top 2: QoS=2, τ=5s, Fidelity=2, Critical=0 (Q=3.90)
25.   Top 3: QoS=1, τ=5s, Fidelity=0, Critical=0 (Q=3.87)
26.
27. State (ΔT=2, buf_T=0, B=2, L=0, D=0, anomT=0):
28.   Learned action: QoS=0, Interval=5s, Fidelity=2, Critical=0
29.   Ideal action:   QoS=0, Interval=5s, Fidelity=2, Critical=0
30.   Reward: 0.24, Critical match: ✓ (True: 0)
31.   Top 1: QoS=0, τ=60s, Fidelity=0, Critical=0 (Q=4.10)
32.   Top 2: QoS=0, τ=5s, Fidelity=2, Critical=0 (IDEAL) (Q=4.08)
33.   Top 3: QoS=2, τ=60s, Fidelity=1, Critical=0 (Q=4.06)
34.
35. State (ΔT=0, buf_T=2, B=2, L=1, D=1, anomT=0):
36.   Learned action: QoS=1, Interval=5s, Fidelity=1, Critical=1
37.   Ideal action:   QoS=1, Interval=5s, Fidelity=2, Critical=1
38.   Reward: 0.32, Critical match: ✓ (True: 1)
39.   Top 1: QoS=0, τ=60s, Fidelity=0, Critical=0 (Q=4.37)
40.   Top 2: QoS=1, τ=5s, Fidelity=1, Critical=0 (Q=4.29)
41.   Top 3: QoS=2, τ=30s, Fidelity=0, Critical=0 (Q=4.25)
42.
43. State (ΔT=1, buf_T=1, B=0, L=2, D=2, anomT=0):
44.   Learned action: QoS=2, Interval=30s, Fidelity=0, Critical=1
45.   Ideal action:   QoS=2, Interval=5s, Fidelity=0, Critical=1
46.   Reward: 0.29, Critical match: ✓ (True: 1)
47.   Top 1: QoS=2, τ=30s, Fidelity=0, Critical=1 (Q=3.78)
48.   Top 2: QoS=2, τ=30s, Fidelity=2, Critical=1 (Q=3.67)
49.   Top 3: QoS=2, τ=60s, Fidelity=0, Critical=1 (Q=3.63)
50.
51. State (ΔT=2, buf_T=2, B=2, L=0, D=0, anomT=1):
52.   Learned action: QoS=1, Interval=5s, Fidelity=1, Critical=1
53.   Ideal action:   QoS=0, Interval=5s, Fidelity=2, Critical=1
54.   Reward: 0.60, Critical match: ✓ (True: 1)
55.   Top 1: QoS=0, τ=5s, Fidelity=2, Critical=0 (Q=3.92)
56.   Top 2: QoS=2, τ=5s, Fidelity=2, Critical=0 (Q=3.90)
57.   Top 3: QoS=0, τ=30s, Fidelity=0, Critical=0 (Q=3.90)
58.
59. State (ΔT=1, buf_T=1, B=1, L=1, D=1, anomT=1):
60.   Learned action: QoS=2, Interval=30s, Fidelity=0, Critical=1
61.   Ideal action:   QoS=1, Interval=5s, Fidelity=2, Critical=1
62.   Reward: -0.10, Critical match: ✓ (True: 1)
63.   Top 1: QoS=1, τ=5s, Fidelity=1, Critical=0 (Q=3.64)
64.   Top 2: QoS=0, τ=60s, Fidelity=0, Critical=0 (Q=3.63)
65.   Top 3: QoS=0, τ=30s, Fidelity=0, Critical=0 (Q=3.58)
66.
67. State (ΔT=0, buf_T=2, B=1, L=1, D=1, anomT=0):
```

```
68.   Learned action: QoS=1, Interval=5s, Fidelity=1, Critical=1
69.   Ideal action:   QoS=1, Interval=5s, Fidelity=2, Critical=1
70.   Reward: 0.32, Critical match: ✓ (True: 1)
71.   Top 1: QoS=0, τ=60s, Fidelity=0, Critical=0 (Q=4.11)
72.   Top 2: QoS=1, τ=5s, Fidelity=1, Critical=0 (Q=4.03)
73.   Top 3: QoS=0, τ=30s, Fidelity=0, Critical=0 (Q=4.00)
74.
75. State (ΔT=2, buf_T=0, B=1, L=0, D=1, anomT=1):
76.   Learned action: QoS=2, Interval=30s, Fidelity=0, Critical=1
77.   Ideal action:   QoS=1, Interval=5s, Fidelity=2, Critical=1
78.   Reward: -0.06, Critical match: ✓ (True: 1)
79.   Top 1: QoS=0, τ=30s, Fidelity=0, Critical=0 (Q=3.82)
80.   Top 2: QoS=0, τ=60s, Fidelity=0, Critical=0 (Q=3.81)
81.   Top 3: QoS=0, τ=5s, Fidelity=2, Critical=0 (Q=3.80)
82.
83. Coverage quick view: pairs_seen=6858 unique_states=167
84.
85. Tuning guardrails applied:
86.    [Guardrails] Precision low → F2 sweep window 0.25-0.45 -> 0.30-0.40
87.
```